

CHAPTER III

TOP LEVEL VIEW OF

COMPUTER FUNCTION



2 INTRODUCTION

Today we will focus on presenting a **top-level view of computer function**.

- Main components of computer and their function?
- How are these components organized?
- How do these components communicate with each other?

3 INTRODUCTION

- At a top level, a computer consists of:
 - Central Processing Unit;
 - Memory;
 - I/O components.
- The components are **interconnected** in order to execute programs.

4 COMPUTER COMPONENTS



Does anyone know who this person is?

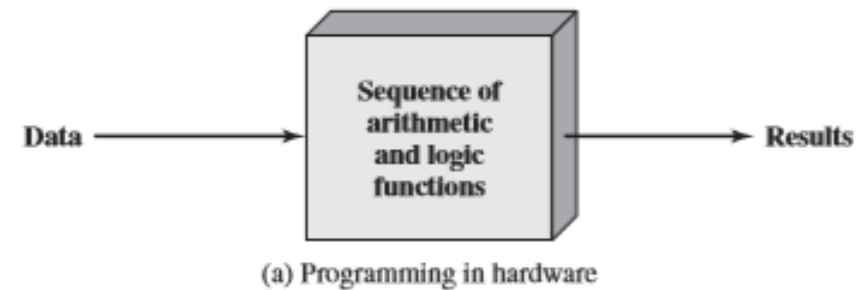
5 COMPUTER COMPONENTS

- Most computer concepts were developed **by John von Neumann**:
- Von Neumann architecture is based on **three** concepts:
 - Ability to **write/read data** from **memory**;
 - Ability to **execute** instruction **sequentially**;
 - Ability to **process inputs/outputs**;

6

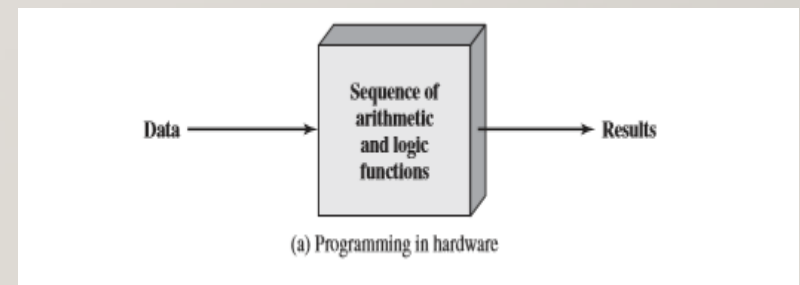
COMPUTER COMPONENTS

- Consider the original method for computation based on digital circuits:
 - Employs a small set of logical gates:
 - AND, OR, NOT, NAND, NOR, XOR, XNOR
 - Employ some type of memory:
 - Flip-flop SR, JK, D



7 DISADVANTAGE WITH USING DIGITAL CIRCUITS...

- Rigid design (hardwired program):
 - Only works for a specific function...
- What if we need to calculate additional functions?
 - Requires generating a new circuit...
 - producing new truth tables;
 - algebraic simplification;
 - circuit design.
 - Basically a lot of work...

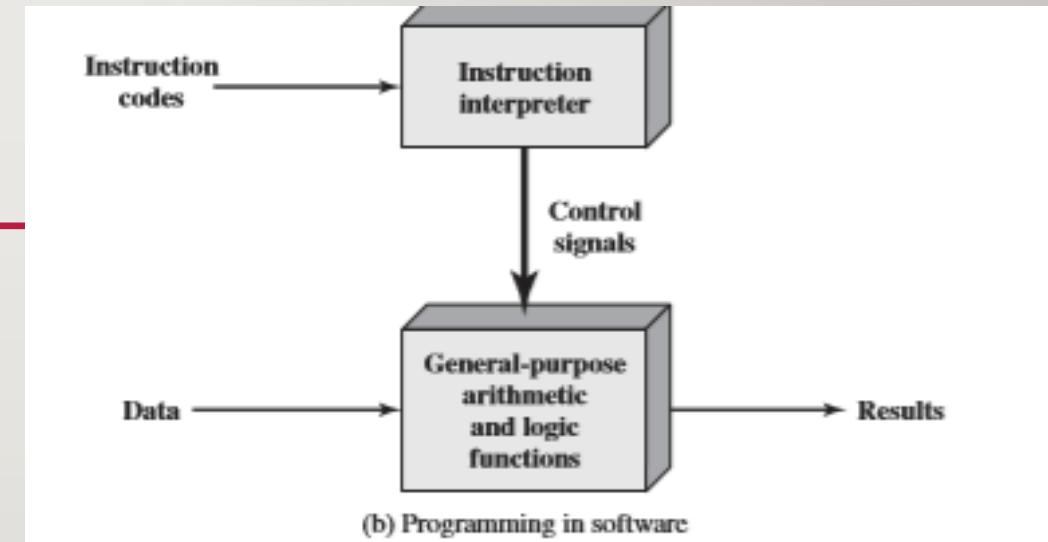


8 SOLUTION...

- Suppose that instead of having such an hardwired circuit we have:
- A module capable of calculating **arithmetic/logic functions**:
 - **Logic functions**:
 - AND, OR, NOT, NAND, NOR, XOR, XNOR
 - **Arithmetic functions**:
 - addition, subtraction, multiplication, division, SHL, SHR,...

9 SOLUTION...

- We also need a way to **control** such a module:
 - Hardware will perform various functions on data depending on **control signals applied** to the hardware
 - The system accepts **data** and **control** signals and produces **results**.
 - Thus, **instead of rewiring** the hardware for each new program, the **programmer** merely needs to supply a new **set of control signals** → **General Purpose computer**



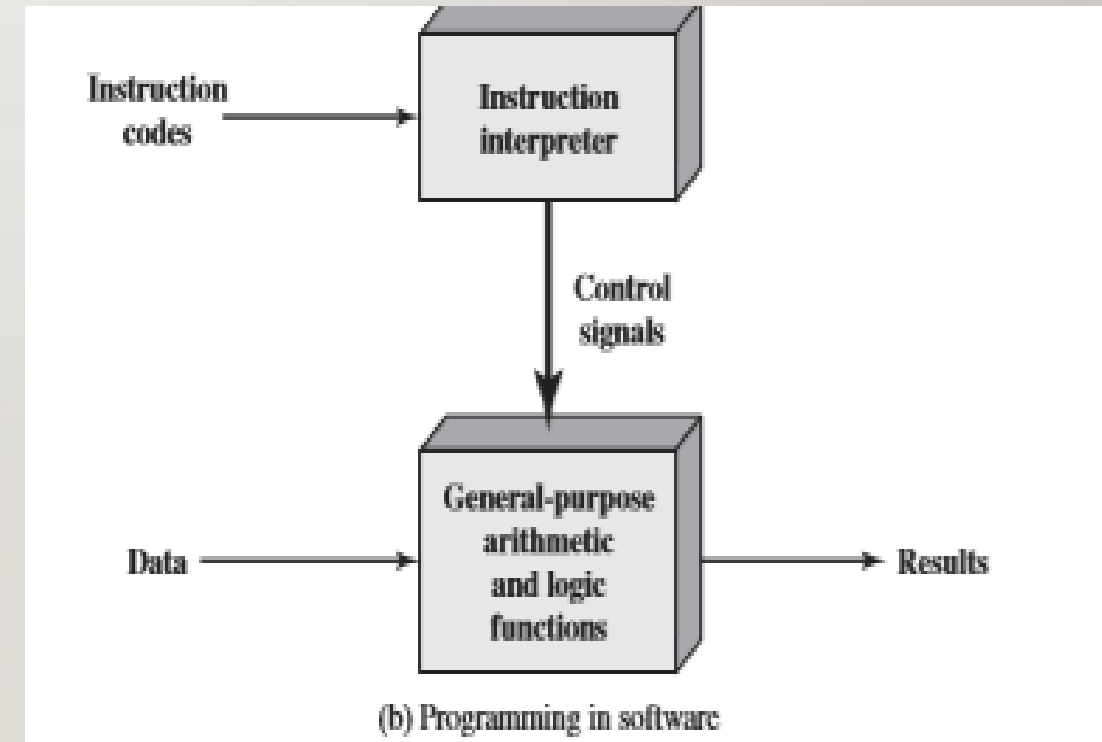
How shall the control signals be supplied?

10 ANSWER...

- Program is a **sequence of steps**;
 - At each step, an arithmetic/logical operation is performed on **data**;
 - Each **instruction** thus requires its own set of **control signals**
 - **Hardware interprets** each **instruction** and **generates** control signals

II COMPUTER COMPONENTS

- An instruction **interpreter** capable of generating **control signals**;
- A **general-purpose** module for arithmetic/logic functions.
 - These two constitute the CPU
- *Is this capability enough to have a functional computer*



12 COMPUTER COMPONENTS

- **Data and instructions** must be provided to the system:
 - **Input module:**
 - contains basic components for accepting data and instructions
 - **Output module:**
 - contains the means of reporting results.
- Taken together, these are referred to as **I/O components**.
- Are these **two** components enough for a **functional computer**?

I3 COMPUTER COMPONENTS

- **Data operations** may require access to more than one element:
 - there must be a place to store both **instructions** and **data**.
 - that module is called **main memory**
 - to distinguish it from **external storage**

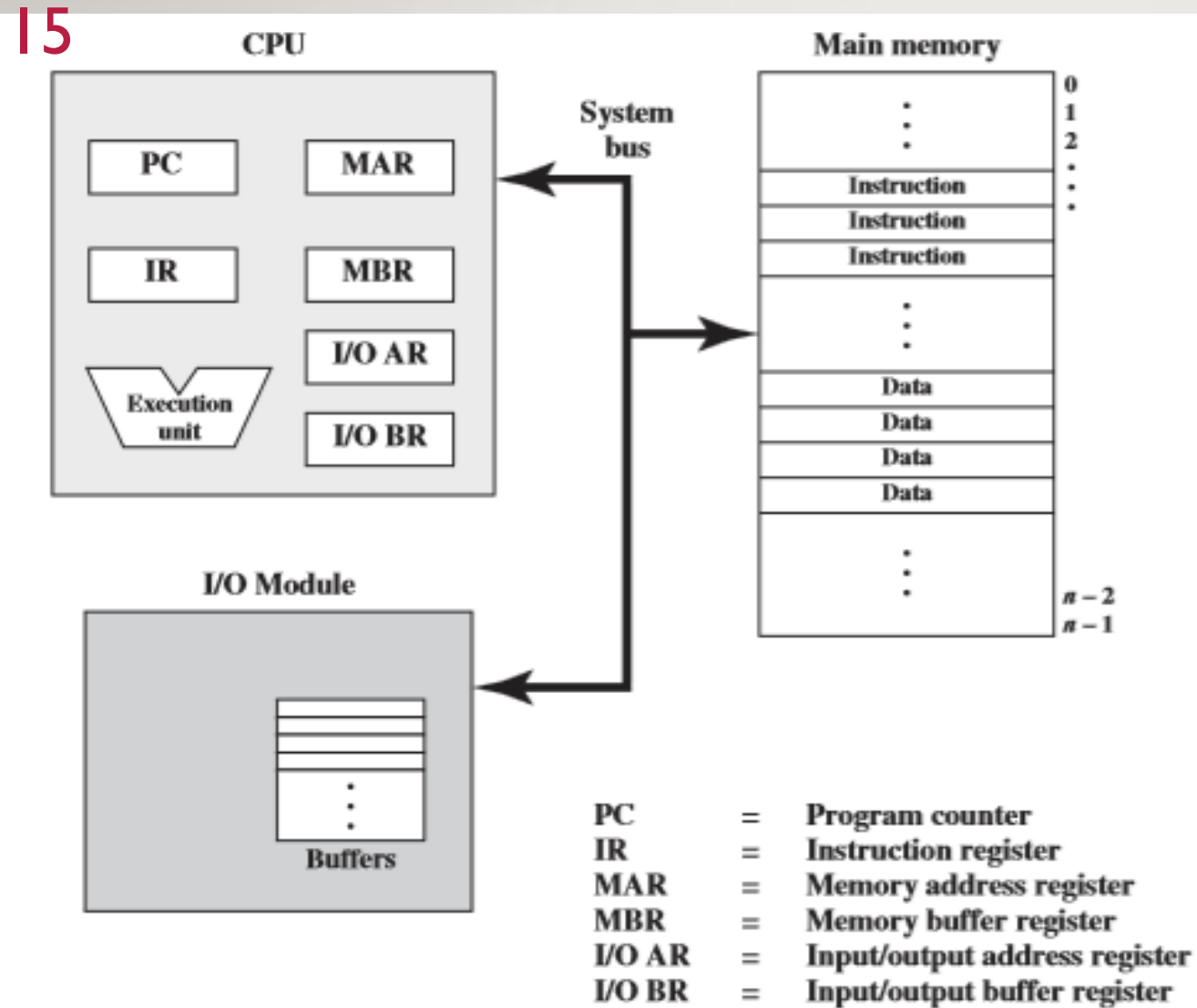
I4 COMPUTER COMPONENTS

- We have thus the main components of the von Neumann architecture:
 - **Memory module**
 - **I/O module**
 - **CPU**

We will be having a more detailed look at these components

COMPUTER COMPONENTS

A top Level view of the main computer components (Source Stallings)



16 COMPUTER COMPONENTS -CPU

- CPU has a set of internal registers :
- **Program Counter (PC):**
 - specifies the memory address of the next instruction to be executed.
- **Instruction Register (IR):**
 - holds the instruction currently being executed or decoded.

17 COMPUTER COMPONENTS -CPU

- **memory address register (MAR):**
 - specifies memory address to be read/written;
- **memory buffer register (MBR):**
 - contains the data to be written into memory or...
 - receives the data read from memory;

18 COMPUTER COMPONENTS - CPU

- **I/O address register (I/OAR):**
 - specifies a particular I/O device;
- **I/O buffer (I/OBR) register:**
 - used for the exchange of data between an I/O module and the CPU;

19 COMPUTER COMPONENTS -MEMORY

- Memory module consists of:
 - Set of **sequentially numbered addresses**;
 - Each location contains **binary information** (word);
 - **Data**;
 - Or **instructions**.

20 COMPUTER COMPONENTS – I/O

- I/O module responsible for:
 - Transfers data from **external devices** to **CPU** and **memory**;
 - and **vice versa**
 - Containing **internal buffers** for temporarily holding data

how do these components function together to execute programs?

21 COMPUTER FUNCTION

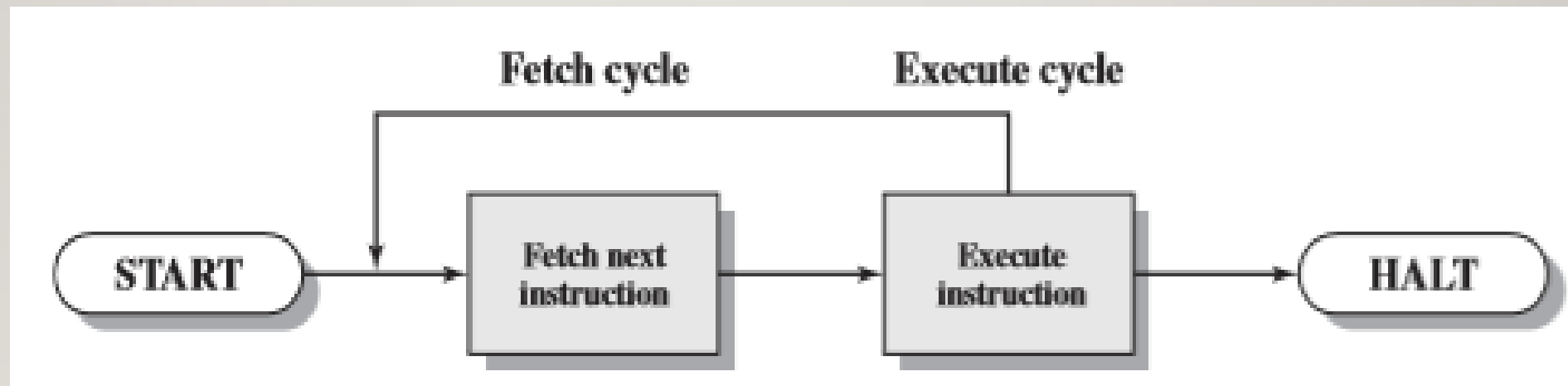
- Basic function **performed** by a computer is **execution** of a program:
 - Consisting of a set of **instructions stored** in **memory**;
 - **Processor** does the actual work by **executing** the specified **instructions**;
- This gives us a hint of how to perform **instruction processing...**

22 BASIC INSTRUCTION CYCLE

- In its simplest form, instruction processing consists of two steps:
- **Fetch stage:**
 - processor **reads** instructions from memory one at a time
- **Execution stage:**
 - processor **executes** the instruction

23 BASIC INSTRUCTION CYCLE

- Program execution consists of a **loop**:
- 1 Fetch instruction stage;
- 2 Execute instruction stage



24 BASIC INSTRUCTION CYCLE

- At the beginning of each instruction cycle:
 - **Program counter** holds the **address** of the instruction to be fetched;
 - Processor **fetches** an instruction from memory;
 - After each instruction fetch the **PC** is incremented.
 - In order to fetch the memory address of next instruction

25 EXAMPLE

- Consider a computer where each instruction occupies 16-bits in memory.
 - Assume that the **PC** is set to memory location **300**;
 - Each location address contains a **16-bit word**;
 - The processor will next fetch the instruction at location **300**.
 - On succeeding instruction cycles, it will **fetch** instructions from locations:
 - **301**;
 - **302**;
 - **303** and so on.
- This sequence may be altered.

26 BASIC INSTRUCTION CYCLE

- The fetched instruction is loaded into a **register** in the processor
 - Known as the **instruction register (IR)**;
 - Instruction **specifies control signals** for the **processor**;
 - Processor **interprets** the **instruction** and **performs** the required **action**.
 - *What types of actions are usually performed?*

27 BASIC INSTRUCTION CYCLE

- In general, these actions fall into four categories:
 - **Processor** $\leftrightarrow$ **memory** data transfers;
 - **Processor** $\leftrightarrow$ **I/O peripherals** data transfers;
 - **Data processing: arithmetic** or **logic** operations on data;
 - **Control**: E.g. alter the **PC** because of **conditional jumps** (IF's)
 - Let's see those in an example

28 EXAMPLE

- Consider a **processor**:
 - Containing a single data register, called an **accumulator** (AC).
 - Both **instructions** and **data** are **16 bits long**;
 - Convenient to organize memory using **16-bit words**;

29 EXAMPLE

- Instruction format is the following:



(a) Instruction format

- **Opcode: code of the operation to be executed (+,-,×, ÷,etc...);**
 - How many **bits** are **reserved** for the **opcode**? Total combinations?
- **Addressing memory,**
 - How many bits are reserved for the addressing memory? Total combinations?

30 EXAMPLE

- **Opcode: code of the operation to be executed** (+,-,×, ÷,etc...);
 - 4 bits are reserved, i.e there can be 2^4 different opcodes;
- **Addressing memory,**
 - 12 bits are reserved, i.e. 2^{12} memory words can be addressed.

3 | EXAMPLE

- Consider that we have the following opcodes:

0001	Load AC from memory
0010	Store AC to memory
0101	Add to AC from memory

- Also, each data requires 16 bits and has the following representation: 1 bit for sign and 15 bits for magnitude



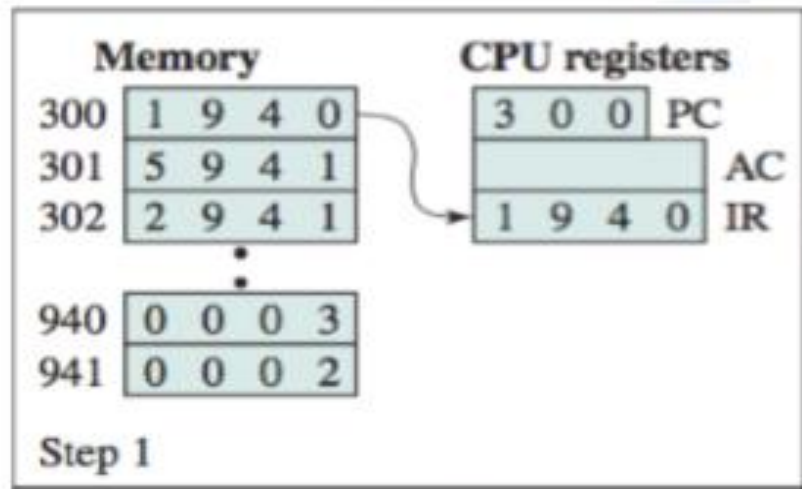
(b) Integer format

32 EXAMPLE

- Now lets try to execute the following instructions (assume PC=**300**):
 - Add **memory contents** at address **940** to the **contents at address 941**;
 - Store the result in the **latter** location.

33 EXAMPLE

Assume **PC=300**:



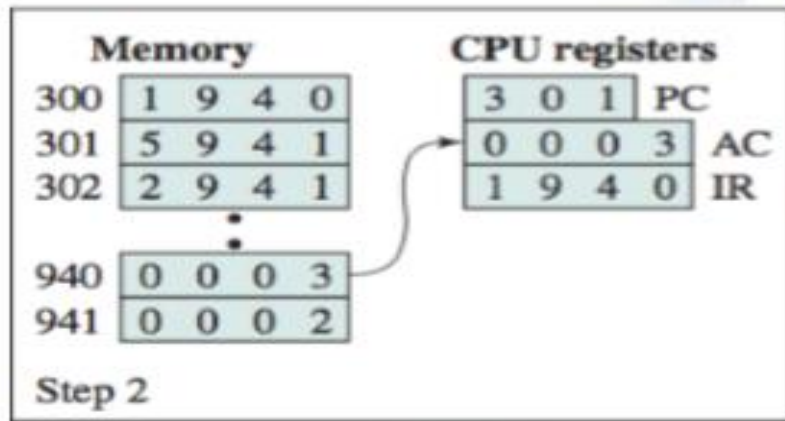
Opcodes:

0001	Load AC from memory
0010	Store AC to memory
0101	Add to AC from memory

Figure: Example of Program Execution (contents of memory and registers in hexadecimal)
(Source: (Stallings, 2015))

34 EXAMPLE

Assume PC=300:



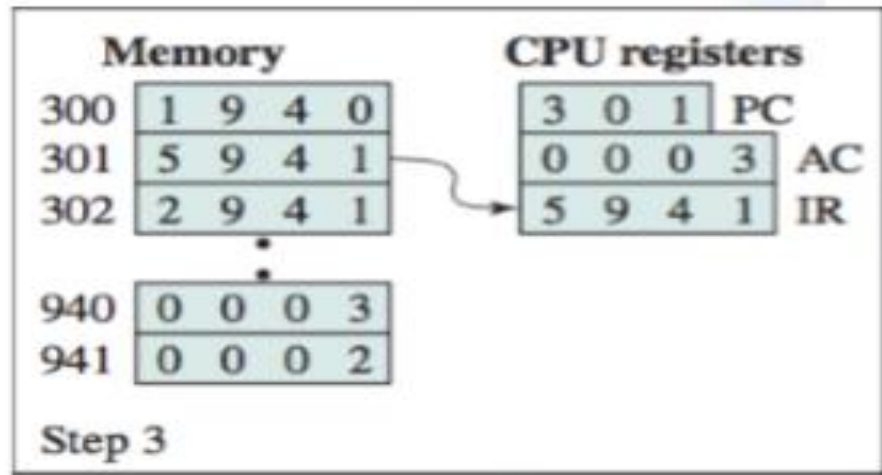
Opcodes:

0001	Load AC from memory
0010	Store AC to memory
0101	Add to AC from memory

Figure: Example of Program Execution (contents of memory and registers in hexadecimal)
(Source: (Stallings, 2015))

35 EXAMPLE

Assume **PC=300**:



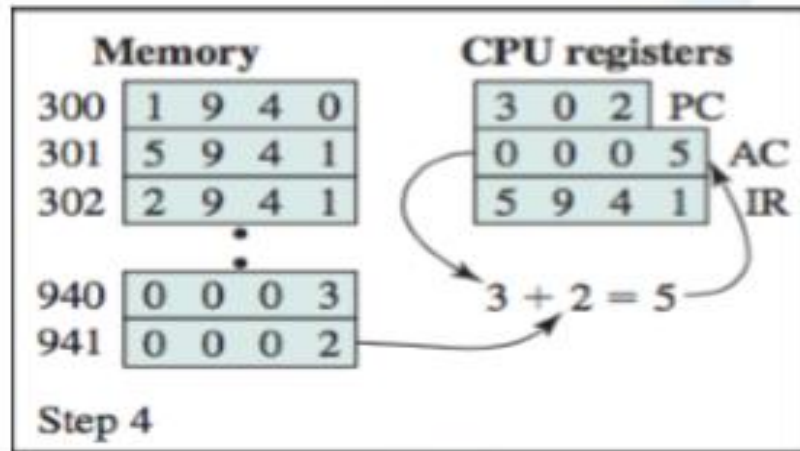
Opcodes:

0001	Load AC from memory
0010	Store AC to memory
0101	Add to AC from memory

Figure: Example of Program Execution (contents of memory and registers in hexadecimal)
(Source: (Stallings, 2015))

36 EXAMPLE

Assume **PC=300**:



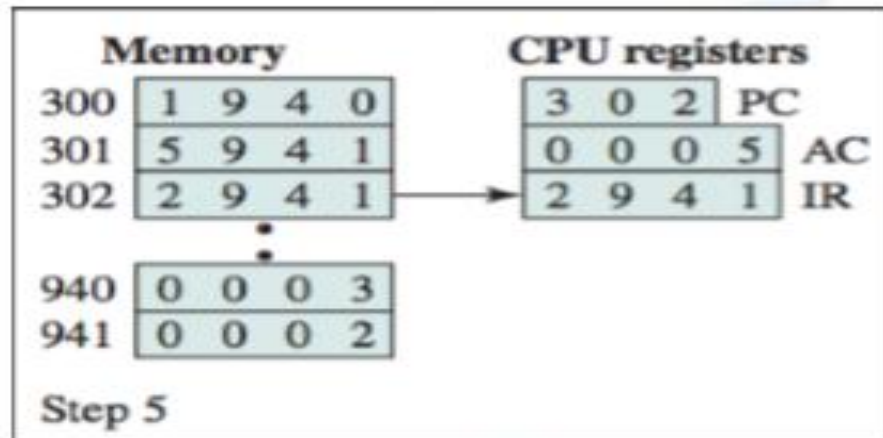
Opcodes:

0001	Load AC from memory
0010	Store AC to memory
0101	Add to AC from memory

Figure: Example of Program Execution (contents of memory and registers in hexadecimal)
(Source: (Stallings, 2015))

37 EXAMPLE

Assume **PC=300**:



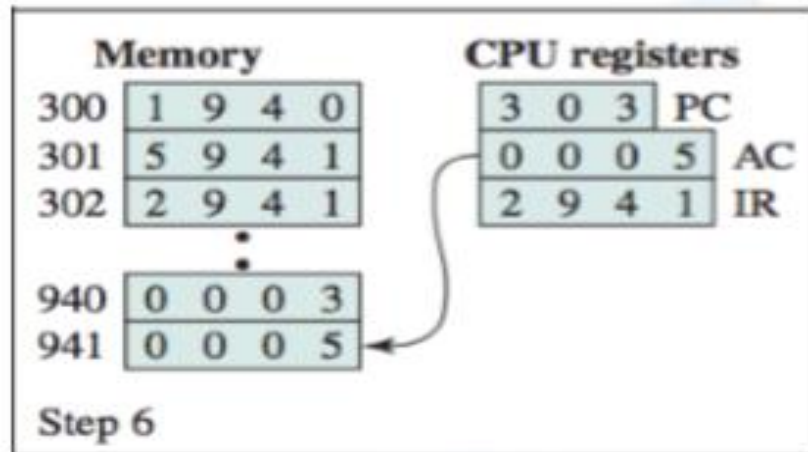
Opcodes:

0001	Load AC from memory
0010	Store AC to memory
0101	Add to AC from memory

Figure: Example of Program Execution (contents of memory and registers in hexadecimal)
(Source: (Stallings, 2015))

38 EXAMPLE

Assume PC=300:



Opcodes:

0001	Load AC from memory
0010	Store AC to memory
0101	Add to AC from memory

Figure: Example of Program Execution (contents of memory and registers in hexadecimal)
(Source: (Stallings, 2015))

39 EXAMPLE

- In textual form:
- The PC contains value 300, the address of the first instruction.
 - This instruction (the value 1940 in hexadecimal) is loaded into the instruction register IR, and the PC is incremented;
 - The first 4 bits (first hexadecimal digit) in the IR indicate that the AC is to be loaded.
 - The remaining 12 bits (three hexadecimal digits) specify the address (940) from which data are to be loaded.
 - The next instruction (5941) is fetched from location 301, and the PC is incremented.
 - The old contents of the AC and the contents of location 941 are added, and the result is stored in the AC.
 - The next instruction (2941) is fetched from location 302, and the PC is incremented.
 - The contents of the AC are stored in location 941.

40 CLASS WORK

- Now lets try to execute the following instructions (assume PC=300):
 - Add **memory contents** at address **940** to the **contents at address 941**;
 - Add **memory contents** at address **941** to the **contents at address 942**;
 - Store the result in the **latter** location -942.

Assume PC=300:

3/16/2026

41

Memory					CPU registers			
300	1	9	4	0	3	0	0	PC
301	5	9	4	1				
302	5	9	4	2				
303	2	9	4	2				
⋮								
⋮								
⋮								
940	0	0	0	3				
941	0	0	0	2				
942	0	0	0	2				

Opcodes:

0001	Load AC from memory
0010	Store AC to memory
0101	Add to AC from memory

Figure: Example of Program Execution (contents of memory and registers in hexadecimal)
(Source: (Stallings, 2015))

42

?



43 BASIC INSTRUCTION CYCLE

- Some important notes:
 - Last example was very simplified;
 - But the overall control structure and information flow is present;
 - However, there are a lot of details missing;
- Lets look into a more detailed instruction cycle...

DETAILED INSTRUCTION CYCLE

3/16/2026

44 A more detailed look at the **basic instruction cycle**:

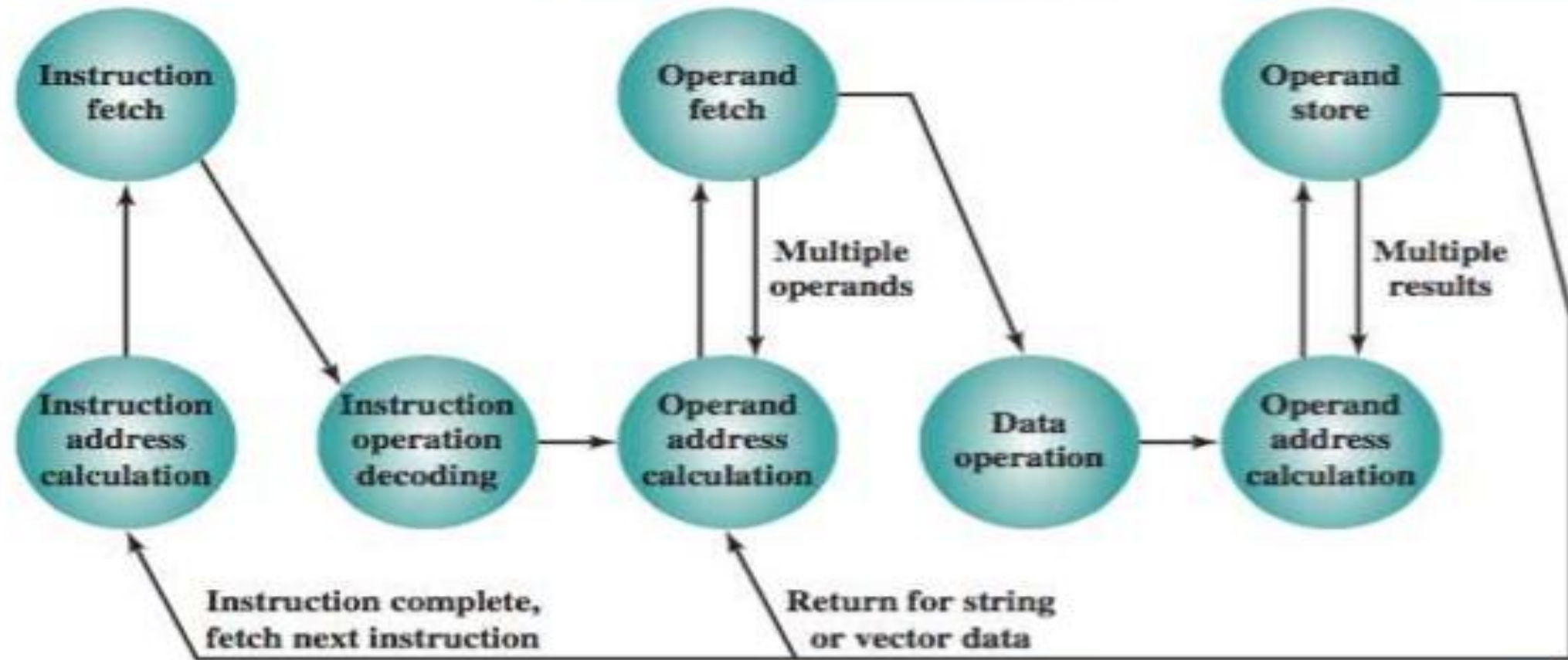


Figure: A more realistic instruction cycle state diagram (Source: (Stallings, 2015))

45 DETAILED INSTRUCTION CYCLE

- The states can be described as follows:
- **Instruction address calculation (IAC):**
 - Determine address of next instruction to be executed;
- **Instruction fetch (IF):**
 - Read instruction from memory into the processor;
- **Instruction operation decoding (IOD):**
 - Determine type of operation to be performed and operand(s) to be used.

46 DETAILED INSTRUCTION CYCLE

- **Operand address calculation (OAC):**
 - Determine the address of the operand.
- **Operand fetch (OF):**
 - Fetch the operand from memory or read it in from I/O.
- **Data operation (DO):**
 - Perform the operation indicated in the instruction.
- **Operand store (OS):**
 - Write the result into memory or to I/O.

47 DETAILED INSTRUCTION CYCLE

- Do you think those steps are enough? Should we add other states? Is there something missing?

48 DETAILED INSTRUCTION CYCLE

- There exists some situations where we may want to interrupt the CPU:
- **Program:** arithmetic overflow, division by zero, segfault, etc..
- **Timer:** execute something periodically;
- **I/O:** exchange communication between I/O devices and the processor;
- **Hardware:** generated by a hardware failure
 - e.g.: power, memory parity error, etc;

49 INTERRUPTS

- Because of these reasons (and some others):
 - Computers provide a mechanism to **interrupt** the processing;
 - Virtually all computers provide a mechanism to **interrupt** the normal processing of the processor.
- How does that work?

50 INTERRUPTS

1. An interrupt request signal is sent to the **processor**.
2. The processor responds by **suspending** current program execution;
3. The processor executes a **routine** capable of dealing with the interrupt;
 - A.k.a. interrupt handler.
4. After the handler **finishes**, the **program** returns to the original execution.

51 INTERRUPTS

There is some **overhead** involved in this process:

- **Extra instructions** must be executed:
 - to determine the **nature** of the **interrupt**;
 - decide the **appropriate action**.
- But what if **multiple interruptions** being generated at the same time?
 - Example:
 - A **key** is pressed;
 - A **segfault** is generated;

52 MULTIPLE INTERRUPTS

- Suppose that **multiple interrupts** can occur, e.g.: a program may be
 - **receiving** data from a **communications** line:
 - an interruption is **generated** every time a unit of **data** arrives.
 - **printing**:
 - an interruption is generated every time a **print** operation is completed;
- *How should the system deal with such cases? (2 possibilities)*

53 FIRST POSSIBILITY

- **Disable** interrupts while an interrupt is being processed:
 - The processor will **ignore interrupt** request signals;
 - Emphasis on the “ignore”, interrupt signals can still be generated.
 - After the interrupt handler routine **completes**:
 - Interrupts are **enabled**;
 - the processor checks to see if any **interrupts** have occurred.

54 ISSUES OF THE FIRST ONE

- Can you see any problem with this approach of **disabling interrupts while processing an interruption?**
 - **Ignoring** interruptions may have serious implications

55 SECOND POSSIBILITY

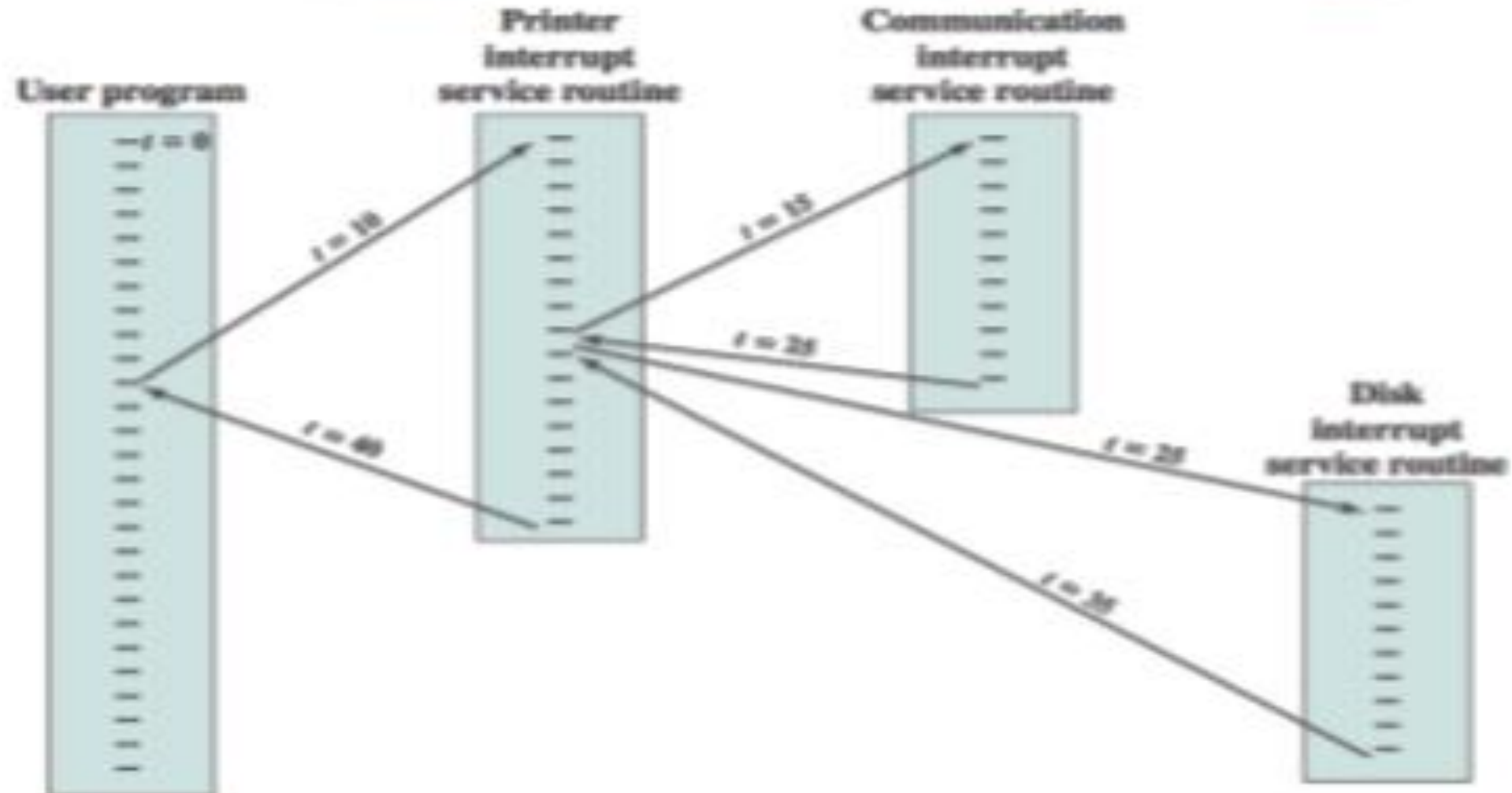
- Don't **ignore** and **allow** each interruption to have a priority:
 - **Higher** priority interruptions can interrupt **lower-priority** interruptions.
 - This strategy is referred to as **nested interrupt processing**

Consider a system with three I/O devices:

3/16/2026

56

Device	Interrupt Priority
Printer	2
Disk	4
Comm. line	5



EXAMPLE

3/16/2026

- 57 • Program begins at $t = 0$ and at $t = 10$, a printer interrupt occurs:

-
1. context is saved;
 2. execution goes to the printer interrupt service routine (ISR);
 3. While this routine is executing ($t = 15$), a communications interrupt occurs:
 1. Higher priority than the printer;
 2. Printer ISR is interrupted, context is saved;
 3. Execution goes to the communication ISR;
 1. While this routine is executing, a disk interrupt occurs ($t = 20$);
 2. Because this interrupt is of lower priority, it is simply held;
 3. and the communications ISR runs to completion.

EXAMPLE

3/16/2026

58

-
4. When the communications ISR completes ($t = 25$):
 1. printer ISR state is restored;
 2. However, there is a pending higher-priority interruption (disk);
 1. Processor transfers control to the disk ISR;
 2. disk ISR completes at $t = 35$;
 3. printer ISR is resumed;
 4. printer ISR terminates at $t = 40$;
 5. control returns to the original program.

59 INTERCONNECTION ISSUES

- A **computer** consists of a set basic types of components:
- These modules need to communicate with each other in order to :
 - exchange **data** and **address**;
 - exchange **control** signals;
- Any idea how this communication can be performed?

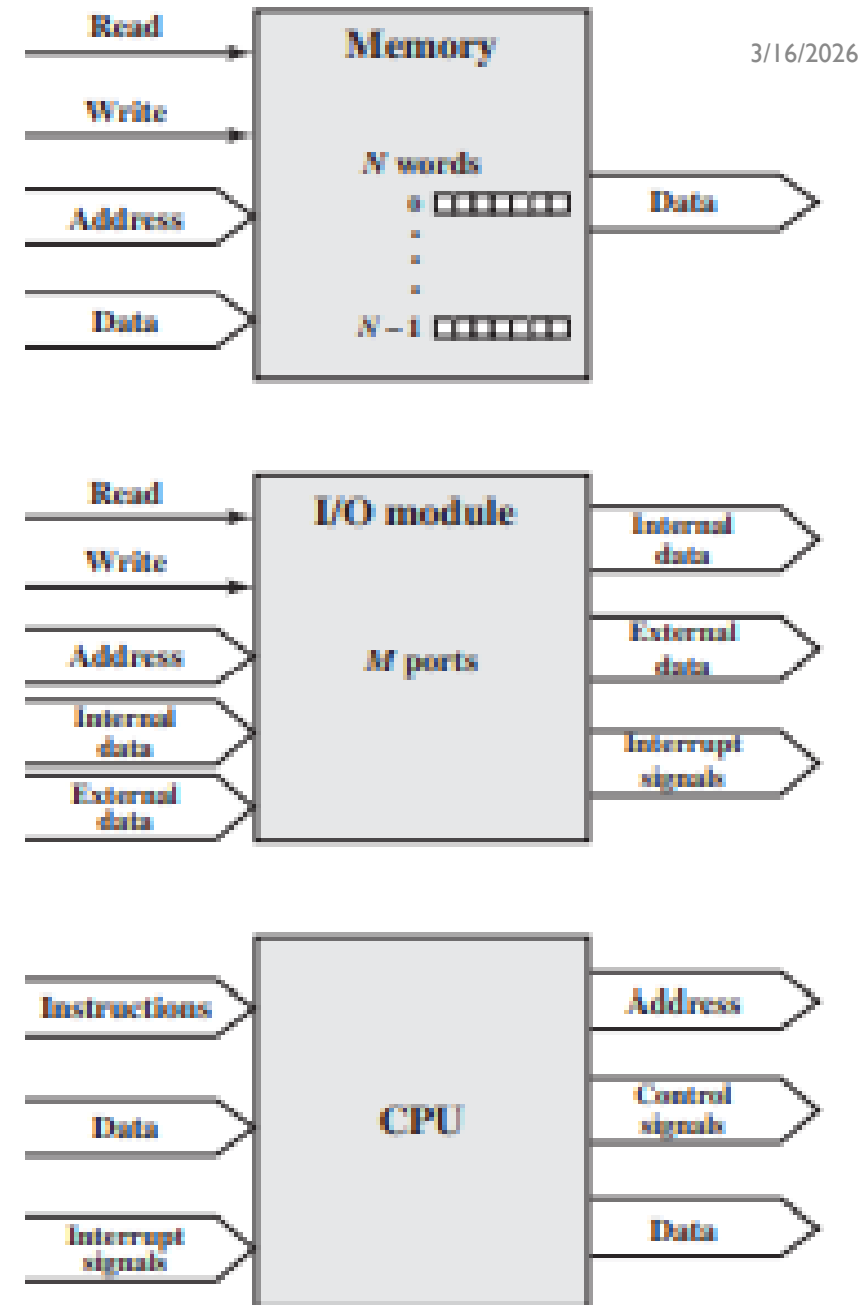


Figure 3.15 Computer Modules

60 BUS INTERCONNECTION

- A **bus** is a communication pathway connecting **two** or **more** devices:
 - **shared** transmission medium;
 - connected devices can **pickup** data from **all other devices**.
 - If several devices transmit during the **same** time period:
 - their signals will **overlap** and become **garbled**:
 - thus, only **one** device at a time can successfully **transmit**.

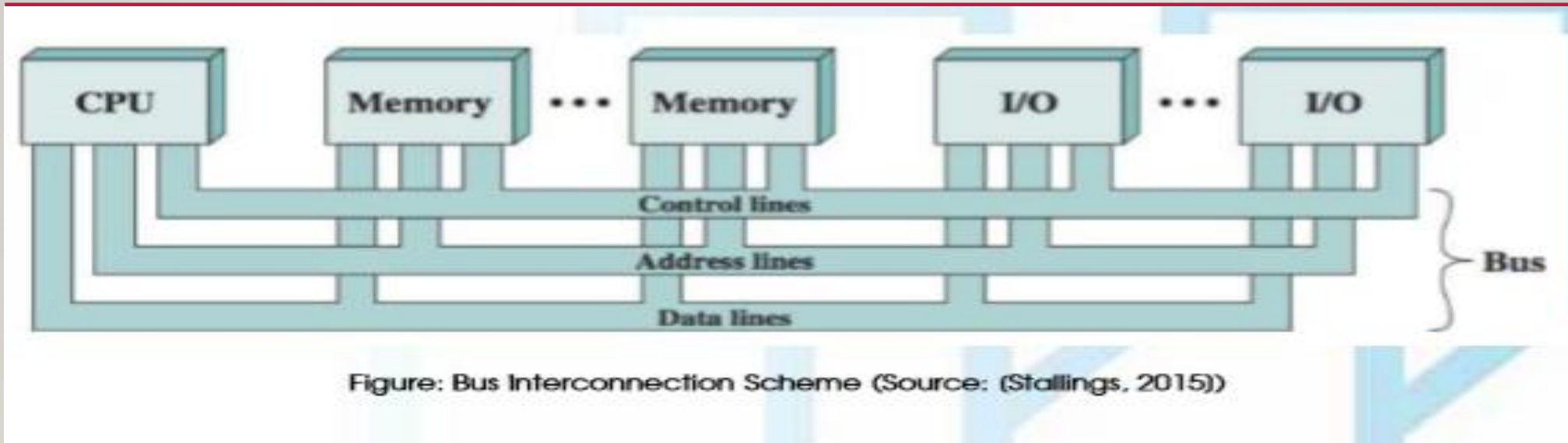
6 | BUS INTERCONNECTION

- A bus consists of multiple communication lines:
 - Each line transmits **binary** signals.
 - A **sequence of binary digits** can be transmitted:
 - using a **single line over time** (i.e. sequentially);
 - **several lines can be used** (i.e. in parallel).
- A bus connecting major computer components is called a **system bus**.
- *What type of information does the bus carry?*

62 BUS STRUCTURE

- **Bus lines** can be classified into **three** functional groups:
 - **data:**
 - for moving **data** among system modules
 - **address**
 - for specifying the **source** or **destination** of the data:
 - **control**
 - for transmitting **command** information among the **modules**.

63 BUS STRUCTURE



64 DATA LINES

- The data bus may consist of **32, 64, 128**, or even more **separate lines**:
 - a.k.a. **width** of the data bus;
- Each line can carry only **1 bit at a time**:
 - the **number of lines** determines how **many** bits can be transferred at a time.
- **Data bus width** is key to **system performance**, e.g.:
 - if the data bus is **32** bits wide and each instruction is **64** bits long;
 - Each instruction requires **two memory accesses**.

• *Lets have a quick look into each one of these...*

65 ADDRESS LINES

- Used to designate the **source** or **destination** of the data on the **data bus**:
 - The **width of the address bus** determines the **maximum system memory**;
 - How much memory the CPU can address directly Ex. 32 bits
 - The address lines are generally also used to address **I/O addresses**;
 - **Higher-order** bits are used to select a **particular module** on the bus;
 - **Lower-order bits** select a **memory location** or **I/O port** within the module.

66 CONTROL LINES

- Command signals specify operations to be performed, e.g.:
 - **Memory write:** write bus data to a memory address;
 - **Memory read:** read memory at memory address;
 - **I/O write:** write bus data to an I/O address;
 - **I/O read:** read data from an I/O address;
 - **Bus request:** a module needs to gain control of the bus;
 - **Bus grant:** a requesting module has been granted bus control;
 - Many more control signals...

67 BUS STRUCTURE

- What happens if a **great number of devices** are connected to the bus?
 - If a great number of devices are connected **bus performance will suffer.**

68 SINGLE BUS PROBLEMS

- Single Bus Problems
 - Lots of devices on one bus leads to:
 - Propagation delays
 - Long data paths mean that co-ordination of bus use can adversely affect **performance**
 - Bus may become **bottleneck**
 - Most systems use **multiple buses** to overcome these problems

69 MULTIPLE-BUS HIERARCHIES

- Idea: Use several buses to distribute communication effort;
- This implies that there needs to be a bus hierarchy, e.g.:

3/16/2026

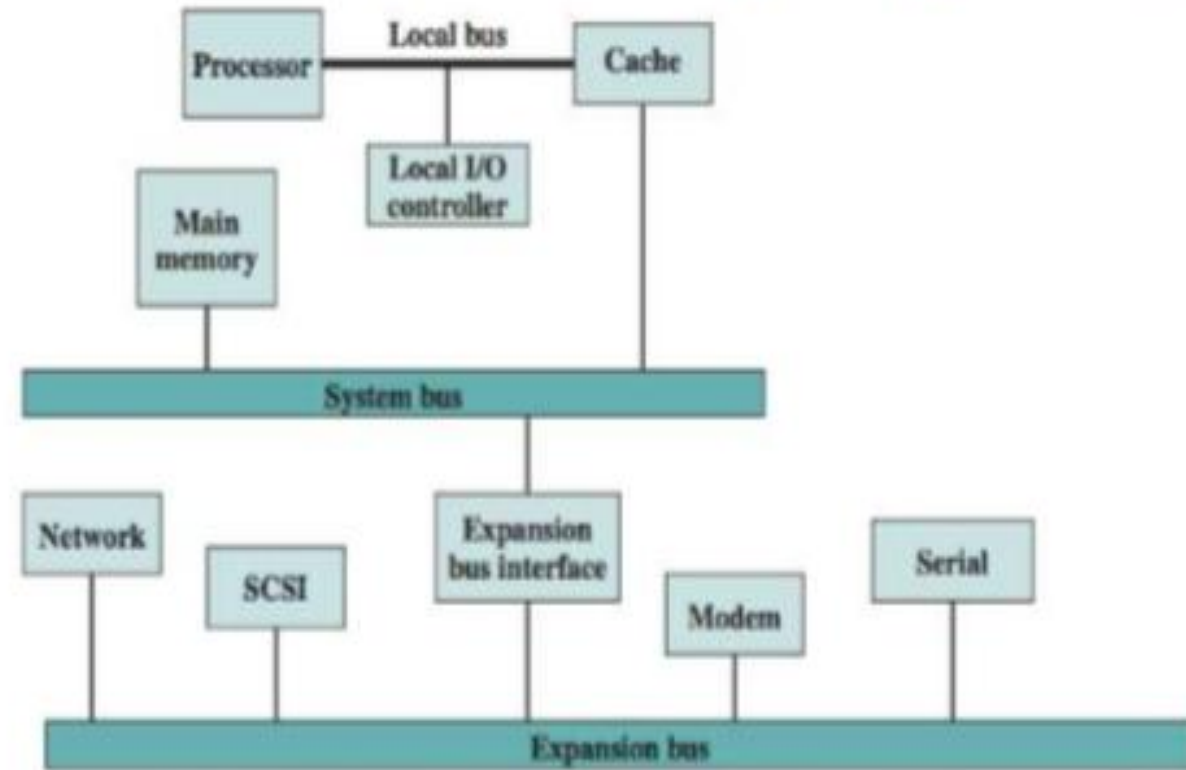


Figure: Traditional Bus Architecture (Source: (Stallings, 2015))

70 MULTIPLE-BUS HIERARCHIES

- Local bus connecting:
 - processor;
 - cache memory;
 - one or more local devices.
- System bus:
 - where the main memory module is attached;
 - that also connects to the cache;
- Note: in contemporary systems, the cache is in the same chip as the processor.

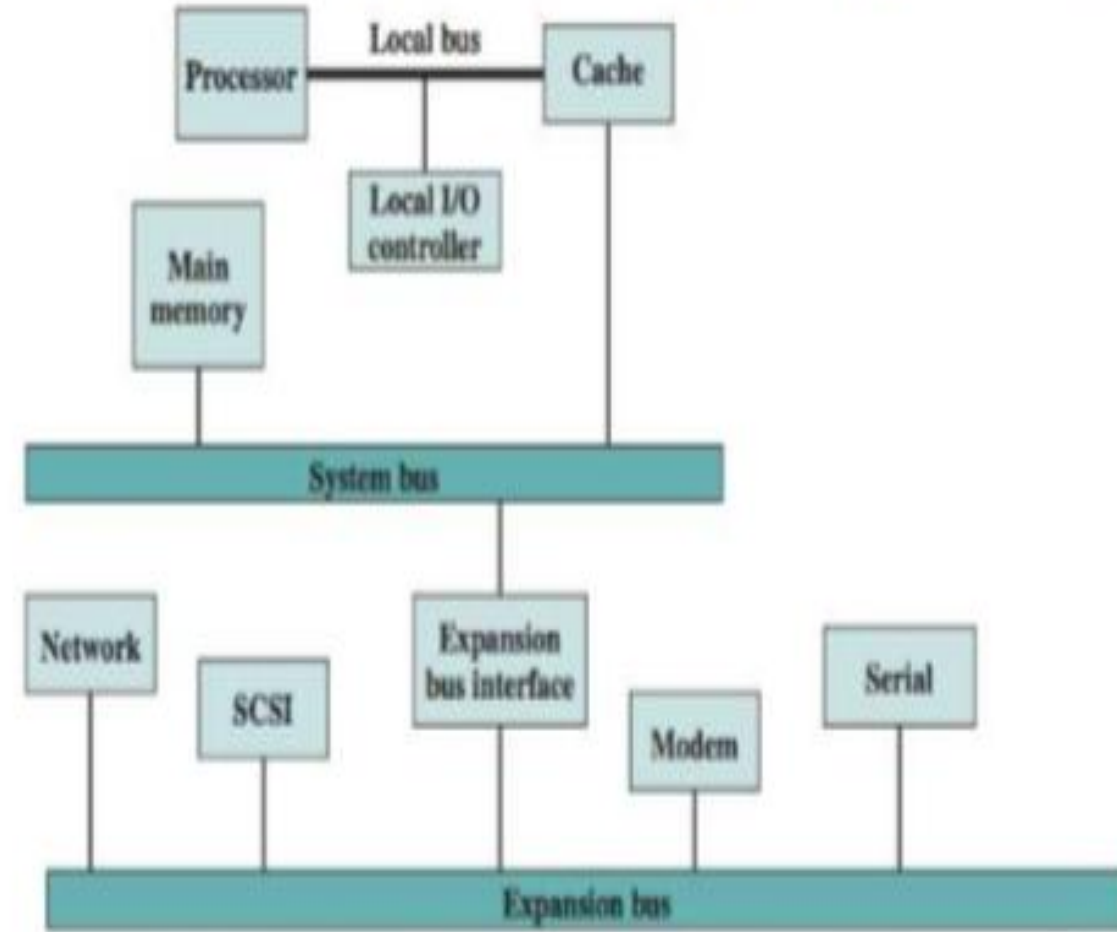


Figure: Traditional Bus Architecture (Source: (Stallings, 2015))

71 ELEMENTS OF BUS DESIGN

- Parameters that can be used to classify and differentiate buses e.g.:

Type	Bus Width
Dedicated	Address
Multiplexed	Data
Method of Arbitration	Data Transfer Type
Centralized	Read
Distributed	Write
Timing	Read-modify-write
Synchronous	Read-after-write
Asynchronous	Block

Figure: Elements of bus design (Source: (Stallings, 2015))

72 BUS TYPE

- Bus lines can be separated into two generic types:
 - **dedicated**: line is used for a single purpose:
 - e.g.: the use of separate dedicated **address** and **data** lines
 - **multiplexed**: line is used for **multiple** purposes:
 - e.g.: address and data information may be transmitted over the same lines
 - Idea: use an Address Valid control line:
 - Activate Address Valid line;
 - Place the address on the bus lines;
 - Transfer the data after.
 - *Can you see any advantages / disadvantages with multiplexing?*

73 BUS TYPE

- **Multiplexing advantage:**
 - use of fewer lines, which saves space and, usually, cost.
- **Multiplexing disadvantage:**
 - more complex circuitry is needed within each module;
 - potential reduction in performance:
 - certain events that share the same lines cannot take place in parallel.

74 ELEMENTS OF BUS DESIGN AND BUS ARBITRATIONS: READING ASSIGNMENT

Design two buses based on the parameter. Compare and contrast each design based on the parameters selected.

75

CH4 CACHE & OTHER MEMORIES

ADERAW S. NOV-2025



76 REMEMBER THIS GUY



Von Neumann Architecture:

- CPU ;
- Memory;
- I/O Module



77 INTRODUCTION

- **Today's focus: Memory Module** of von Neumann's architecture

78 INTRODUCTION

- Although simple in concept computer memory exhibits wide range of:
 - **Location:** internal/external, permanent/temporary
 - **Technology:** optical/magnetic/semiconductor
 - **Organization:** words/bytes/blocks
 - **Performance:** fast/slow
 - and **cost:** Expensive/cheap
- No single technology is optimal in satisfying all of these

79 INTRODUCTION

Typically:

- Higher performance → higher cost;
- Lower performance → lower cost

80 INTRODUCTION

A computer has a **hierarchy** of memory subsystems:

- some **internal** to the system
 - i.e. directly accessible by the processor;
- some **external**
 - accessible via an I/O module;

CLASSIFICATION OF MEMORY SYSTEMS ACCORDING TO THEIR KEY CHARACTERISTICS:

81

Location

Internal (e.g. processor registers, main memory, cache)

External (e.g. optical disks, magnetic disks, tapes)

Capacity

Number of words

Number of bytes

Unit of Transfer

Word

Block

Access Method

Sequential

Direct

Random

Associative

Performance

Access time

Cycle time

Transfer rate

Physical Type

Semiconductor

Magnetic

Optical

Magneto-optical

Physical Characteristics

Volatile/nonvolatile

Erasable/nonerasable

82 CHARACTERISTICS OF MEMORY SYSTEMS

Location: either **internal** or **external** to the processor.

- Forms of internal memory:
 - **registers;**
 - **cache;**
 - **RAM**
- Forms of external memory:
 - **disk;**
 - **magnetic tape**
 - devices that are accessible to the processor via **I/O** controllers.

83 CHARACTERISTICS OF MEMORY SYSTEMS

- **Capacity: amount** of information the memory can hold.
 - Typically expressed in terms of **bytes** (1 byte = 8 bits) or **words**;
 - A **word** represents each **addressable** block of the memory
 - common word lengths are **8, 16, and 32 bits**;
 - External memory capacity is typically expressed in terms of **bytes**;

84 CHARACTERISTICS OF MEMORY SYSTEMS

- **Unity of transfer:**
 - For **internal** memory, the unit of transfer is equal to the number of **electrical lines** into and out of the memory module.
 - This may be equal to the **word length**, but is often larger, such as 64, 128, or 256 bits.
 - To clarify this point, consider three related concepts for internal memory:

85 CHARACTERISTICS OF MEMORY SYSTEMS

- **Word:**
 - The “**natural**” unit of **organization** of memory.
 - The size of the word is typically equal to the number of bits used to represent an **integer** and to the **instruction length**.
 - there are many exceptions.
- **Addressable units:**
 - the addressable unit is the **word**.
 - However, many systems allow addressing at the **byte** level.
 - In any case, the relationship between the length in bits A of an address and the number N of addressable units is $2^A = N$.
- **Unit of transfer:**
 - this is the **number of bits** read out of or written into memory at a time.
 - The unit of transfer need not equal a **word** or an **addressable unit**.
 - For external memory,
 - data are often transferred in much larger units than a word, and these are referred to as **blocks**.

86 CHARACTERISTICS OF MEMORY SYSTEMS

- **Access Method:** How are the units of memory accessed?
 - **Sequential Method:** Memory is organized into units of data, called records.
 - Access must be made in a specific **linear** sequence;
 - Stored addressing information is used to assist in the retrieval process.
 - A shared read-write head is used;
 - The head must be moved from its one location to the another;
 - Passing and rejecting each intermediate record;
 - **Highly variable** times.



87 CHARACTERISTICS OF MEMORY SYSTEMS

- **Direct Access Memory:**

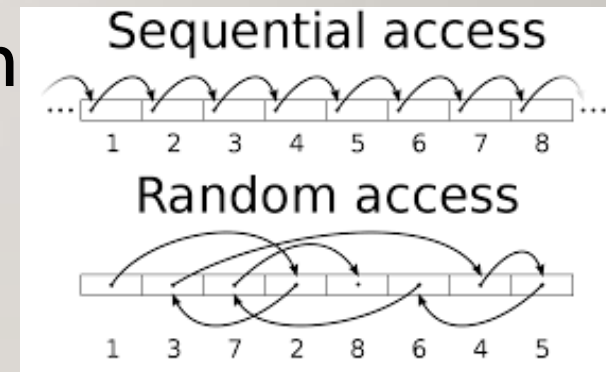
- Involves a shared read-write mechanism;
- Individual records have a unique address;
- Requires accessing general record vicinity plus sequential searching, counting, or waiting to reach the final location;
- Access time is also **variable**;



Magnetic Disk

88 CHARACTERISTICS OF MEMORY SYSTEMS

- **Random Access:** Each addressable location in memory has a unique, **physically wired-in** addressing mechanism
 - Constant time;
 - independent of the sequence of prior accesses;
 - Any location can be selected at random and directly accessed;
 - Main memory and some cache systems are random access



89 CHARACTERISTICS OF MEMORY SYSTEMS

- **Associative:** a memory whose storage locations are identified by their contents, or by a part of their contents, rather than by their names or positions
 - Word is retrieved based on a portion of its contents rather than its address;
 - Retrieval time is constant independent of location or prior access patterns

90 CHARACTERISTICS OF MEMORY SYSTEMS

- **Performance:**
 - **Access time (latency):**
 - For RAM: time to perform a read or write operation;
 - For Non-RAM: time to position the read-write head at desired location;
 - **Memory cycle time:** Primarily applied to RAM:
 - Access time + additional time required before a second access;
 - Required for electrical signals to be terminated/regenerated;
 - Concerns the system bus.

9 | CHARACTERISTICS OF MEMORY SYSTEMS

- **Transfer time:** Rate at which data can be transferred in / out of memory;
 - For RAM: $\frac{1}{\text{cycle time}}$
 - For Non-RAM: $T_n = T_A + \frac{n}{R}$, where:
 - T_n : Average time to read or write n bits;
 - T_A : Average access time;
 - n : Number of bits
 - R : Transfer rate, in bits per second (bps.)

92 CHARACTERISTICS OF MEMORY SYSTEMS

- **Physical characteristics:**
 - **Volatile:** information **decays** naturally or is lost when **powered off**;
 - RAM
 - **Nonvolatile:** information **remains** without deterioration until **changed**:
 - no electrical power is needed to retain information.;
 - E.g.: Magnetic-surface memories are nonvolatile;
 - Semiconductor memory (memory on integrated circuits) may be either volatile or nonvolatile.

93 MEMORY HIERARCHY

Design **constraints** on memory can be summed up by three questions:

- **How much?**
 - If memory exists, applications will likely be developed to use it.
- **How fast?**
 - Best **performance** achieved when memory **keeps up with the processor**;
 - i.e. as the processor execute instructions, memory should minimize **pausing / waiting** for instructions or operands.
- **How expensive?**
 - Cost of memory must be reasonable in relationship to other components

94 MEMORY HIERARCHY

Memory **tradeoffs** are a sad part of reality

- **Faster** access time, greater **cost** per bit;
- Greater **capacity**:
 - **Smaller** cost per bit;
 - **Slower** access time;

95 MEMORY HIERARCHY

These tradeoff imply a **dilemma**:

- Large **capacity** memories are **desired**:
 - low **cost** and because the **capacity** is needed;
- However, to meet **performance** requirements, the designer needs:
 - to use **expensive**, relatively **lower-capacity** memories with **short access** times

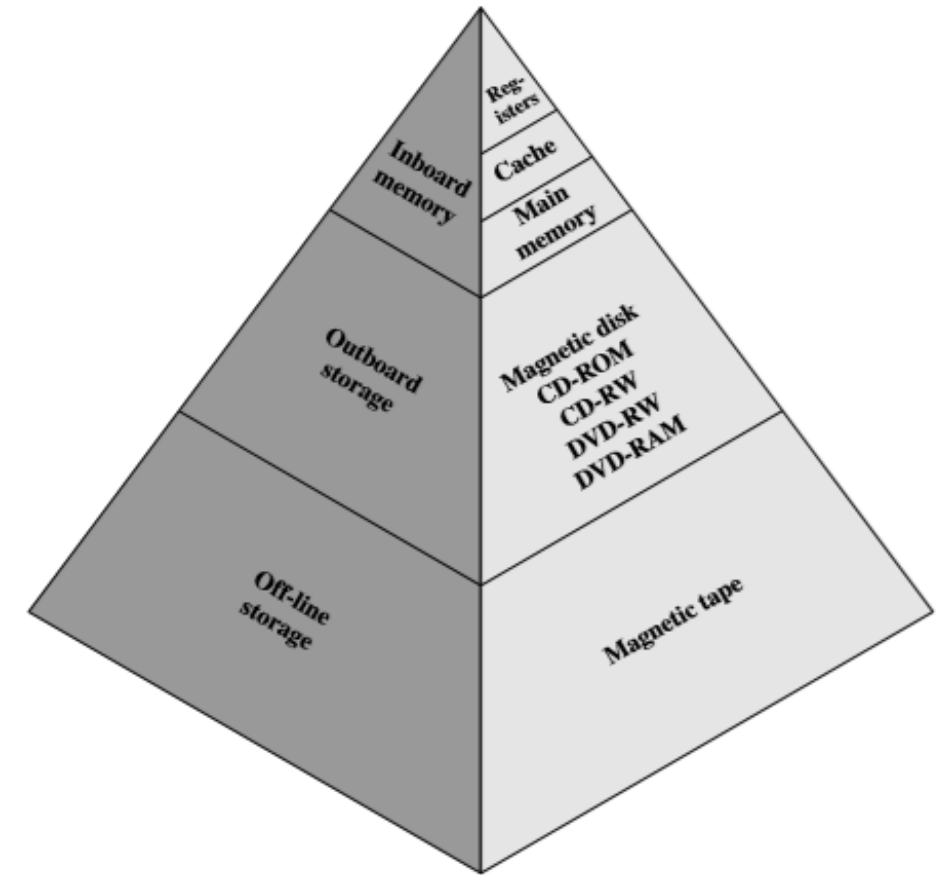
How can we solve this issue? Or at least mitigate the problem?



96 MEMORY HIERARCHY

The way out of this **dilemma**:

- Don't rely on a **single memory**;
- Instead employ a **memory hierarchy**;
- **Supplement**:
 - smaller, more expensive, faster memories with...
 - ...larger, cheaper, slower memories;



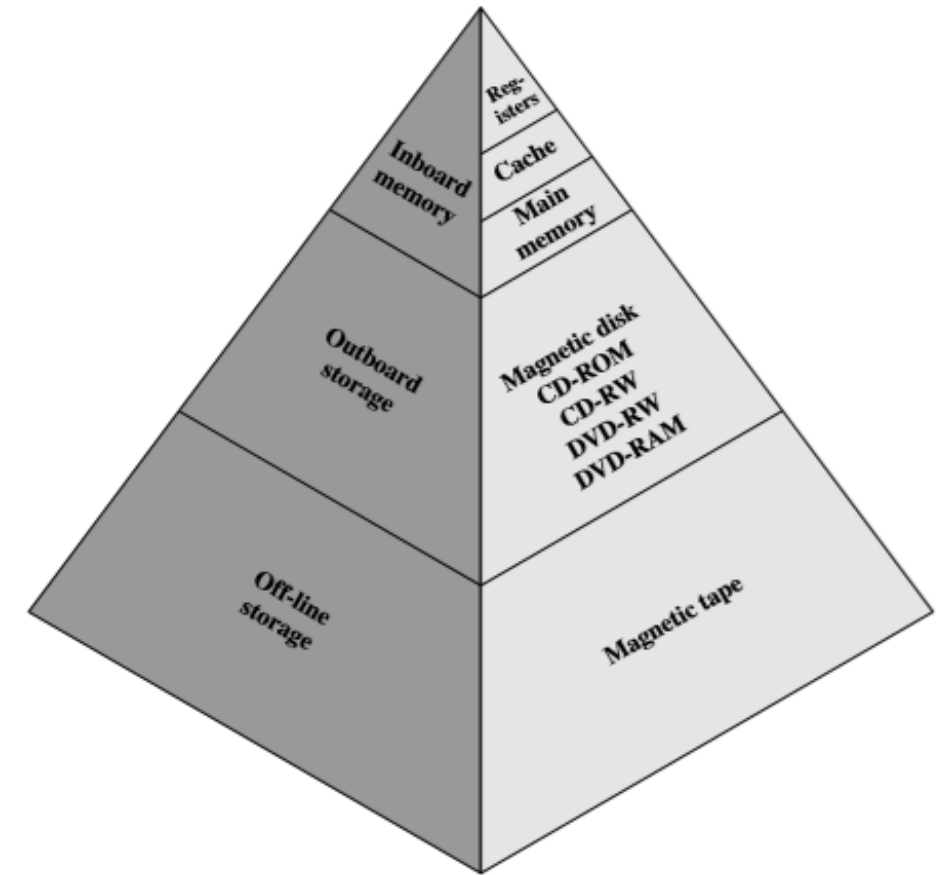
97 MEMORY HIERARCHY

As one goes down the hierarchy:

- **Decreasing cost** per bit;
- **Increasing capacity**;
- **Increasing access time**;
- **Decreasing frequency** of access of memory by processor

Decreasing frequency of memory access by processor.

But why is this key to success?



98 MEMORY HIERARCHY

But why is this key to success?

- As we go down the **hierarchy** we gain in **size** but lose in **speed**;
- Therefore: not **efficient** for the processor to **access** these **memories**;
- **Requires having specific** strategies to minimize such **accesses**;

How can we develop strategies to minimize these accesses?



99 MEMORY HIERARCHY

-
- **Registers**
 - **L1 Cache**
 - **L2 Cache**
 - **Main memory**
 - **Disk cache**
 - **Disk**
 - **Optical**
 - **Tape**
- **As one goes down the hierarchy**
 - **Decreasing cost per bit**
 - **Increasing capacity**
 - **Increasing access time**
 - **Decreasing frequency of access of the memory by the processor – locality of reference**

100 MEMORY HIERARCHY

- This leads us to some definitions.
 - A *hit* is when data is found at a given memory level.
 - A *miss* is when it is not found.
 - The *hit rate* is the percentage of time data is found at a given memory level.
 - The *miss rate* is the percentage of time it is not.
 - Miss rate = $1 - \text{hit rate}$.
 - The *hit time* is the time required to access data at a given memory level.
 - The *miss penalty* is the time required to process a miss, including the time that it takes to replace a block of memory plus the time it takes to deliver the data to the processor.

101 MEMORY HIERARCHY

Hit: data appears in level X: Hit Rate: the fraction of memory accesses found in the upper level

Miss: data needs to be retrieved from a block in the lower level (Block Y) Miss Rate = $1 - (\text{Hit Rate})$

Hit Time: Time to access the upper level which consists of Time to determine hit/miss + memory access time

Miss Penalty: Time to replace a block in the upper level + Time to deliver the block to the processor

Note: Hit Time $\ll$ Miss Penalty

102 MEMORY HIERARCHY

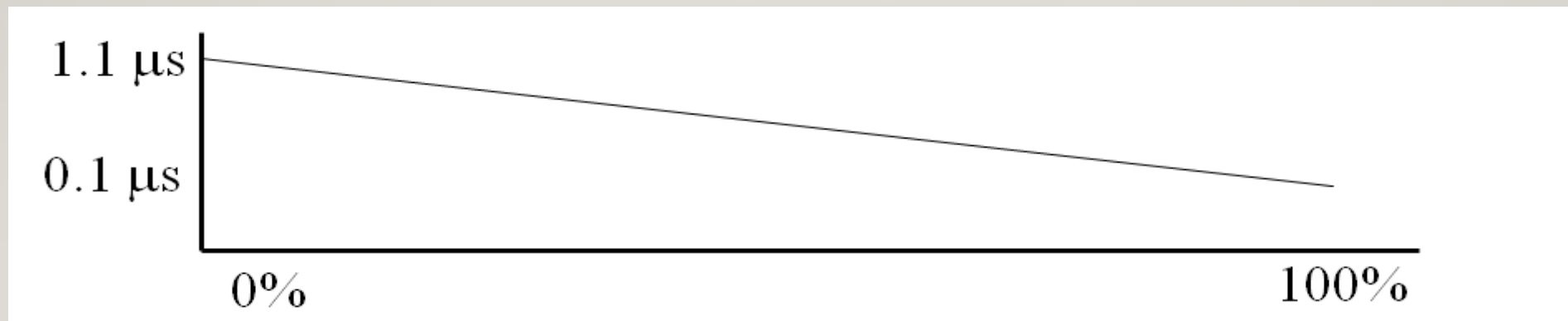
Cache operation - overview

- CPU requests contents of memory location
- Check cache for this data
- If present, get from cache (fast)
- If not present, read required block from main memory to cache
- Then deliver from cache to CPU
- Cache includes tags to identify which block of main memory is in each cache slot

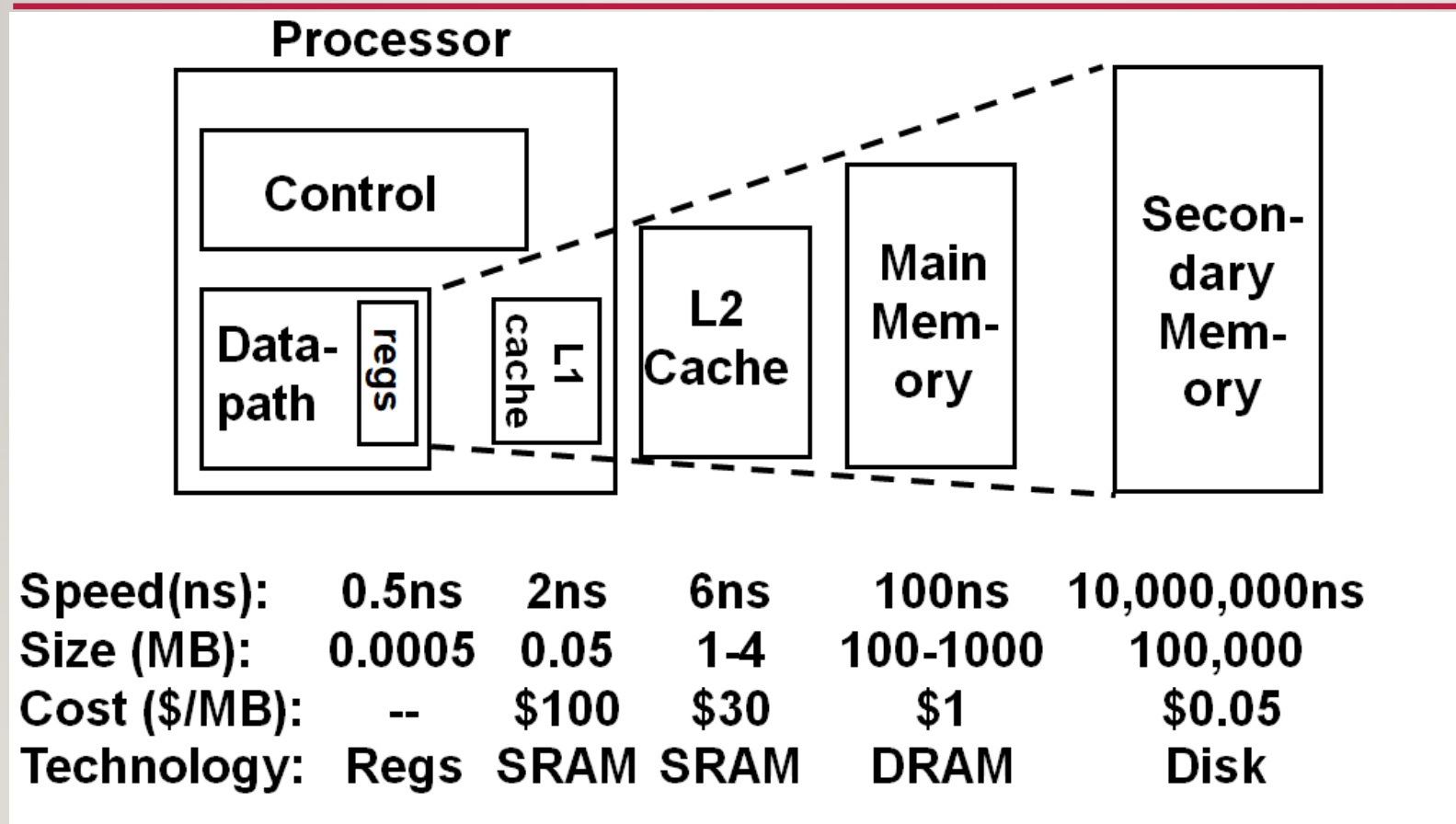
CACHE EXAMPLE

3/16/2026

- 103 • Consider a Level 1 cache capable of holding 1000 words with a $0.1 \mu\text{s}$ access time. Level 2 cache capable of holding 100000 words is memory with a $1 \mu\text{s}$ access time.
-
- If 95% of memory access is in the cache of Level 1 :
 - $T = (0.95) * (0.1 \mu\text{s}) + (0.05) * (0.1 + 1 \mu\text{s}) = 0.15 \mu\text{s}$
 - If 5% of memory access is in the cache of Level 1 :
 - $T = (0.05) * (0.1 \mu\text{s}) + (0.95) * (0.1 + 1 \mu\text{s}) = 1.05 \mu\text{s}$
 - Want as many cache hits as possible!



104 MEMORY HIERARCHY



105 MEMORY HIERARCHY

How can we develop strategies to minimize these accesses?

Space and Time locality of reference principle:

- **Locality in Space (spatial locality):**

- if we **access** a memory **location**, **close by addresses** will very likely be accessed;
=> Move blocks consists of contiguous words to the upper levels

- **Locality in Time (temporal locality):**

- if we **access** a memory **location**, we will very likely access it **again**;

But why does this happen?

=> Keep most recently accessed data items closer to the processor

106 MEMORY HIERARCHY

- Temporal Locality
 - Programs tend to reference the same memory locations at a future point in time
 - Due to loops and iteration, programs spending a lot of time in one section of code
- Spatial Locality
 - Programs tend to reference memory locations that are near other recently-referenced memory locations
 - Due to the way contiguous memory is referenced, e.g. an array or the instructions that make up a program
 - Sequential Locality
 - Instructions tend to be accessed sequentially
- Locality of reference does not always hold, but it usually holds

-
- An entire blocks of data is copied after a hit because the *principle of locality* tells us that once a byte is accessed, it is likely that a nearby data element will be needed soon.
 - There are three forms of locality:
 - *Temporal locality*- Recently-accessed data elements tend to be accessed again.
 - *Spatial locality* - Accesses tend to cluster.
 - *Sequential locality* - Instructions tend to be accessed sequentially.

108 MEMORY HIERARCHY

Suppose that the processor has access to **two levels of memory**:

- **Level 1 - L1:**
 - contains **1000** words and has an access time of **0.01 μ s**;
- **Level 2 - L2:**
 - contains **100,000 words** and has an access time of **0.1 μ s**.
- Assume that:
 - if word $\in$ **L1**, then the processor accesses it **directly**;
 - If word $\in$ **L2**, then word is transferred to **L1** and then accessed by the processor.

109 CONT...

- For high percentages of L1 access, the average total access time is much closer to that of L1 than that of L2; Now lets consider the following scenario:
- Suppose 95% of the memory accesses are found in L1.
- What is the average time to access a word in L2 and L1:

110 MEMORY HIERARCHY

Strategy to minimize accesses should be:

- Organize data across the hierarchy such that
 - % of accesses to **lower levels** is substantially **less than** that of **upper levels**
- I.e. **L2** memory contains all **program instructions** and **data**:
 - **Data** that is **currently** being used should be in **L1**;
 - Eventually:
 - Data $\in$ **L1** will be **swapped** to **L2** to make room for **new data**;
 - On average, most **references** will be to data contained in **L1**.

||| MEMORY HIERARCHY

This principle can be applied across more than two levels of memory:

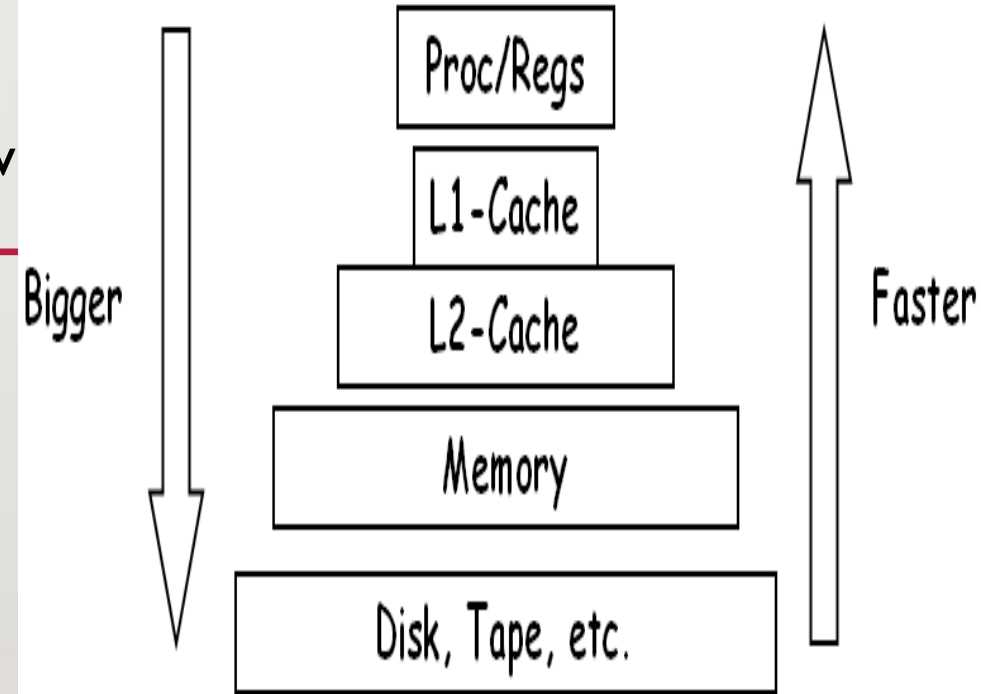
- **Processor registers:**
 - **Fastest, smallest, and most expensive** type of memory
- Followed immediately by the **cache**:
 - Stages data movement **between registers** and **main memory**;
 - Improves performance;
 - Is not usually visible to the **processor**;
 - Is not usually visible to the **programmer**.
- Followed by **main memory**:
 - Principal internal memory system of the computer;
 - Each location has a unique address

WHAT IS A CACHE?

3/16/2026

112

- Small, fast storage used to improve average access time to slow memory.
- Exploits spacial and temporal locality
- In computer architecture, almost everything is a cache!
 - Registers a cache on variables
 - First-level cache a cache on second-level cache
 - Second-level cache a cache on memory
 - Memory a cache on disk (virtual memory)
 - TLB a cache on page table (A special, small, fast, lookup hardware cache called Translation Lookaside Buffer (TLB) is used to cache small number of entries from the page table. Each TLB entry consists of two parts: key(or tag) and value, here key is the page number and value is the frame number.)
 - Branch-prediction a cache on prediction information?



CACHE

3/16/2026

113

-
- The purpose of cache memory is to speed up accesses by storing recently used data closer to the CPU, instead of storing it in main memory.
 - Although cache is much smaller than main memory, its access time is a fraction of that of main memory.
 - Unlike main memory, which is accessed by address, cache is typically accessed by content; hence, it is often called *content addressable memory*.
 - Because of this, a single large cache memory isn't always desirable-- it takes longer to search.

CACHE

114

-
- The “content” that is addressed in content addressable cache memory is a subset of the bits of a main memory address called a *field*.
 - The fields into which a memory address is divided provide a many-to-one mapping between larger main memory and the smaller cache memory.
 - Many blocks of main memory map to a single block of cache. A *tag* field in the cache block distinguishes one cached memory block from another.

• Cache

- is a small very fast memory (SRAM, expensive)
- contains copies of the most recently accessed memory locations (data and instructions): **temporal locality**
- is fully managed by hardware (unlike virtual memory)
- storage is organized in *blocks* of contiguous memory locations: **spatial locality**
- unit of transfer to/from main memory (or L2) is the cache block

• General Structure

- n **blocks** per cache organized in s **sets**
- b **bytes** per block
- total cache size $n*b$ **bytes**

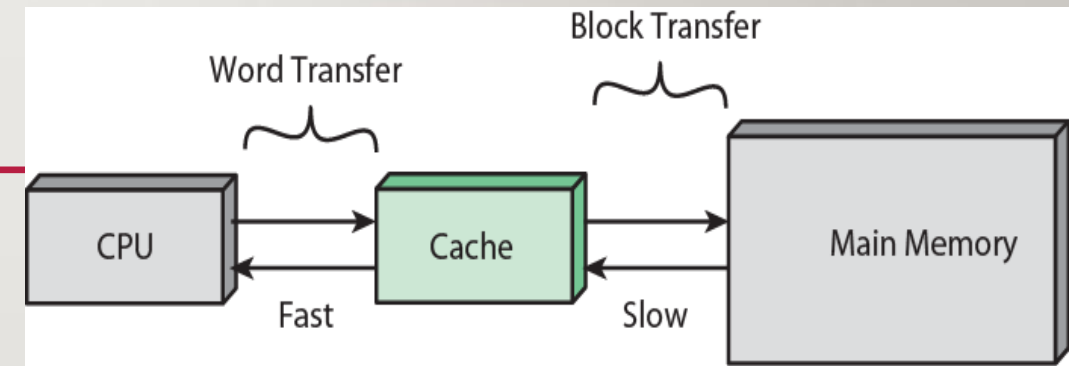
116 CACHE

- Small amount of fast memory
- Sits between normal main memory and CPU
- May be located on CPU chip or module
 - An entire blocks of data is copied from memory to the cache because the principle of locality tells us that once a byte is accessed, it is likely that a nearby data element will be needed soon.

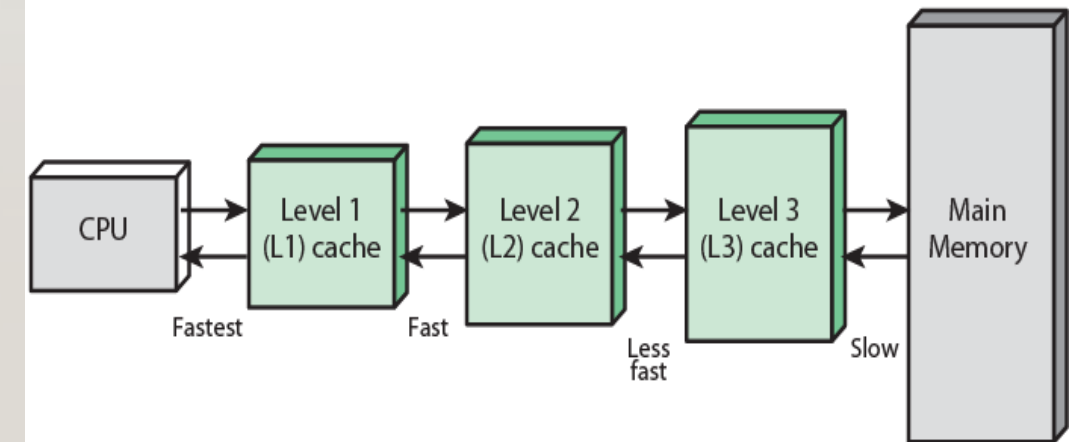
117 CACHE MEMORY PRINCIPLES

Cache memory is designed to **combine** :

- **Memory access time** of expensive, high speed memory combined with...
- ...the **large memory size** of less expensive, **lower-speed memory**.



(a) Single cache



(b) Three-level cache organization

118 CACHE MEMORY

Typically:

- Higher performance → higher cost;
- Lower performance → lower cost

CACHE STRUCTURE

119

3/16/2026

- Cache includes tags to identify the address of the block of main memory contained in a line of the cache
- Each word in main memory has a unique n -bit address
- There are $M=2^n/K$ blocks of K words in main memory, which means M is the total blocks in RAM
- Cache contains C lines of K words each plus a tag uniquely identifying the block of K words

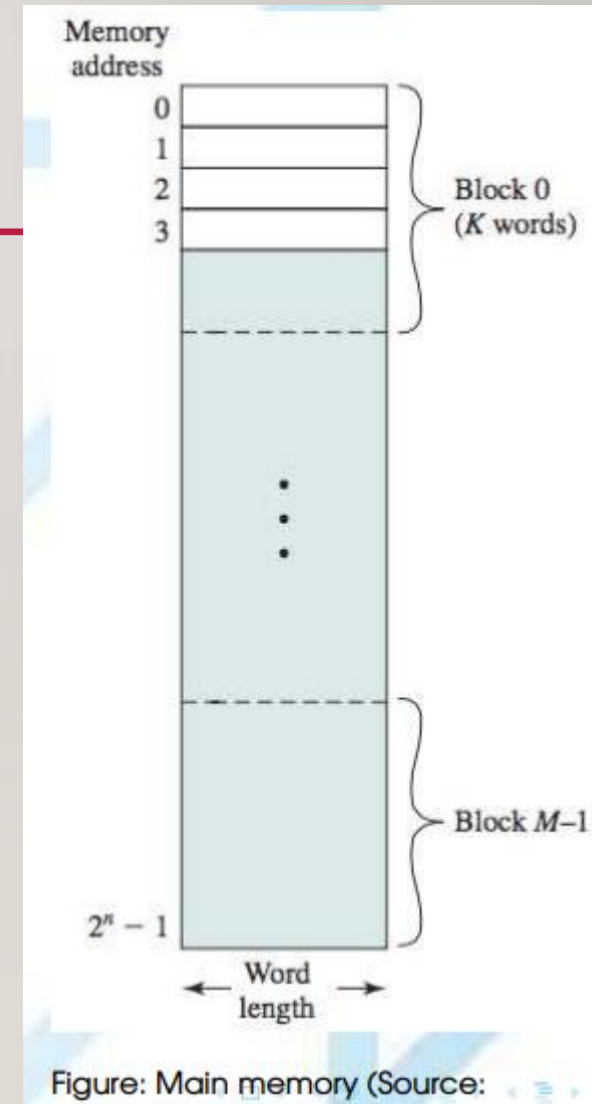
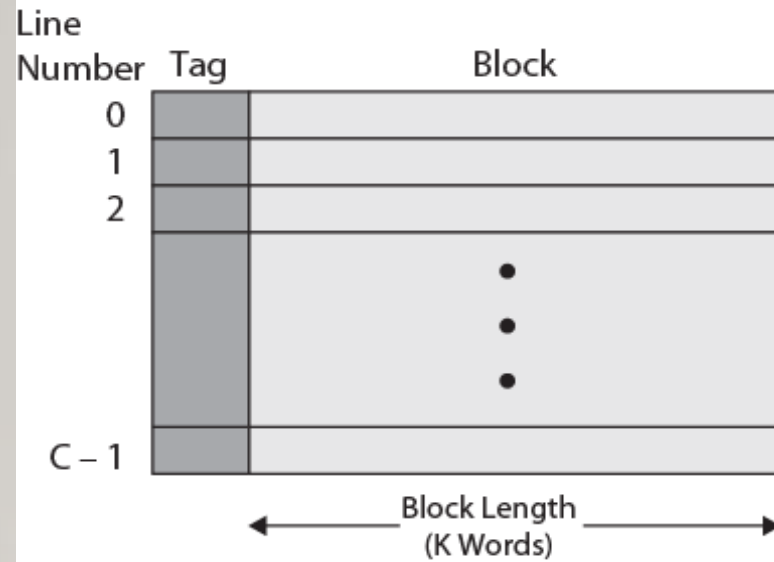


Figure: Main memory (Source: )

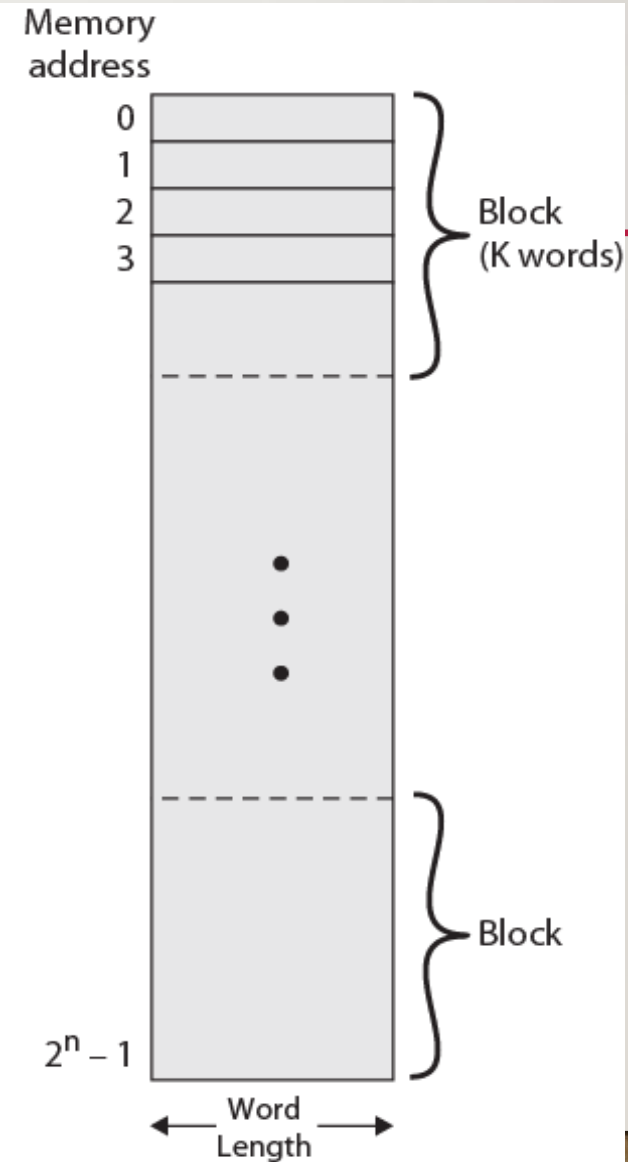
CACHE/MAIN MEMORY STRUCTURE

3/16/2026

I20



(a) Cache



(b) Main memory

CACHE OPERATION – OVERVIEW

121

- CPU requests contents of memory location
- Check cache for this data
- If present, get from cache (fast)
- If not present, read required block from main memory to cache
- Then deliver from cache to CPU
- Cache includes tags to identify which block of main memory is in each cache slot

CACHE OPERATION

3/16/2026

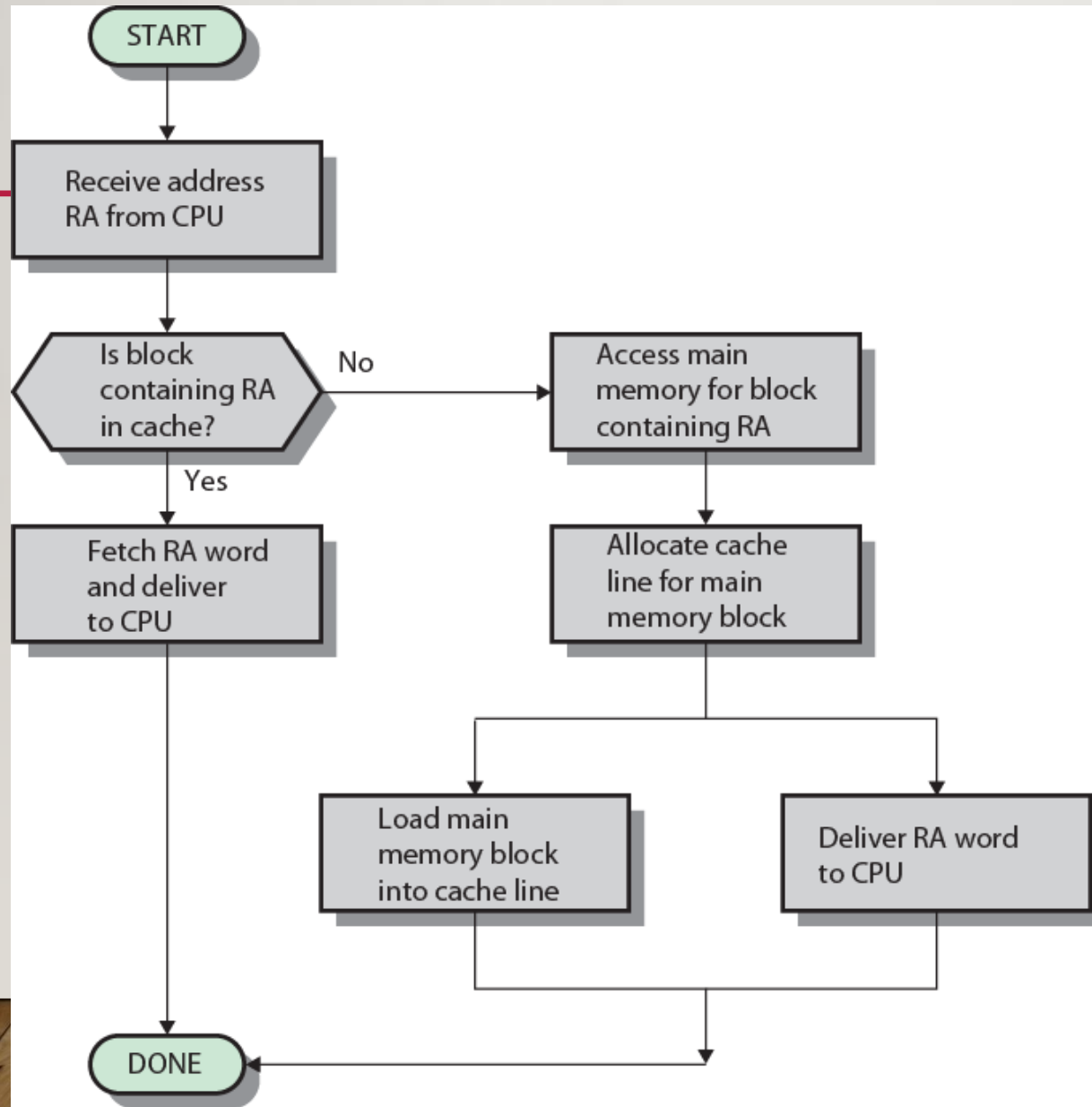
122

- **Read operation:**
 - Processor generates read address (RA) of word to be read;
 - If the word $\in$ cache, it is delivered to the processor;
- **Otherwise:**
 - Block containing that word is loaded into the cache;
 - Word is delivered to the processor;
 - These last two operations occurring in parallel.

CACHE READ OPERATION - FLOWCHART

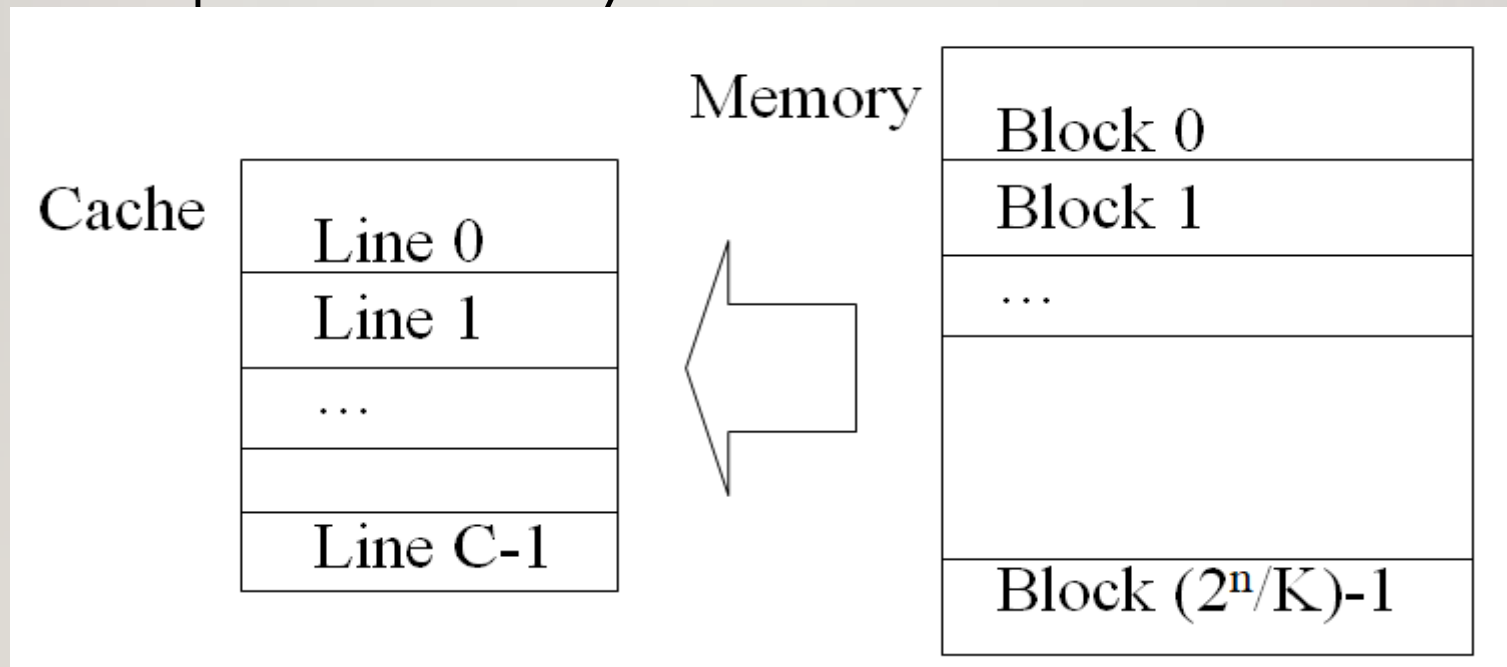
3/16/2026

123



CACHE DESIGN

- 124
- If memory contains 2^n addressable words
 - Memory can be broken up into blocks with K words per block. Number of blocks = $2^n / K$
-
- Cache consists of C **lines** or **slots**, each consisting of K words
 - $C \ll M$
 - How to map blocks of memory to lines in the cache?



I25

- **Addressing**
 - **Size**
 - **Mapping Function**
 - **Replacement Algorithm**
 - **Write Policy**
 - **Block Size**
 - **Number of Caches**
-

CACHE DESIGN

3/16/2026

Cache architectures can be classified according to key elements:

126

Cache Addresses	Write Policy
Logical	Write through
Physical	Write back
Cache Size	Line Size
Mapping Function	Number of Caches
Direct	Single or two level
Associative	Unified or split
Set associative	
Replacement Algorithm	
Least recently used (LRU)	
First in first out (FIFO)	
Least frequently used (LFU)	
Random	

Figure: Elements of cache design (Source: (Stallings, 2015))

CACHE ADDRESSING

3/16/2026

127 • Where does cache sit?

- Between processor and virtual memory management unit
- ~~• Between MMU and main memory~~
- Logical cache (virtual cache) stores data using virtual addresses
 - Processor accesses cache directly, not thorough physical cache
 - Cache access faster, before MMU address translation
 - Virtual addresses use same address space for different applications
 - Must flush cache on each context switch
- Physical cache stores data using main memory physical addresses
- Size does matter: Cost and Speed
 - More cache is expensive
 - More cache is faster (up to a point)
 - Checking cache for data takes time

CACHE ADDRESSING

3/16/2026

128

- There are two types of cache addresses:
- Physical addresses: Actual memory addresses;
- Logical addresses: Virtual-memory addresses;

Virtual memory performs mapping between:

- Logical addresses used by a program into physical addresses.
- Why is this important? We will see in a later chapter...

129 CACHE ADDRESSES

- Main idea behind virtual memory: Disregard amount of main memory available;
- Transparent transfers to/from: main memory and... ..secondary memory:
- Idea:
use RAM, when space runs out use HD ;
- requires a hardware memory management unit (MMU):
- to translate each virtual address into a physical address in main memory.

WITH VIRTUAL MEMORY CACHE MAY BE PLACED:

5/16/2026

130

- between the processor and the MMU;

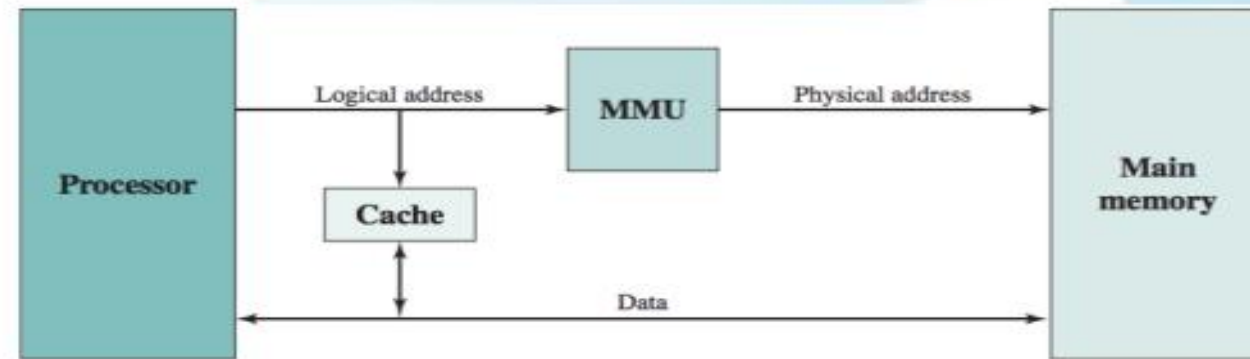


Figure: Virtual Cache (Source: (Stallings, 2015))

- between the MMU and main memory;

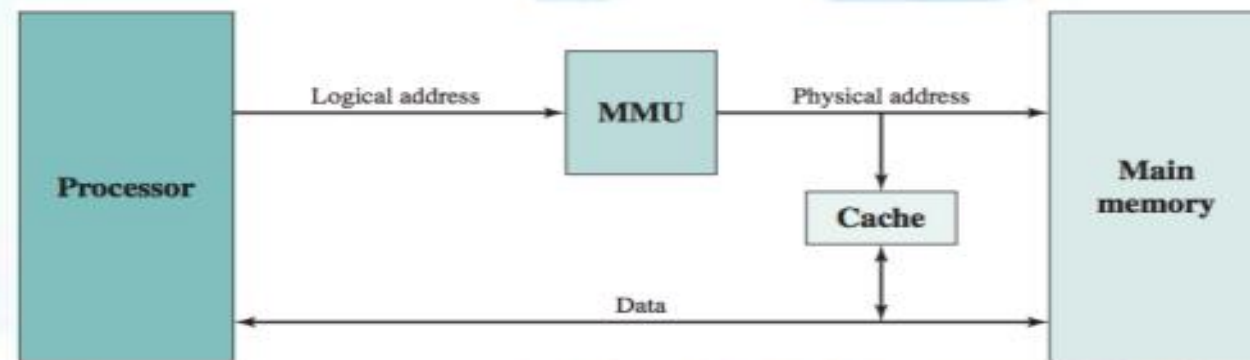


Figure: Physical Cache (Source: (Stallings, 2015))

CACHE ADDRESSING

3/16/2026

131

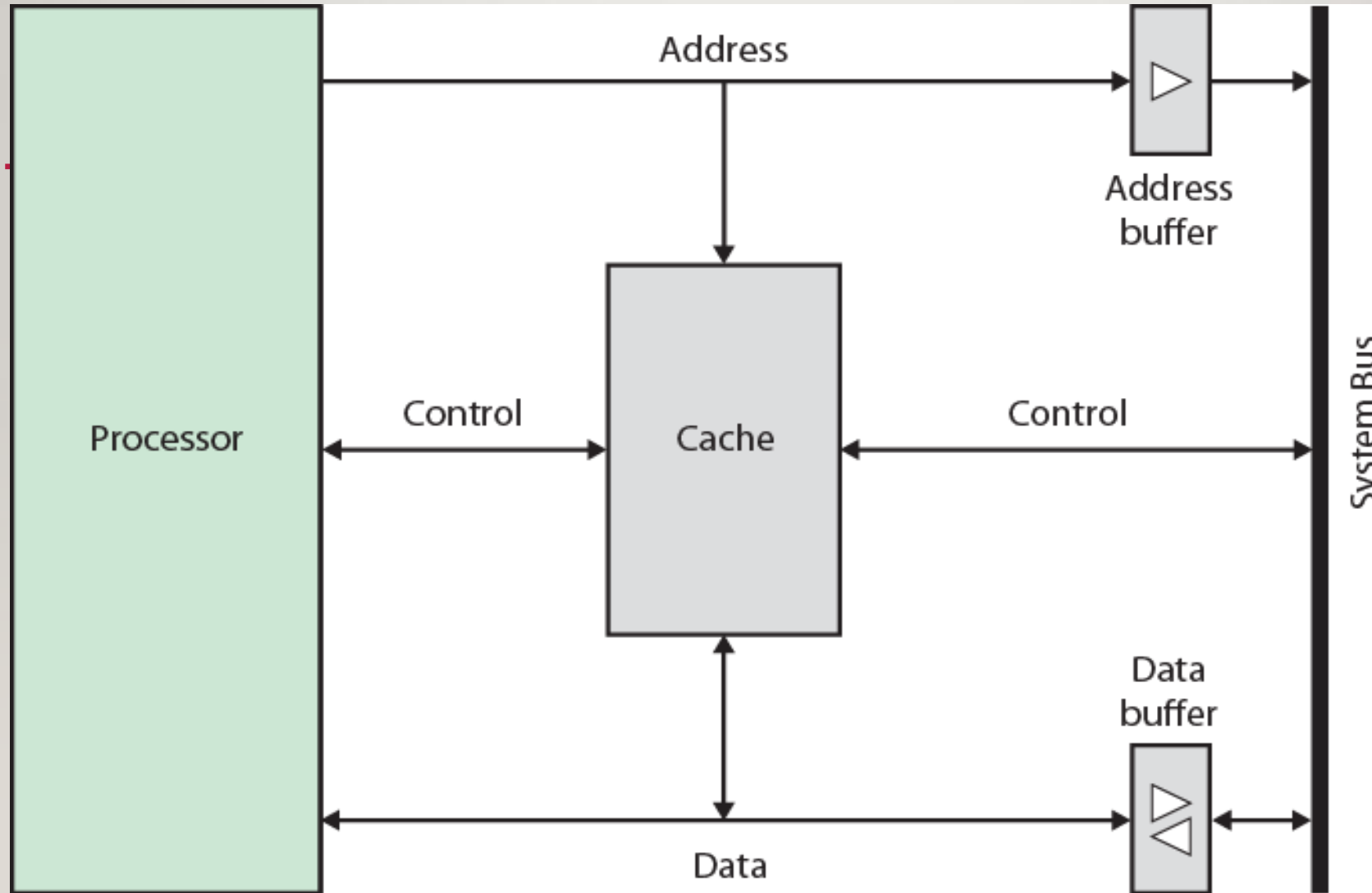
Virtual cache stores data using logical addresses.

- Processor accesses the cache directly, without going through the MMU.
- Advantage:
 - Faster access speed;
 - Cache can respond without the need for an MMU address translation;
- Disadvantage:
 - Same virtual address in two different applications refers to two different physical addresses;
 - Therefore cache must be flushed with each application context switch...
 - ...or extra bits must be added to each cache line
 - to identify which virtual address space this address refers to.

TYPICAL CACHE ORGANIZATION

3/16/2026

132



CACHE ORGANIZATION

3/16/2026

133

-
- In this organization the cache:
 - connects to the processor via data, control, and address lines;
 - the data and address lines also attach to data and address buffers:
 - which attach to a system bus...
 - ...from which main memory is reached.

CACHE ORGANIZATION

3/16/2026

134 When a cache hit occurs (word is in cache):

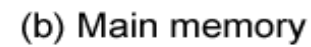
- the data and address buffers are disabled;
- communication is only between processor and cache;
- no system bus traffic.

When a cache miss occurs (word is not in cache):

- the desired address is loaded onto the system bus;
- the data are returned through the data buffer...
- ...to both the cache and the processor

3/16/2026

(a) Cache



136

Cache size should be:

- small enough so that the overall average cost per bit is close to that of main memory alone;
- large enough so that the overall average access time is close to that of the cache alone;

The larger the cache, the more complex the addressing logic:

- The result is that large caches tend to be slightly slower than small ones

The available chip and board area also limits cache size.

137

❖ Cache Size

The larger the cache , the larger the number of gates involve in addressing the cache. The result is that larger cache tend to be slightly lower than small ones---even when built with the same integrated circuit technology and put in the same place on chip and circuit board.

138 ❖ Mapping Function

- ❖ Because there are fewer cache lines than main memory blocks, an algorithm is needed for mapping main memory blocks into cache lines. Further a means is needed for determining which main memory block currently occupies a cache line. The choice of mapping function dictates how the cache is organized.

MAPPING FUNCTION

3/16/2026

139

- A mapping function is the method used to locate a memory address within a cache
- It is used when copying a block from main memory to the cache and it is used again when trying to retrieve data from the cache
- There are three kinds of mapping functions
 - Direct
 - Associative
 - Set Associative

CACHE EXAMPLE

3/16/2026

I 40

- These notes use an example of a cache to illustrate each of the mapping functions. The characteristics of the cache used are:
- Suppose we have the following configuration
 - Word size of 1 byte
 - Cache of 16 bytes
 - Cache line / Block size is 2 bytes
 - i.e. cache is $16/2 = 8$ (2^3) lines of 2 bytes per line
 - Will need 8 cache line addresses for a block in the cache
 - Main memory of 64 bytes
 - 6 bit address needed to reference 64 bytes
 - ($2^6 = 64$)
 - 64 bytes / 2 bytes-per-block $\rightarrow$ 32 Memory Blocks
 - Somehow we have to map the 32 memory blocks to the 8 lines in the cache. Multiple memory blocks will have to map to the same line in the cache!

MAPPING FUNCTION – 64K CACHE EXAMPLE

3/16/2026

- 141
- Suppose we have the following configuration
 - Word size of 1 byte
-
- Cache of 64 KByte
 - Cache line / Block size is 4 bytes
 - i.e. cache is $64 \text{ Kb} / 4 \text{ bytes} = 16,384 (2^{14})$ lines of 4 bytes
 - Main memory of 16MBytes
 - 24 bit address
 - ($2^{24} = 16\text{M}$)
 - $16\text{Mb} / 4\text{bytes-per-block} \rightarrow 4 \text{ Meg of Memory Blocks}$
 - Somehow we have to map the 4 Meg of blocks in memory onto the 16K of lines in the cache. Multiple memory blocks will have to map to the same line in the cache!

- 142 • Each block of main memory maps to only one cache line
 - i.e. if a block is in cache, it must be in one specific place

- Address is in two parts
 - Least Significant w bits identify unique word
 - Most Significant s bits specify which one memory block
- All the MSBs are split into
 - a cache line field r and
 - a tag of $s-r$ (most significant)

EXAMPLE DIRECT MAPPING FUNCTION

3/16/2026

- 143 • 16MBytes main memory
 - i.e. memory address is 24 bits

- ($2^{24}=16\text{M}$) bytes of memory
- Cache of 64k bytes with Cache block of 4 bytes
 - 64kb/4b i.e. cache is 16k
 - (2^{14}) lines of 4 bytes each
- Cache block of 4 bytes
 - i.e. block is 4 bytes
 - (2^2) bytes of data per block

EXAMPLE DIRECT MAPPING ADDRESS STRUCTURE

3/16/2026

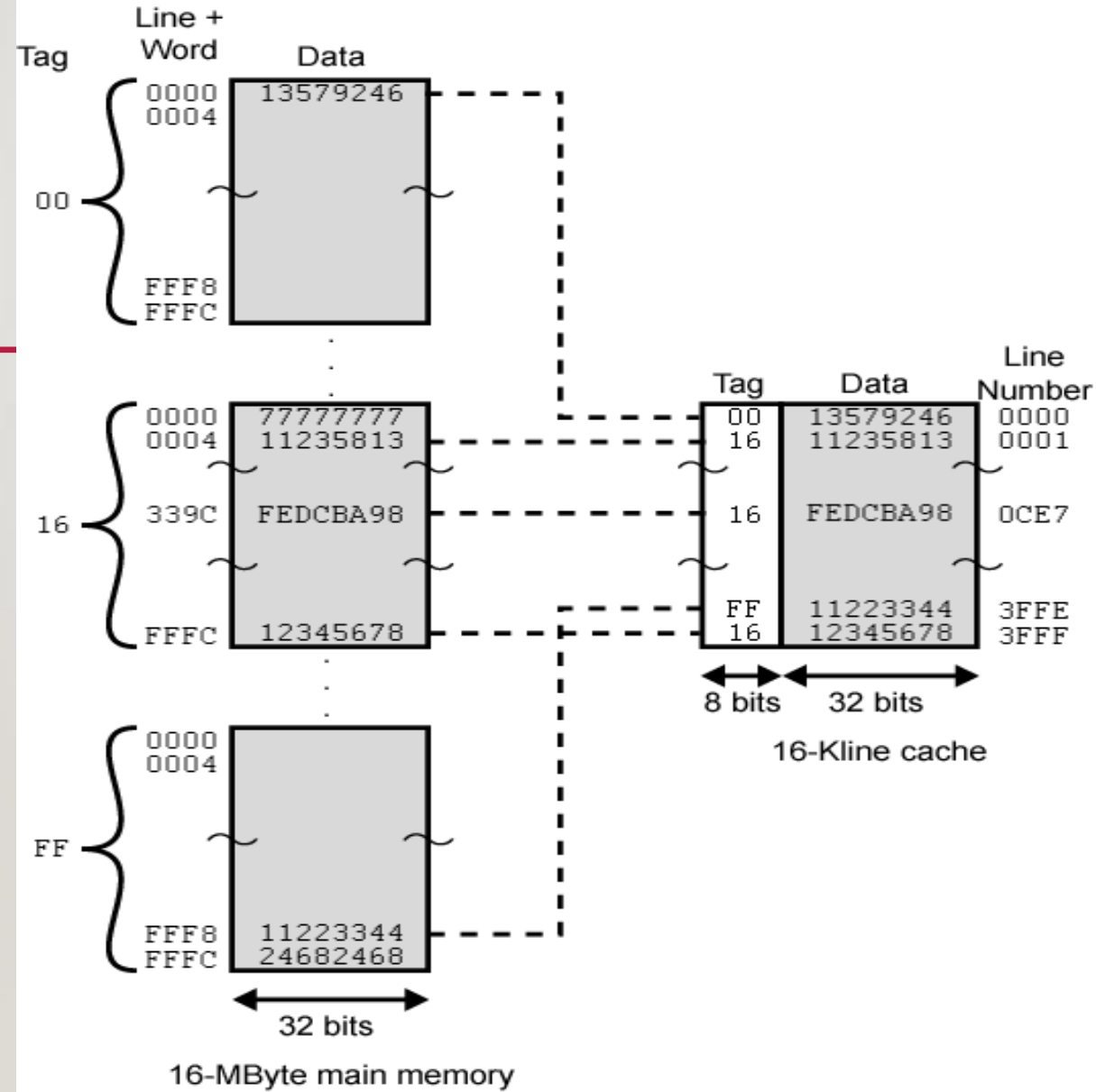
I44

Tag s-r	Line or Slot r	Word w
8	14	2

- 24 bit address
- 2 bit word identifier (4 byte block)
 - likely it would be wider
- 22 bit block identifier
 - 8 bit tag (=22-14)
 - 14 bit slot or line
- No two blocks sharing the same line have the same Tag field
- Check contents of cache by finding line and checking Tag

ILLUSTRATION OF EXAMPLE 145

3/16/2026



	Tag	Line	Word
Main memory address =	8	14	2

DIRECT MAPPING

3/16/2026

I46

- Simplest mapping technique - each block of main memory maps to only one cache line
 - i.e. if a block is in cache, it must be in one specific place
- Formula to map a memory block to a cache line:
 - $i = j \bmod c$
 - i =Cache Line Number
 - j =Main Memory Block Number
 - c =Number of Lines in Cache
 - i.e. we divide the memory block by the number of cache lines and the remainder is the cache line address

DIRECT MAPPING

3/16/2026

147

-
- The simplest cache mapping scheme is *direct mapped cache*.
 - In a direct mapped cache consisting of N blocks of cache, block X of main memory maps to cache block $Y = X \bmod N$.
 - Thus, if we have 10 blocks of cache, block 7 of cache may hold blocks 7, 17, 27, 37, . . . of main memory.
 - Once a block of memory is copied into its slot in cache, a *valid* bit is set for the cache block to let the system know that the block contains valid data.

DIRECT MAPPING

148

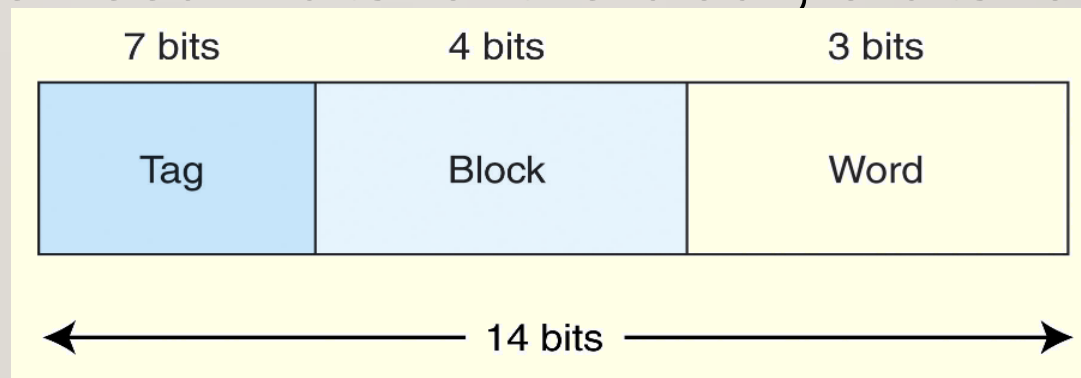
- The diagram below is a schematic of what cache looks like.

Block	Tag	Data	Valid
0	00000000	words A, B, C,...	1
1	11110101	words L, M, N,...	1
2	-----		0
3	-----		0

- Block 0 contains multiple words from main memory, identified with the tag 00000000. Block 1 contains words identified with the tag 11110101.
- The other two blocks are not valid.

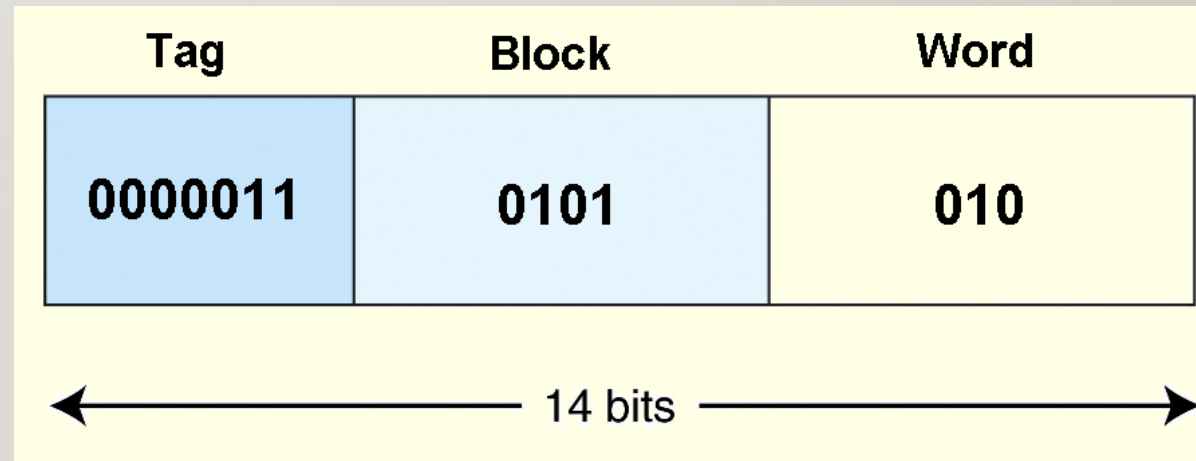
149 DIRECT MAPPING

- The size of each field into which a memory address is divided depends on the size of the cache.
- Suppose our memory consists of 2^{14} words, cache has $16 = 2^4$ blocks, and each block holds 8 words.
 - Thus memory is divided into $2^{14} / 2^3 = 2^{11}$ blocks.
- For our field sizes, we know we need 4 bits for the block, 3 bits for the word, and the tag is what's left over:



150 DIRECT MAPPING

- As an example, suppose a program generates the address **1AA**. In 14-bit binary, this number is: **00000110101010**.
- The first 7 bits of this address go in the tag field, the next 4 bits go in the block field, and the final 3 bits indicate the word within the block.



151 DIRECT MAPPING

- If subsequently the program generates the address **1AB**, it will find the data it is looking for in block **0101**, word **011**.

Tag	Block	Word
0000011	0101	010

- However, if the program generates the address, **3AB**, instead, the block loaded for address **1AA** would be evicted from the cache, and replaced by the blocks associated with the **3AB** reference.

DIRECT MAPPING

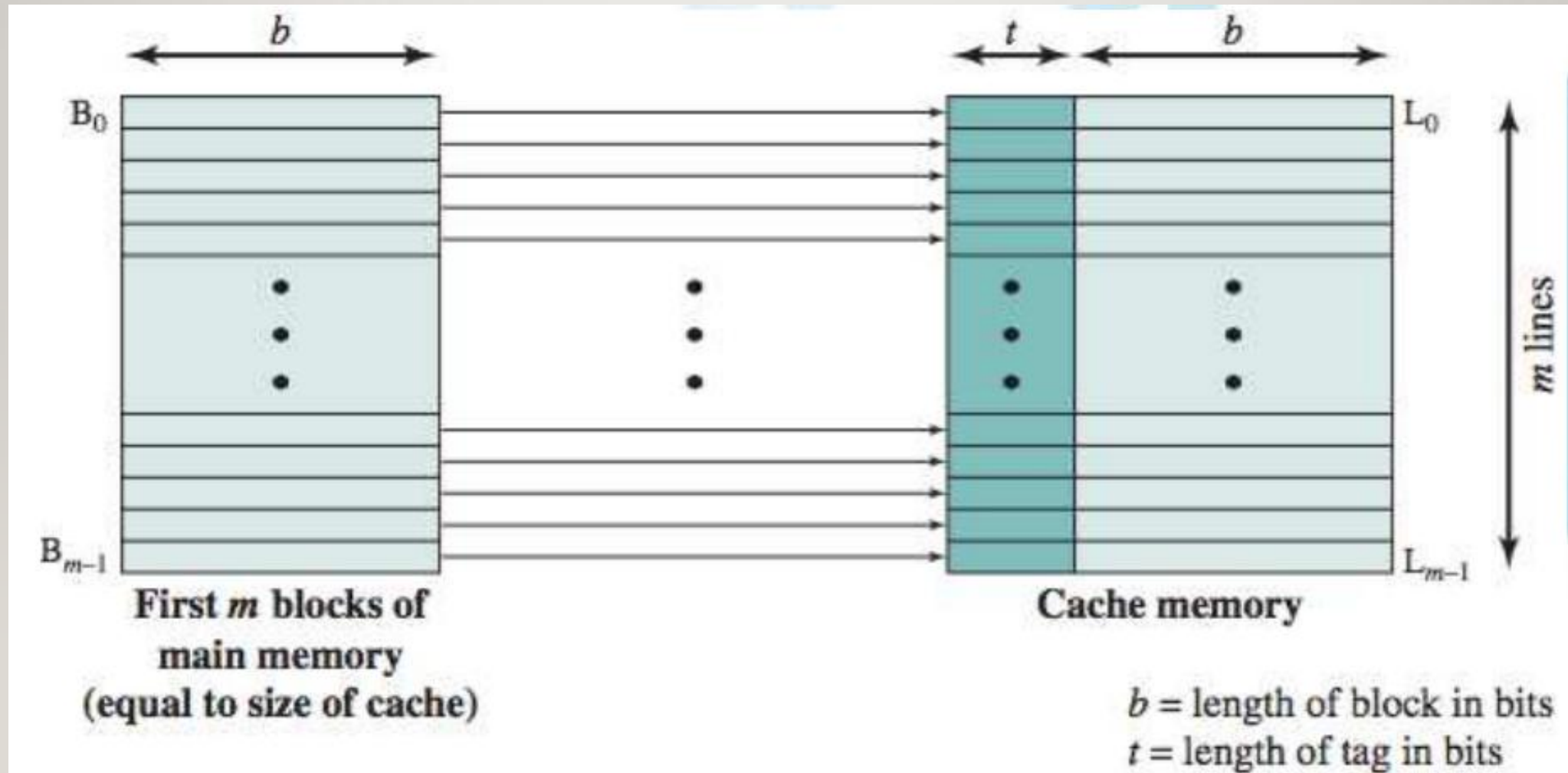
152

- Suppose a program generates a series of memory references such as: 1AB, 3AB, 1AB, 3AB, . . . The cache will continually evict and replace blocks.
- The theoretical advantage offered by the cache is lost in this extreme case.
- This is the main disadvantage of direct mapped cache.
- Other cache mapping schemes are designed to prevent this kind of thrashing.

DIRECT MAPPING

3/16/2026

153



154

- Previous picture shows mapping of main memory blocks into cache:
 - First m main memory blocks map into each line of the cache;
 - Next m blocks of main memory map in the following manner:
 - B_m maps into line L_0 of cache;
 - B_{m+1} maps into line L_1 ;
 - and so on...
 - Modulo operation implies repetitive structure;

155

With direct mapping blocks are assigned to lines as follows:

Cache line	Main memory blocks assigned
0	$0, m, 2m, \dots, 2^s - m$
1	$1, m + 1, 2m + 1, \dots, 2^s - m + 1$
$\vdots$	$\vdots$
$m - 1$	$m - 1, 2m - 1, 3m - 1, \dots, 2^s - 1$

Figure: (Source: (Stallings, 2015))

- Over time:
- each line can have a different main memory block;
- we need the ability to distinguish between these;
- the most significant $s - r$ bits (tag) serve this purpose.

DIRECT MAPPING WITH $C=4$

3/16/2026

156

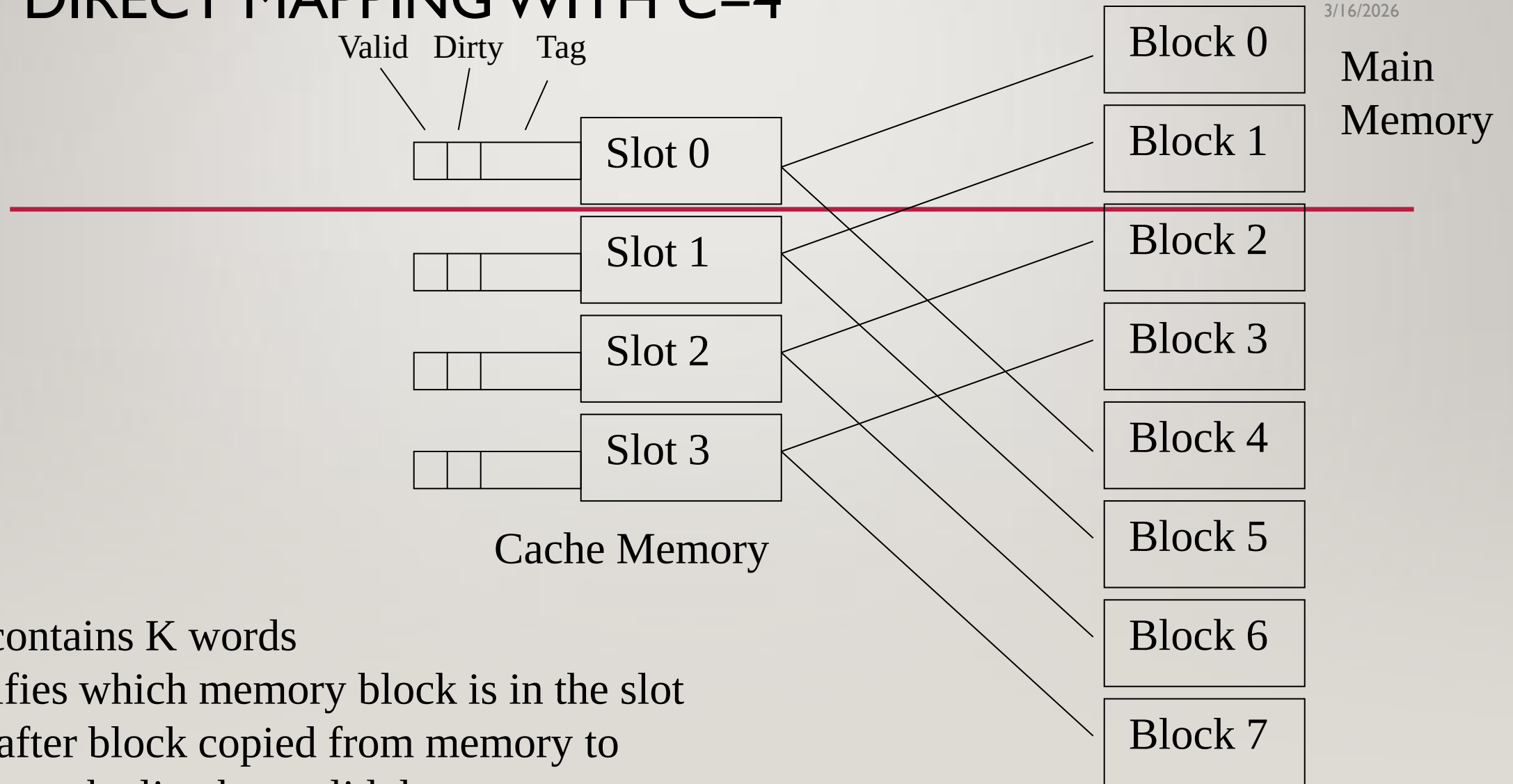
- Shrinking our example to a cache line size of 4 slots (each slot/line/block still contains 4 words):
-

Cache Line	Memory Block Held
• 0	0, 4, 8, ...
• 1	1, 5, 9, ...
• 2	2, 6, 10, ...
• 3	3, 7, 11, ...
• In general:	
• 0	0, C , $2C$, $3C$, ...
• 1	1, $C+1$, $2C+1$, $3C+1$, ...
• 2	2, $C+2$, $2C+2$, $3C+2$, ...
• 3	3, $C+3$, $2C+3$, $3C+3$, ...

DIRECT MAPPING WITH C=4

3/16/2026

157



Each slot contains K words

Tag: Identifies which memory block is in the slot

Valid: Set after block copied from memory to indicate the cache line has valid data

158

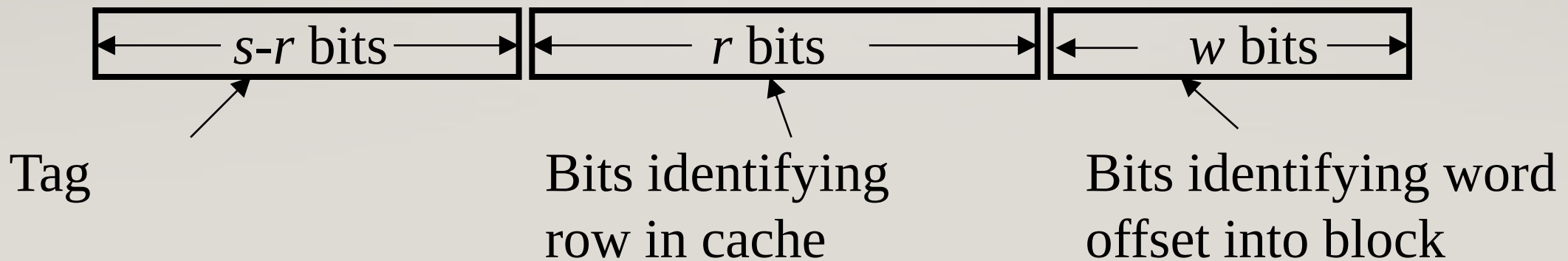
Each main **memory address** ($s + w$ bits) can be viewed as:

- **Offset** (w bits): identifies a word within a block of main memory;
- **Line** (r bits): specify one of the 2^r cache lines;
- **Tag** ($s - r$ bits): to distinguish blocks that are mapped to the same line:
 - Note that:
 - s allows addressing of all blocks in main memory...
 - ...whilst the number of available cache lines is much smaller than 2^s
 - Tag fields identifies the block that is currently mapped to a cache line;

DIRECT MAPPING

Each main memory address can be divided into three fields

- Least Significant w bits identify unique word within a block
- Remaining bits (s) specify which block in memory. These are divided into two fields
 - Least significant r bits of these s bits identifies which line in the cache
 - Most significant $s-r$ bits uniquely identifies the block within a line of the cache



DIRECT MAPPING ADDRESS STRUCTURE

3/16/2026

I60

- There is an easy way to compute

$$i = j \bmod c$$

based upon the bits in the address we are trying to access

- Address is in three parts
 - Least Significant w bits identify unique word within a cache line
 - w must be enough bits to address a specific word in a cache line; e.g. if 4 words per cache line, then we need 2 bits for w
 - Next Significant s bits specify which line this address maps into
 - s must be enough bits to address a specific line in the cache; e.g. if 16 cache lines, then we need 4 bits for r
 - Remaining t bits used as a tag to identify the memory block

Tag t	Line or Slot r	Word w
2	4	2

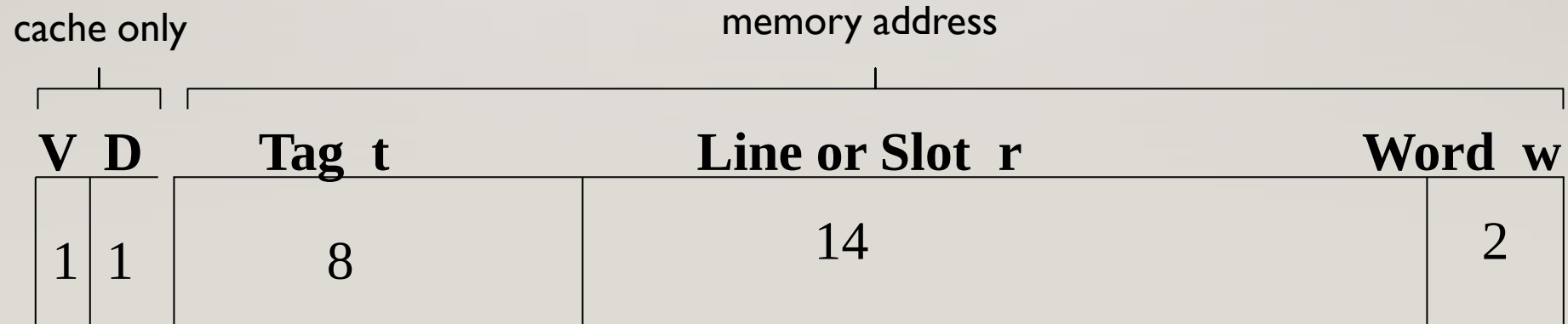
8 bit address

DIRECT MAPPING ADDRESS 64K CACHE EXAMPLE

3/16/2026

161

- Given a 24 bit address (to access 16Mb)
- 2 bit word identifier (4 byte block)
- Need 14 bits to address the cache slot/line
- Leaves 8 bits left for tag (=22-14)
- No two blocks in the same line have the same Tag field
- Check contents of cache by finding line and checking Tag
- Also need a Valid bit and a Dirty bit
 - Valid – Indicates if the slot holds a block belonging to the program being executed
 - Dirty – Indicates if a block has been modified while in the cache. Will need to be written back to memory before slot is reused for another block



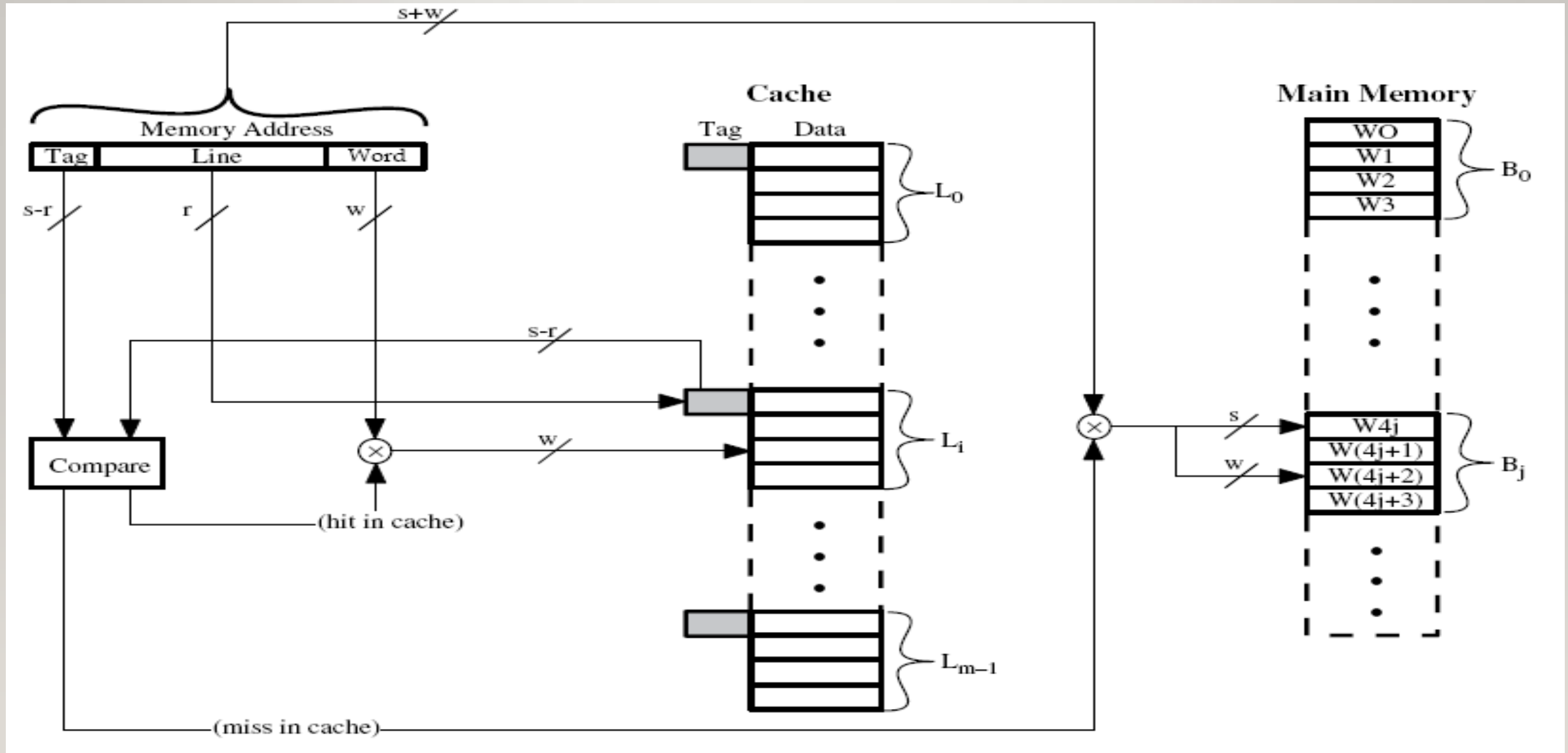
DIRECT MAPPING

- Why are the r -bits used to identify which line in cache?
- More likely to have unique r bits than $s-r$ bits based on principle of **locality of reference**

DIRECT MAPPING CACHE LINE TABLE

Cache line	Main Memory blocks held
0	$0, m, 2m, 3m \dots 2^s - m$
1	$1, m+1, 2m+1 \dots 2^s - m + 1$
$m-1$	$m-1, 2m-1, 3m-1 \dots 2^s - 1$

DIRECT MAPPING CACHE ORGANIZATION



DIRECT MAPPING

3/16/2026

165

To determine whether a block is in the cache:

- 1 Use the line field of the memory address to index the cache line;
- 2 Compare the tag from the memory address with the line tag;

1 If **both match**, then **Cache Hit**:

- 1 Use the line field of the memory address to index the cache line;
- 2 Retrieve the corresponding word from the cache line;

2 If **both do not match**, then **Cache Miss**:

- 1 Use the line field of the memory address to index the cache line;
- 2 Update the cache line (word + tag);

DIRECT MAPPING EXAMPLES

What cache line number will the following addresses be stored to, and what will the minimum address and the maximum address of each block they are in be if we have a cache with 4K lines of 16 words to a block in a 256 Meg memory space (24-bit address)?

Tag s-r	Line or slot r	Word w
8	14	2

MORE DIRECT MAPPING EXAMPLES

Assume that a portion of the tags in the cache in our example looks like the table below. Which of the following addresses are contained in the cache?

- a.) $438EE8_{16}$ b.) $F18EFF_{16}$ c.) $6B8EF3_{16}$ d.) $AD8EF3_{16}$

Tag (binary)	Line number (binary)	Addresses wi/ block			
		00	01	10	11
0101 0011	1000 1110 1110 10				
1110 1101	1000 1110 1110 11				
1010 1101	1000 1110 1111 00				
0110 1011	1000 1110 1111 01				
1011 0101	1000 1110 1111 10				
1111 0001	1000 1110 1111 11				

DIRECT MAPPING SUMMARY

- Address length = $(s + w)$ bits
- Number of addressable units = 2^{s+w} words or bytes
- Block size = line width = 2^w words or bytes
- Number of blocks in main memory = $2^{s+w}/2^w = 2^s$
- Number of lines in cache = $m = 2^r$
- Size of tag = $(s - r)$ bits

169 DIRECT MAPPING PROS & CONS

- Advantage:

 - Simple
 - Inexpensive
- Disadvantage:
- Fixed location for given block
 - If a program accesses 2 blocks that map to the same line repeatedly,
 - Then cache misses are very high – condition called **thrashing**

DIRECT MAPPING

3/16/2026

170

- Instead of placing memory blocks in specific cache locations based on memory address, we could allow a block to go anywhere in cache.
- In this way, cache would have to fill up before any blocks are evicted.
- This is how *fully associative* cache works.
- A memory address is partitioned into only two fields: the tag and the word.

ASSOCIATIVE MAPPING

3/16/2026

171

- A fully associative mapping scheme can overcome the problems of the direct mapping scheme
 - A main memory block can load into any line of cache
 - Memory address is interpreted as tag and word
 - Tag uniquely identifies block of memory
 - Every line's tag is examined for a match
 - Also need a Dirty and Valid bit
- But Cache searching gets expensive!
 - Ideally need circuitry that can simultaneously examine all tags for a match
 - Lots of circuitry needed, high cost
- Need replacement policies now that anything can get thrown out of the cache (will look at this shortly)

ASSOCIATIVE MAPPING

3/16/2026

172

Overcomes the disadvantage of direct mapping by:

- permitting each block to be loaded into any cache line:

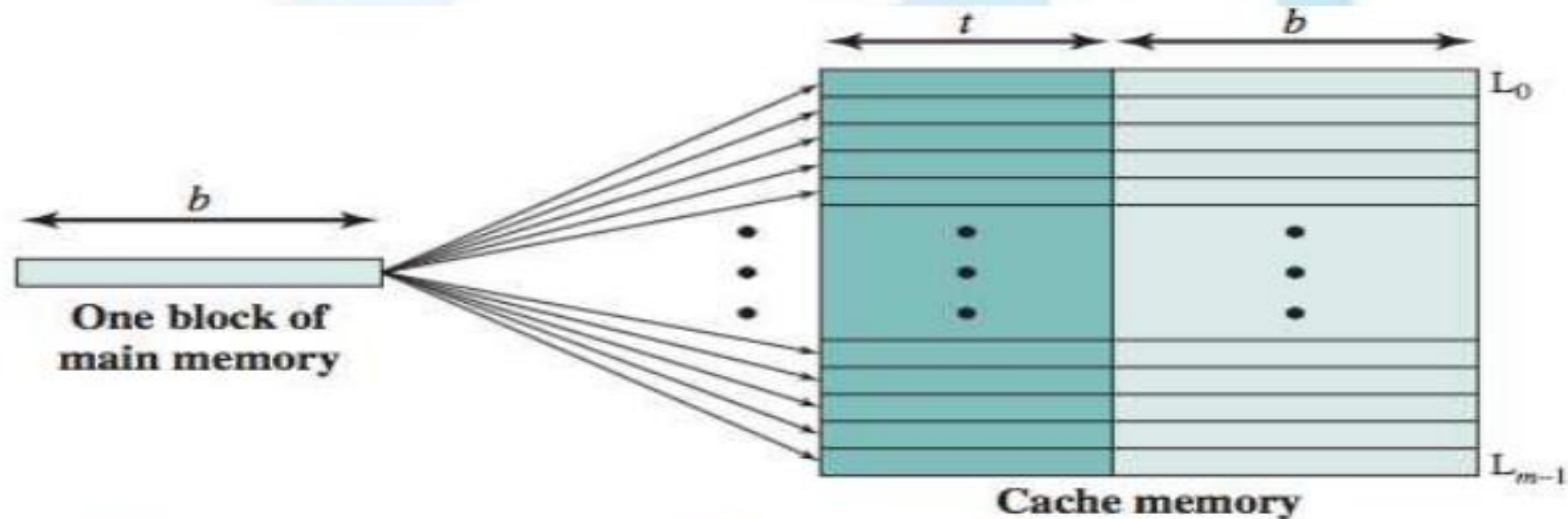


Figure: Associative Mapping (Source: (Stallings, 2015))

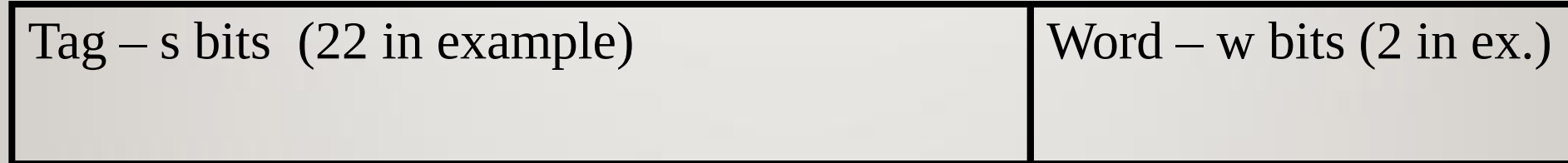
ASSOCIATIVE MAPPING

3/16/2026

173

- A main memory block can load into any line of cache
- Memory address is interpreted as:
 - Least significant w bits = word position within block
 - Most significant s bits = tag used to identify which block is stored in a particular line of cache
- Every line's tag must be examined for a match and Cache searching gets expensive and slower
- Cache interprets a memory address as a Tag and a Word field:
 - Tag: (s bits) uniquely identifies a block of main memory;
 - Word: (w bits) uniquely identifies a word within a block;

ASSOCIATIVE MAPPING ADDRESS STRUCTURE EXAMPLE



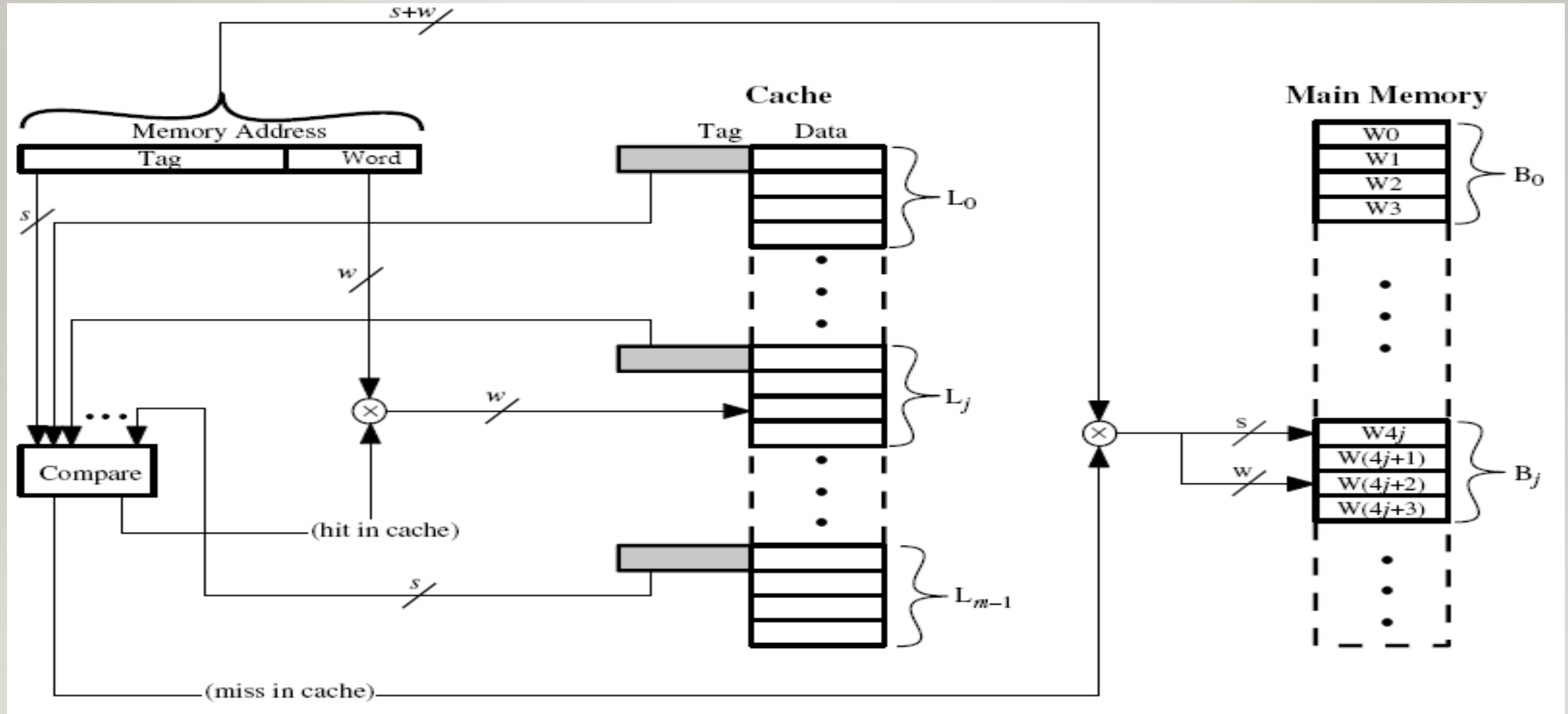
- 22 bit tag stored with each 32 bit block of data
- Compare tag field with tag entry in cache to check for hit
- Least significant 2 bits of address identify which of the four 8 bit words is required from 32 bit data block

ASSOCIATIVE MAPPING 64K CACHE EXAMPLE

Tag 22 bit	Word 2 bit
------------	---------------

- 22 bit tag stored with each slot in the cache – no more bits for the slot line number needed since all tags searched in parallel
- Compare tag field of a target memory address with tag entry in cache to check for hit
- Least significant 2 bits of address identify which word is required from the block, e.g.:
 - Address: FFFFFC = 1111 1111 1111 1111 1111 1100
 - Tag: Left 22 bits, truncate on left:
 - 11 1111 1111 1111 1111 1111
 - 3FFFFFF
 - Address: 16339C = 0001 0110 0011 0011 1001 1100
 - Tag: Left 22 bits, truncate on left:
 - 00 0101 1000 1100 1110 0111
 - 058CE7

ASSOCIATIVE CACHE MAPPING ORGANIZATION



ASSOCIATIVE CACHE MAPPING ORGANIZATION

3/16/2026

177

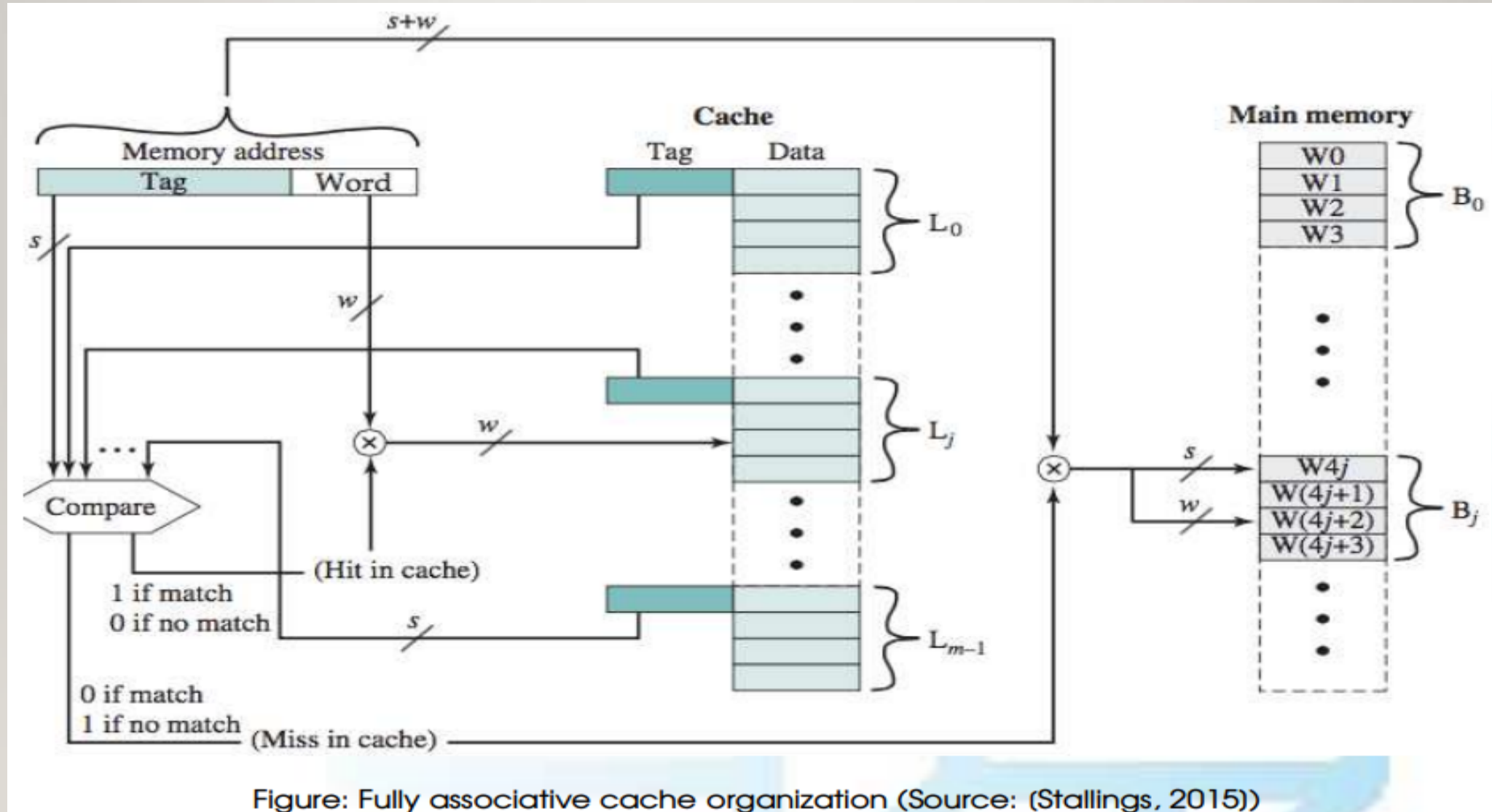


Figure: Fully associative cache organization (Source: (Stallings, 2015))

ASSOCIATIVE MAPPING

3/16/2026

178

- To determine whether a block is in the cache:
 - simultaneously compare every line's tag for a match:
 - If a match exists, then Cache Hit:
 - 1 Use the tag field of the memory address to index the cache line;
 - 2 Retrieve the corresponding word from the cache line;
 - If a match does not exist, then Cache Miss:
 - 1 Choose a cache line. How?
 - 2 Update the cache line (word + tag);

ASSOCIATIVE MAPPING EXAMPLE

Assume that a portion of the tags in the cache in our example looks like the table below.

Which of the following addresses are contained in the cache?

a.) $438EE8_{16}$

b.) $F18EFF_{16}$

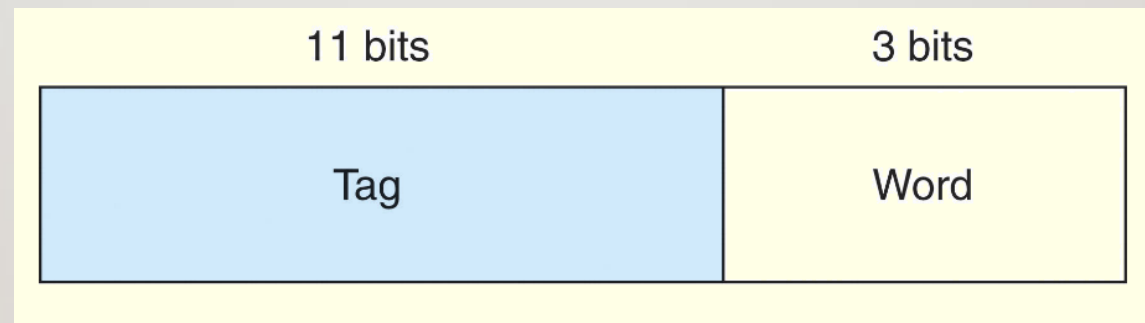
c.) $6B8EF3_{16}$

d.) $AD8EF3_{16}$

Tag (binary)	Addresses wi/ block			
	00	01	10	11
0101 0011 1000 1110 1110 10				
1110 1101 1100 1001 1011 01				
1010 1101 1000 1110 1111 00				
0110 1011 1000 1110 1111 11				
1011 0101 0101 1001 0010 00				
1111 0001 1000 1110 1111 11				

180 ASSOCIATIVE MAPPING EXAMPLE

- Suppose, we have 14-bit memory addresses and a cache with 16 blocks, each block of size 8. The field format of a memory reference is:



- When the cache is searched, all tags are searched in parallel to retrieve the data quickly.
- This requires special, costly hardware.

181 ASSOCIATIVE MAPPING

- You will recall that direct mapped cache evicts a block whenever another memory reference needs that block.
- With fully associative cache, we have no such mapping, thus we must devise an algorithm to determine which block to evict from the cache.
- The block that is evicted is the *victim block*.
- There are a number of ways to pick a victim, we will discuss them set associative cache mapping.

182

- Associative mapping technique:
 - Advantage:
flexibility as to which block to replace when a new block is read into the cache;
 - Disadvantage:
complex circuitry required to examine the tags of all cache lines in parallel.

SET ASSOCIATIVE MAPPING

3/16/2026

183

-
- The Idea behind: Perform less comparisons
 - Instead of comparing the tag against all lines
 - compare only against a subset of the cache lines.
 - Welcome to set-associative mapping →

184

SET ASSOCIATIVE MAPPING

- Compromise between fully-associative and direct-mapped cache
 - Cache is divided into a number of sets
 - Each set contains a number of lines
 - A given block maps to any line in a specific set
 - Use direct-mapping to determine which set in the cache corresponds to a set in memory
 - Memory block could then be in any line of that set
 - e.g. 2 lines per set
 - 2 way associative mapping
 - A given block can be in either of 2 lines in a specific set
 - e.g. K lines per set
 - K way associative mapping
 - A given block can be in one of K lines in a specific set
 - Much easier to simultaneously search one set than all lines

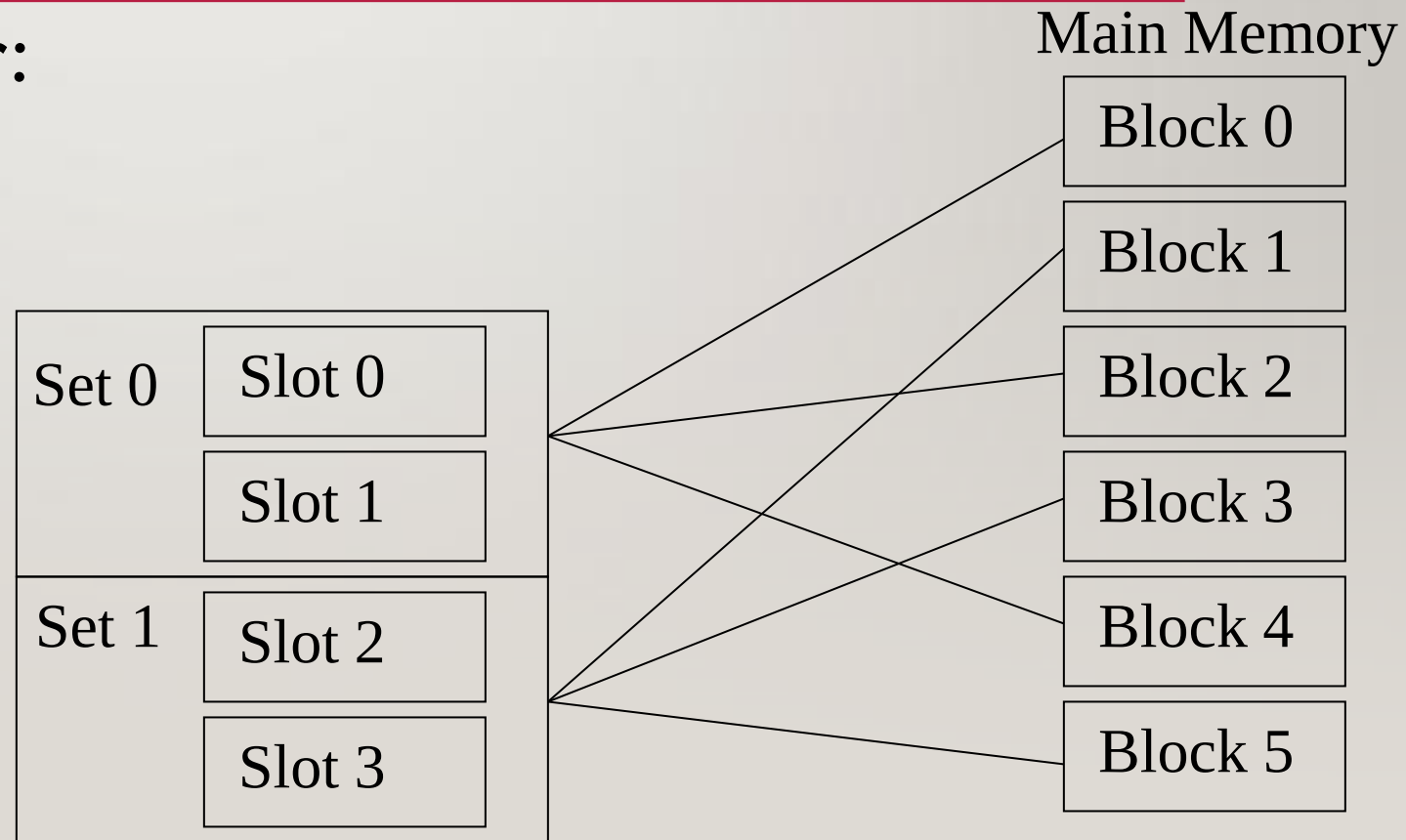
185 SET ASSOCIATIVE MAPPING

- Set associative cache combines the ideas of direct mapped cache and associative mapped cache.
- An N -way set associative cache mapping is like direct mapped cache in that a memory reference maps to a particular location in cache.
- Unlike direct mapped cache, a memory reference maps to a set of several cache blocks, similar to the way in which fully associative cache works.
- Instead of mapping anywhere in the entire cache, a memory reference can map only to the subset of cache slots.

186 SET ASSOCIATIVE MAPPING

- To compute cache set number:

- $\text{SetNum} = j \bmod v$
 - j = main memory block number
 - v = number of sets in cache



SET ASSOCIATIVE MAPPING 64K CACHE EXAMPLE

3/16/2026

187

Tag 9 bit	Set 13 bit	Word 2 bit
-----------	------------	------------

- E.g. Given our 64Kb cache, with a line size of 4 bytes, we have 16384 lines. Say that we decide to create 8192 sets, where each set contains 2 lines. Then we need 13 bits to identify a set ($2^{13}=8192$)
- Use set field to determine cache set to look in
- Compare tag field of all slots in the set to see if we have a hit, e.g.:
 - Address = 16339C = 0001 0110 0011 0011 1001 1100
 - Tag = 0 0010 1100 = 02C
 - Set = 0 1100 1110 0111 = 0CE7
 - Word = 00 = 0
 - Address = 008004 = 0000 0000 1000 0000 0000 0100
 - Tag = 0 0000 0001 = 001
 - Set = 0 0000 0000 0001 = 0001
 - Word = 00 = 0

188 SET ASSOCIATIVE MAPPING

- Combination of direct and associative approaches:
 - Cache consists of a number of sets, each consisting of a number of lines.
- From direct mapping:
 - each block can only be mapped into a single set;
 - I.e. Block B_j always maps to set j ;
 - Done in a modulo way $=)$
- From associative mapping:
 - each block can be mapped into any cache line of a certain set.

SET ASSOCIATIVE MAPPING

189

$$m = v \times k$$

$$i = j \bmod v$$

where:

- i = cache set number;
- j = main memory block number;
- m = number of lines in the cache;
- v = number of sets;
- k = number of lines in each set

SET ASSOCIATIVE MAPPING

3/16/2026

190

Cache interprets a memory address as a Tag, a Set and a Word field:

- Set: identifies a set (d bits, $v = 2^d$ sets);
- Tag: used in conjunction with the set bits to identify a block ($s - d$ bits);
- Word: identifies a word within a block;

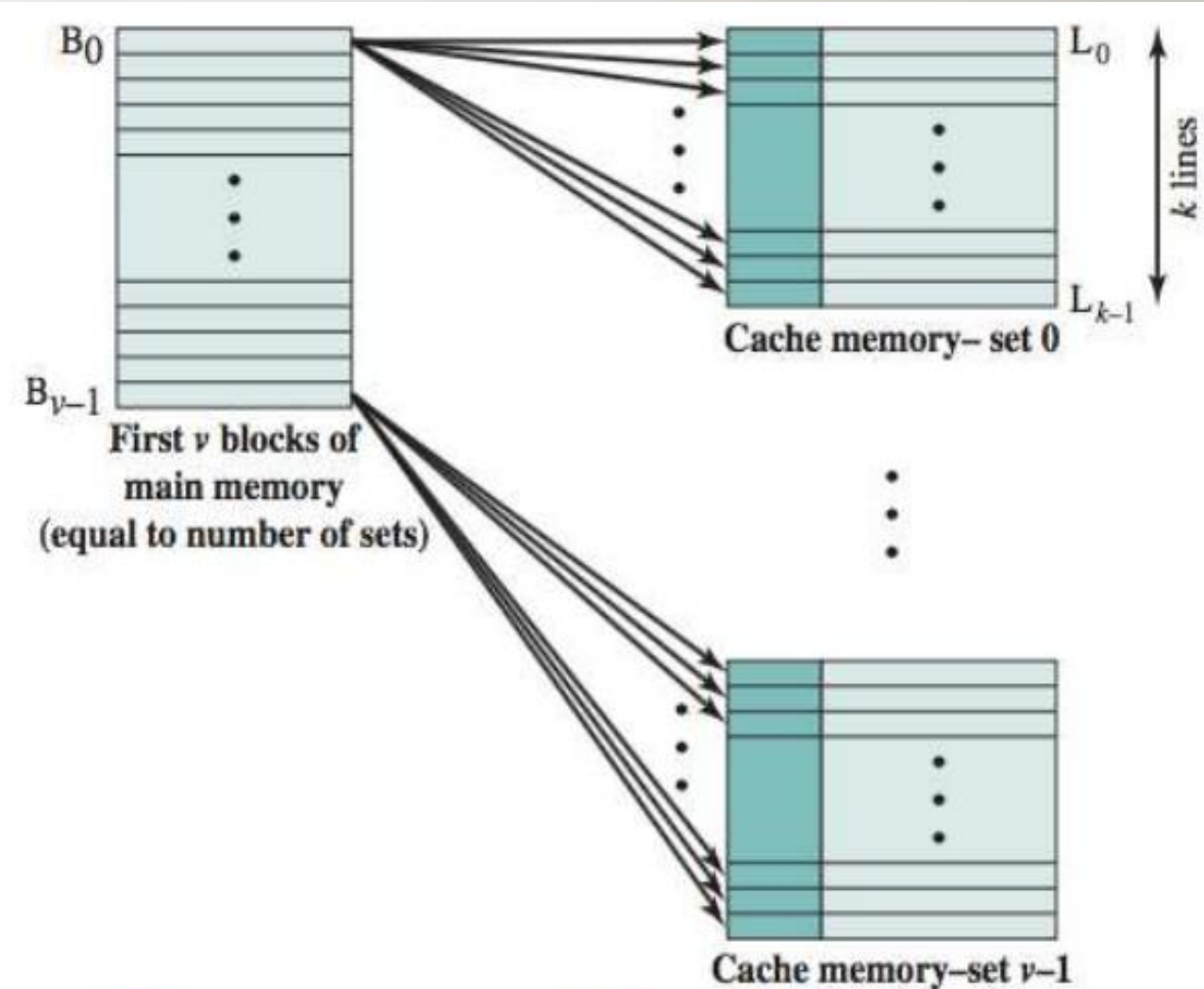
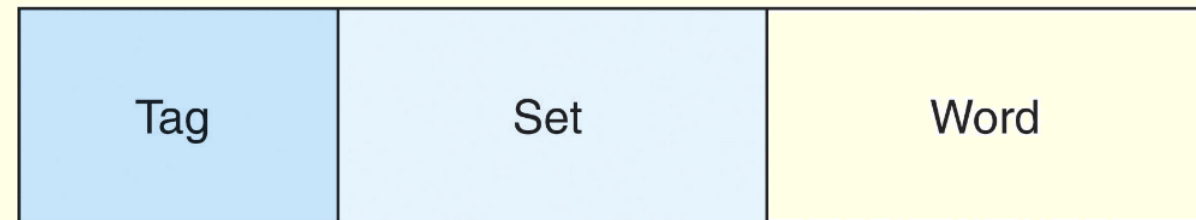


Figure: v associative mapped caches (Source: (Stallings, 2015))

191 SET ASSOCIATIVE MAPPING

- In set associative cache mapping, a memory reference is divided into three fields: tag, set, and word, as shown below.
- As with direct-mapped cache, the word field chooses the word within the cache block, and the tag field uniquely identifies the memory address.
- The set field determines the set to which the memory block maps.



192 SET ASSOCIATIVE MAPPING EXAMPLE

- The number of cache blocks per set in set associative cache varies according to overall system design.
- For example, a 2-way set associative cache can be conceptualized as shown in the schematic below.
- Each set contains two different memory blocks.

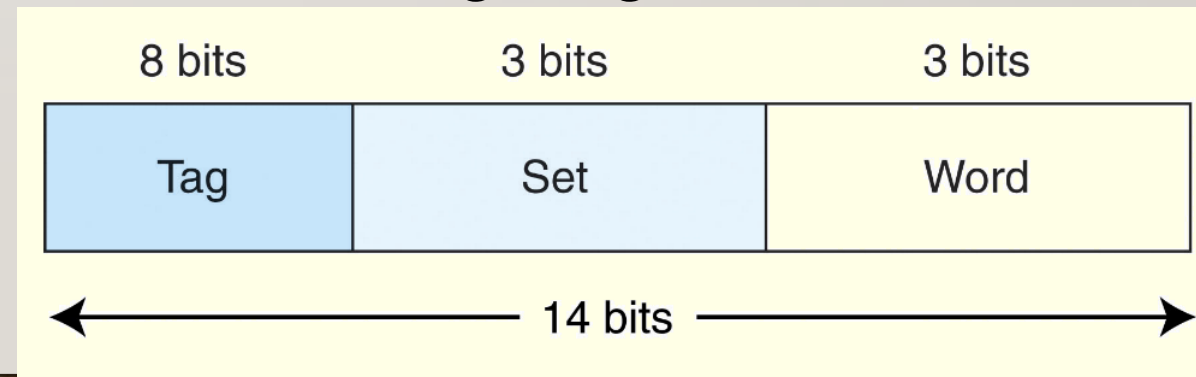
Set	Tag	Block 0 of set	Valid	Tag	Block 1 of set	Valid
0	00000000	Words A, B, C, . . .	1	-----		0
1	11110101	Words L, M, N, . . .	1	-----		0
2	-----		0	10111011	P, Q, R, . . .	1
3	-----		0	11111100	T, U, V, . . .	1

SET ASSOCIATIVE MAPPING EXAMPLE

3/16/2026

193

- Suppose we have a main memory of 2^{14} bytes.
- This memory is mapped to a 2-way set associative cache having 16 blocks where each block contains 8 words.
- Since this is a 2-way cache, each set consists of 2 blocks, and there are 8 sets.
- Thus, we need 3 bits for the set, 3 bits for the word, giving 8 leftover bits for the tag:



SET ASSOCIATIVE MAPPING

3/16/2026

194

- To determine whether a block is in the cache:

- 1 Determine the set through the set fields;
- 2 Compare address tag simultaneously with all cache line tags;
- 3 If a match exists, then Cache Hit:
 - 1 Retrieve the corresponding word from the cache line;
- 4 If a match does not exist, then Cache Miss:
 - 1 Choose a cache line within the set. How?
 - 2 Update the cache line (word + tag);

195

- To determine whether a block is in the cache:

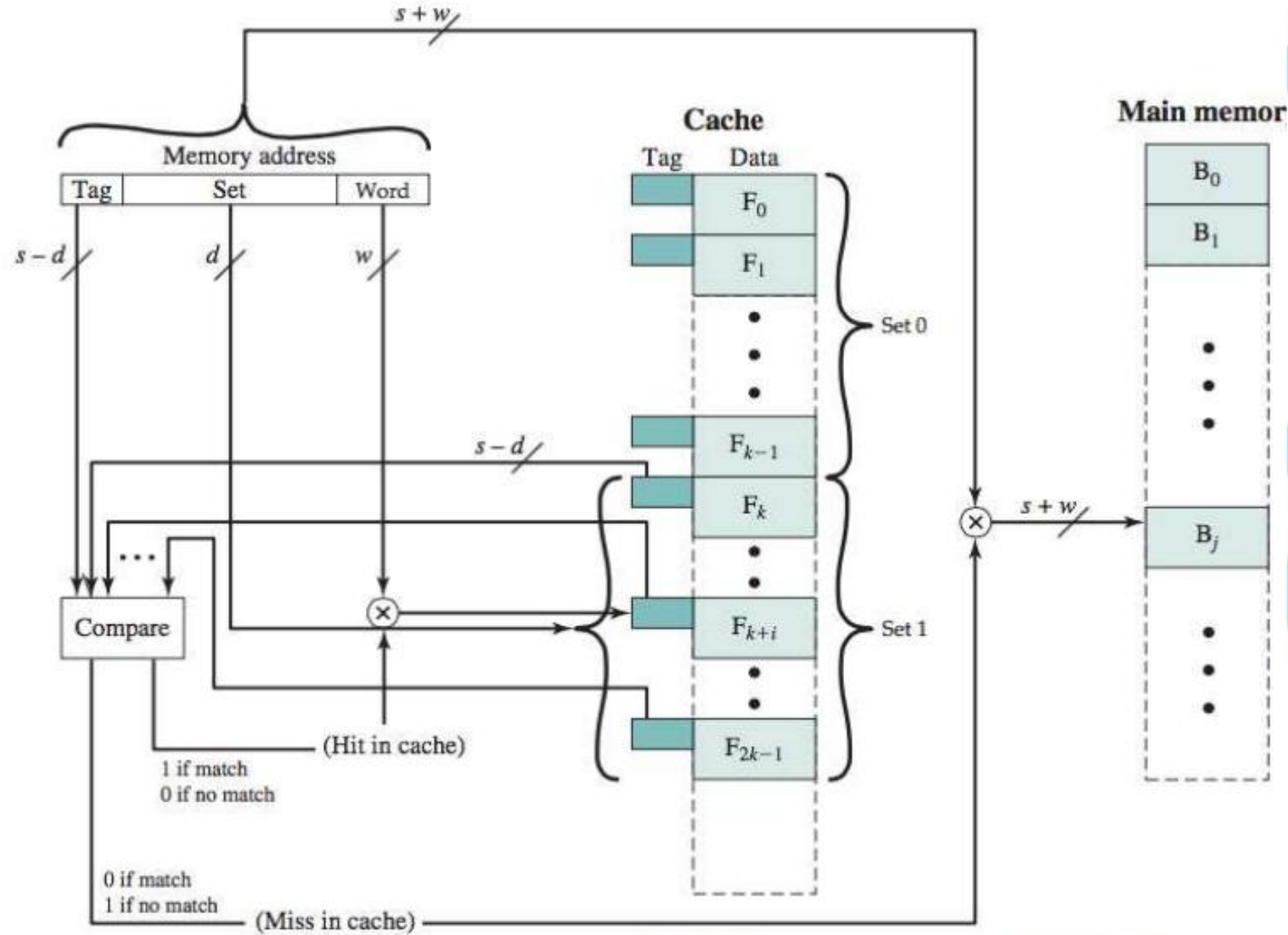


Figure: K-Way Set Associative Cache Organization (Source: (Stallings, 2015))

SUMMARY OF SET ASSOCIATIVE MAPPING

3/16/2026

196

-
- Address length is $s + w$ bits
 - Cache is divided into a number of sets, $v = 2^d$
 - k blocks/lines can be contained within each set
 - k lines in a cache is called a k -way set associative mapping
 - Number of lines in a cache = $v \cdot k = k \cdot 2^d$
 - Size of tag = $(s-d)$ bits

SUMMARY OF SET ASSOCIATIVE MAPPING

3/16/2026

197

- Hybrid of Direct and Associative
 $k = 1$, this is basically direct mapping
 $v = 1$, this is associative mapping
- Each set contains a number of lines, basically the number of lines divided by the number of sets
- A given block maps to any line within its specified set – e.g. Block B can be in any line of set i.
- 2 lines per set is the most common organization.
 - Called 2 way associative mapping
 - A given block can be in one of 2 lines in only one specific set
 - Significant improvement over direct mapping

SUMMARY OF SET ASSOCIATIVE MAPPING

3/16/2026

198

-
- Let's go to two-way set associative mapping
 - Divides the 16K lines into 8K sets
 - This requires a 13 bit set number
 - With 2 word bits, this leaves 9 bits for the tag
 - Blocks beginning with the addresses 000000_{16} , 008000_{16} , 010000_{16} , 018000_{16} , 020000_{16} , 028000_{16} , etc. map to the same set, Set 0.
 - Blocks beginning with the addresses 000004_{16} , 008004_{16} , 010004_{16} , 018004_{16} , 020004_{16} , 028004_{16} , etc. map to the same set, Set 1.

SET ASSOCIATIVE MAPPING EXAMPLE

3/16/2026

199

- For each of the following addresses, answer the following questions based on a 2-way set associative cache with 4K lines, each line containing 16 words, with the main memory of size 256 Meg memory space (28-bit address):
- What cache set number will the block be stored to?
- What will their tag be?
- What will the minimum address and the maximum address of each block they are in be?

1. 9ABCDEF₁₆

2. 1234567₁₆

Tag s-r	Set s	Word w
13	11	4

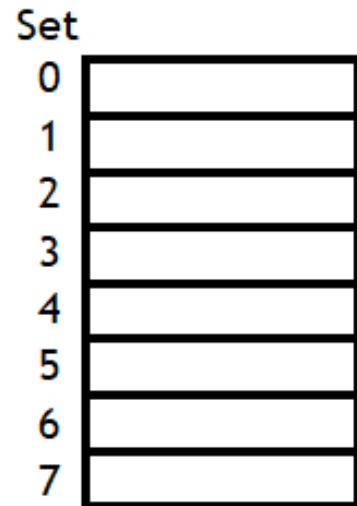
Set associative caches are a general idea

6/2026

200

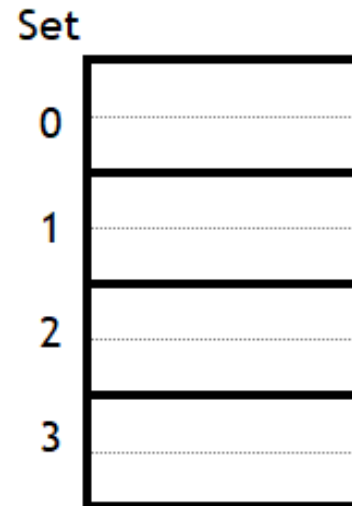
- By now you may have noticed the 1-way set associative cache is the same as a **direct-mapped** cache.
- Similarly, if a cache has 2^k blocks, a 2^k -way set associative cache would be the same as a **fully-associative** cache.

1-way
8 sets,
1 block each



direct mapped

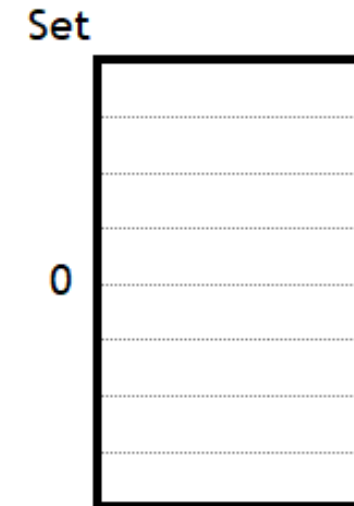
2-way
4 sets,
2 blocks each



4-way
2 sets,
4 blocks each



8-way
1 set,
8 blocks



fully associative

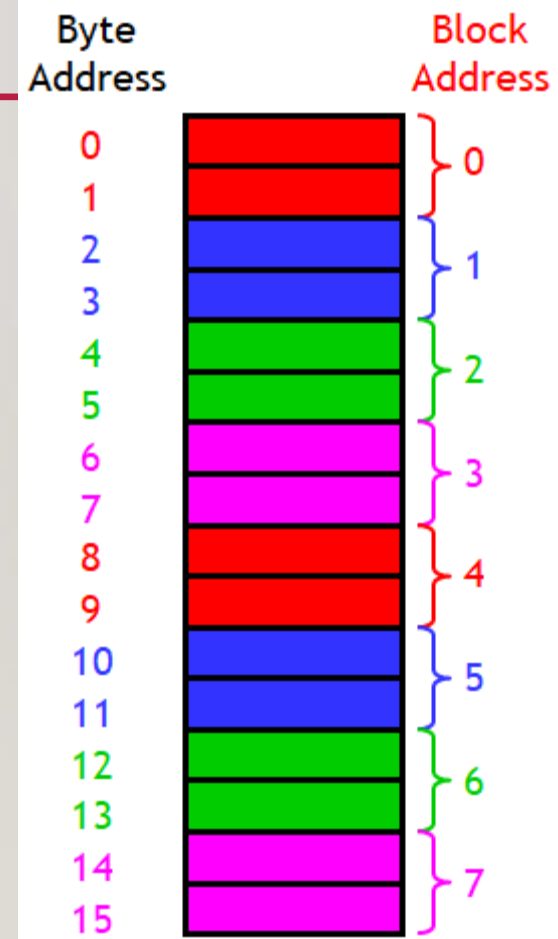
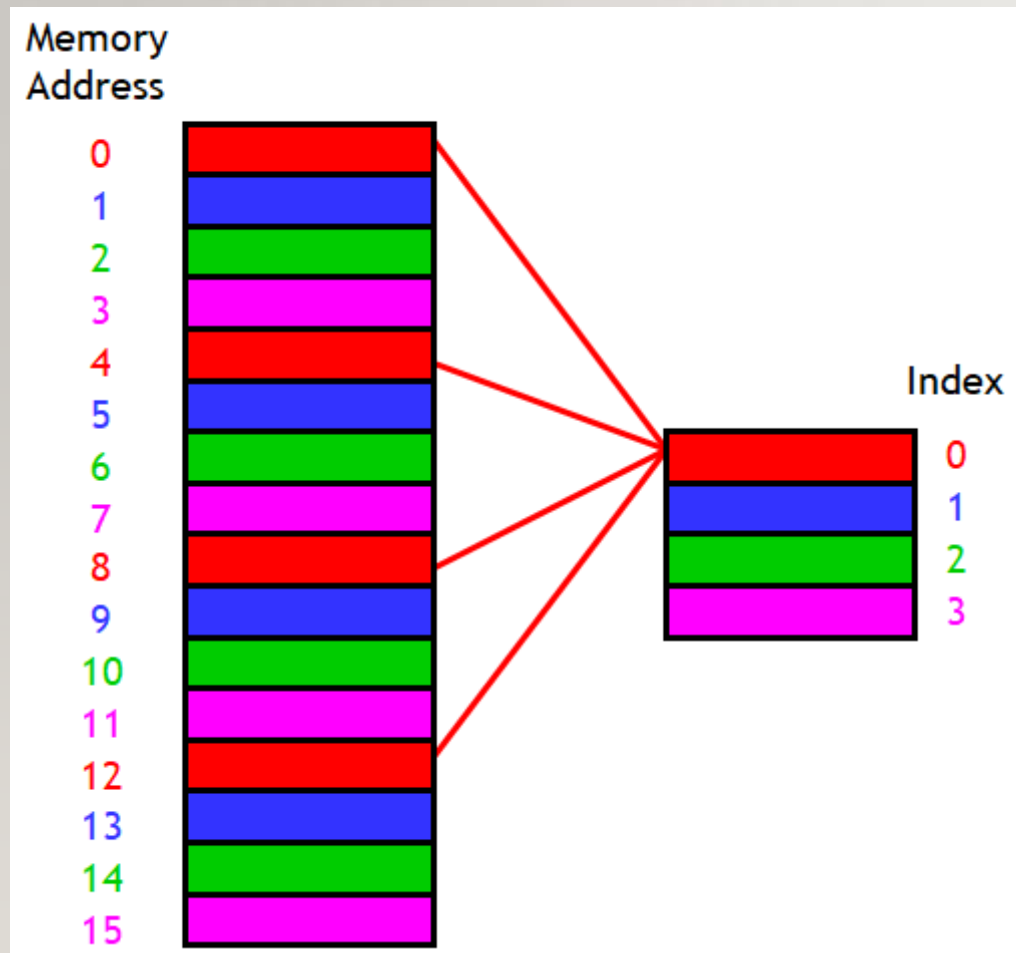
REPLACEMENT ALGORITHM, WRITE POLICY, LINE SIZE & NUMBER OF CACHES (READING ASSIGNMENT)

Cache architectures can be classified according to key elements:

Cache Addresses	Write Policy
Logical	Write through
Physical	Write back
Cache Size	Line Size
Mapping Function	Number of Caches
Direct	Single or two level
Associative	Unified or split
Set associative	
Replacement Algorithm	
Least recently used (LRU)	
First in first out (FIFO)	
Least frequently used (LFU)	
Random	

Figure: Elements of cache design (Source: (Stallings, 2015))

202



203

INTERNAL MEMORY

ADERAW S. NOV-2025



204 INTRODUCTION

- Previous chapter discussed memory and its characteristics.
- This chapter presents:
 - semiconductor main memory subsystems.
 - **ROM**;
 - **DRAM**;
 - **SRAM** memories

205 INTRODUCTION

- Remember these guys?



- Serial memory takes different **lengths of time** to access information:
 - depends on where the desired **location** is relative to the **current position**;

206 SEMICONDUCTOR MAIN MEMORY

- Memory is a collection of cells with the following properties
 - They exhibit **two stable states**, used to represent binary 1 and 0;
 - They are capable of being **written into** (at least once), to **set** the state;
 - They are capable of being **read** to **sense** the state;
 - Access time is the **same** regardless of the location;

207 SEMICONDUCTOR MAIN MEMORY

- Cell has **three** terminals capable of carrying an **electrical signal**:
 - **select**: selects a memory cell for a read / write operation;
 - **control**: indicates whether a read / write operation is being performed;
 - **read/write**:
 - For writing, terminal sets the state of the cell to 1 or 0.
 - For reading, terminal is used for output of the cell's state.



Figure: Memory cell write operation (Source: (Stallings, 2015))

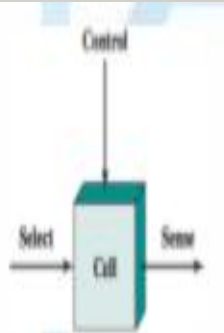


Figure: Memory cell read operation (Source: (Stallings, 2015))

208 SEMICONDUCTOR MAIN MEMORY

- Consider, a memory with a capacity of **1K words of 16 bits** each:
 - Capacity:
 - $1K \times 16 = 16Kbits = 2Kbytes$;
 - Each word has an address:
 - 0 to 1023
 - When a word is **read** or **written**:
 - memory acts on all **16 bits**;
- Based on these concepts how do you think a write operation works? Any ideas?

Memory Address		Memory Contents
Binary	Decimal	
000000000	0	10110101 01011100
000000001	1	10101011 10001001
000000010	2	00001101 01000110
.	.	.
.	.	.
.	.	.
.	.	.
.	.	.
111111101	1021	10011101 00010101
111111110	1022	00001101 00011110
111111111	1023	11011110 00100100

209 SEMICONDUCTOR MAIN MEMORY

- **Write** operation steps:
 - **Apply** the **binary address** of the desired **word** to the **address lines**.
 - **Apply** the **data bits** that must be **stored** in memory to the **data input lines**.
 - **Activate** the **Write control line**.
 - **Memory unit** will then transfer the bits to **that address**.

210 SEMICONDUCTOR MAIN MEMORY

- Read operation steps:
 - Apply the **binary address** of the desired word to the **address lines**.
 - Activate the **Read** control line.
 - **Memory unit** will then **transfer** the bits to the **data output lines**.

211 TIMING WAVEFORMS

- These **memory unit operation** are **controlled** by an **external device** :
- CPU provides the **control signals**
 - **Control signals** are employed for memory **read / write** and takes some time.
 - **Access time** - time required for specifying an **address** and obtaining the **word**;
 - **Write time** - time required for specifying an **address** and storing the **word**;
- These **control signals** are synchronized with **its clock**:
- This implies that:
 - **read / write operations** will take a certain number of **clock periods**;
 - *Lets see with an example..*

212 EXAMPLE (1/7)

- **Assume:**
 - **CPU** with a clock frequency of **50 Mhz**;
 - Memory **access** time: 65ns;
 - Memory **write** time: 75ns
 - How many clock pulses do we need for read / write operations?

213 EXAMPLE (2/7)

- CPU with a clock frequency of 50 Mhz:
- How long does one clock pulse take? Any Ideas?
- $f = 1/p \Leftrightarrow$
- $p = 1/f \Leftrightarrow$
- $p = 1/50\text{Mhz} \Leftrightarrow$
- $p = 1 / 50 \times 10^6 \text{hz} \Leftrightarrow$
- $p = 2 \times 10^{-8} \text{s} \Leftrightarrow 20\text{ns}$

214 EXAMPLE (3/7)

- CPU clock pulse: **20ns**;
- Assumed:
 - Memory access time: **65ns**;
 - Memory write time: **75ns**
- How many **clock pulses** do we need for **read / write** operations?
 - **4 clock pulses** for read / write operations;

EXAMPLE (4/7)

3/16/2026

215

For the **write operation**:

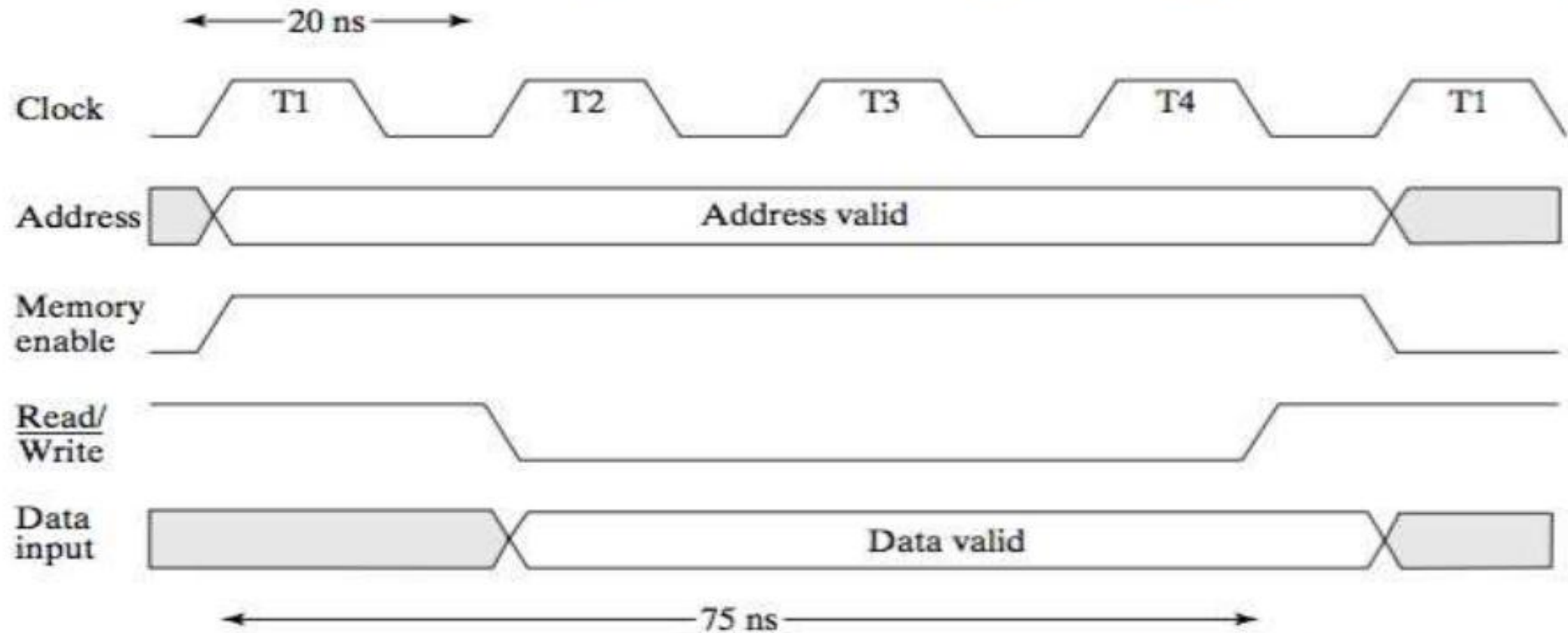


Figure: Write cycle (Source: (Mano and Kime, 2007))

216 EXAMPLE (5/7)

- For the write operation:
 - 1 CPU must provide the address (T1 pulse);
 - 2 Memory enable is set (T1 pulse);
 - 3 Data is supplied (T2 pulse);
 - 4 Read/Write signal set to 0 for write operation (T2 pulse):
 - CPU waits for T2 to let the **address signals stabilize**:
 - Signal must stay activated long enough for the operation to finish;
 - Otherwise: wrong address may be used!
 - 5 When T4 completes, write operation has ended with 5ns to spare

EXAMPLE (6/7)

3/16/2026

217

For the **read operation**:

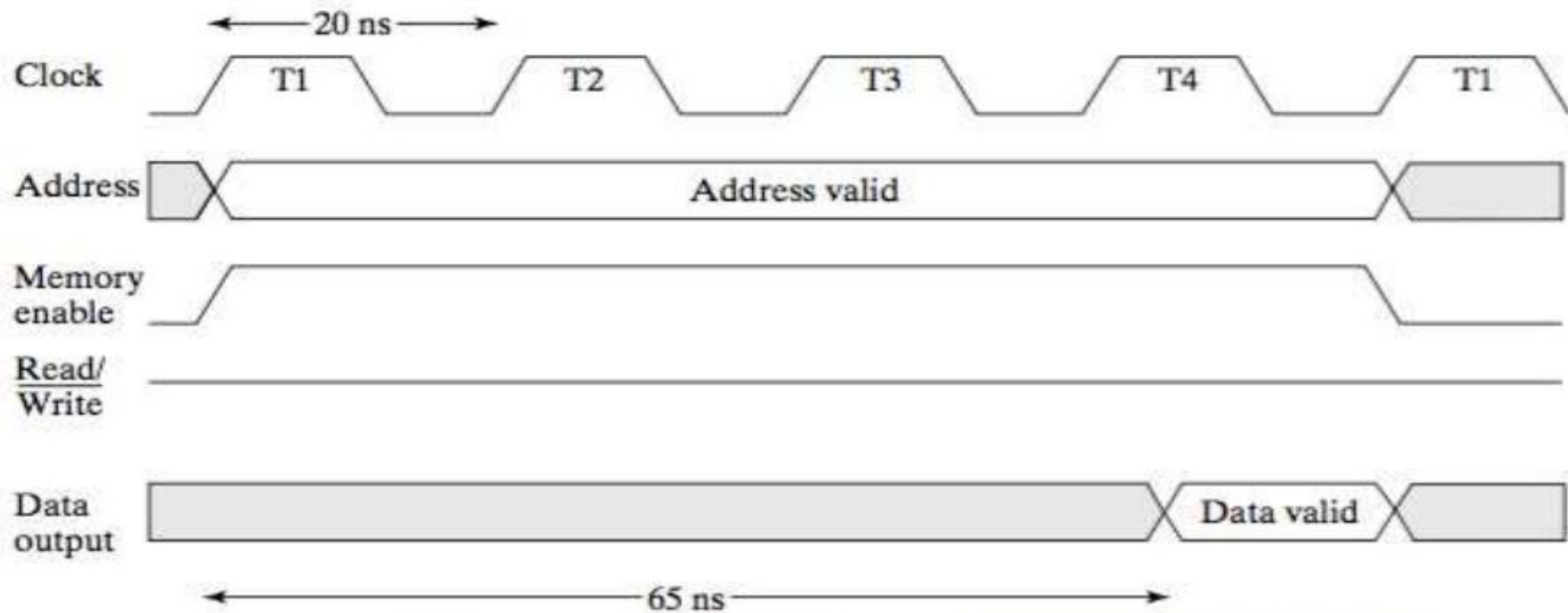


Figure: Read cycle (Source: (Mano and Kime, 2007))

218 EXAMPLE (7/7)

- For the read operation:
- 1 CPU must provide the **address** (T1 pulse);
- 2 Memory enable is set (T1 pulse);
- 3 Read/Write signal set to 1 for **read operation** (T2 pulse):
 - Signal must stay activated long enough for the operation to finish;
 - Selected word is placed onto the **data output lines**;
- 4 When T4 completes, **read** operation has ended with **15ns** to spare

219 MAJOR TYPES OF SEMICONDUCTOR MEMORY

LET'S HAVE A LOOK AT THE DIFFERENT TYPES OF MEMORY TECHNOLOGIES:

Memory Type	Category	Erasure	Write Mechanism	Volatility
Random-access memory (RAM)	Read-write memory	Electrically, byte-level	Electrically	Volatile
Read-only memory (ROM)	Read-only memory	Not possible	Masks	Nonvolatile
Programmable ROM (PROM)				
Erasable PROM (EPROM)	Read-mostly memory	UV light, chip-level	Electrically	
Electrically Erasable PROM (EEPROM)		Electrically, byte-level		
Flash memory		Electrically, block-level		

Figure: Semiconductor memory types (Source: (Stallings, 2015))

220 RANDOM-ACCESS MEMORY

- **Randomly access** any possible address;
- Possible both to **read** and **write** data;
- **Volatile**: requires a **power supply**;
- For a chip with **m words** with **n bits** per **word**:
 - Consists of an array with **m × n** binary storage cells
- E.g.: **DRAM** and **SRAM**.
 - Lets have a look at these

22 | DYNAMIC RAM (DRAM)

- Made with cells that store data as charge on **capacitors**:
- Capacitor **charge presence** or **absence** is interpreted as a **binary 1** or **0**;
- **Capacitors** have a natural tendency to **discharge**;
- Requires **periodic charge** refreshing to maintain **data storage**

222 DRAM CELL COMPONENTS:

- If voltage goes to the address line:
 - Transistor closes:
 - Current flows to capacitor;
- If no voltage goes to the address line:
 - Transistor opens:
 - No current flows to capacitor
- *Based on the DRAM cell components:*
- *How do you think a write operation works?*

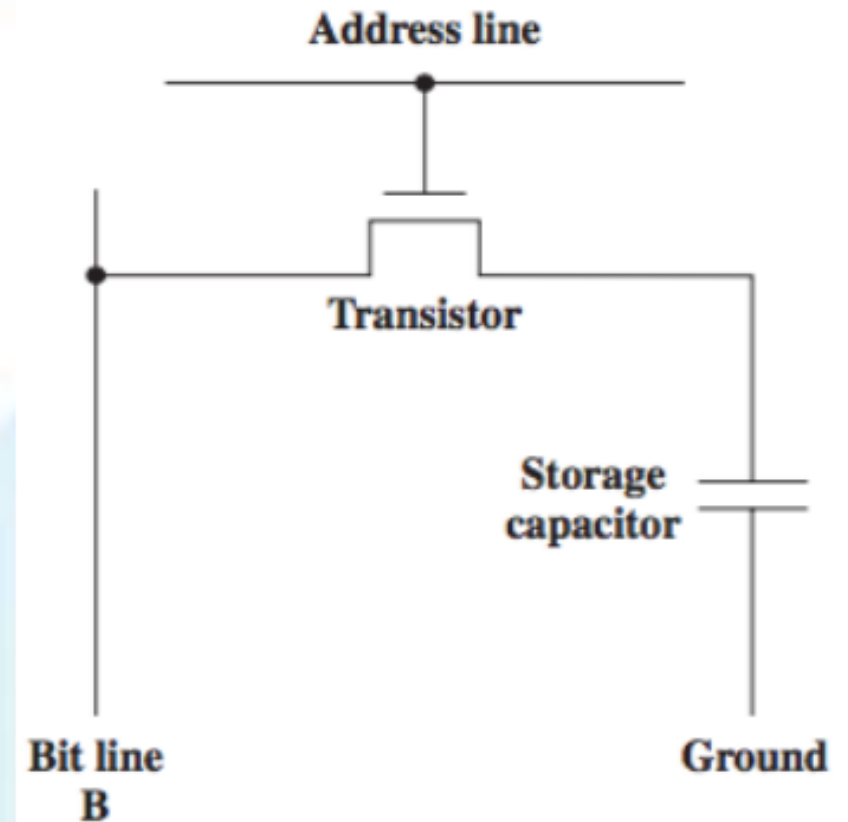


Figure: DRAM Cell (Source: (Stallings, 2015))

223 DYNAMIC RAM (DRAM)

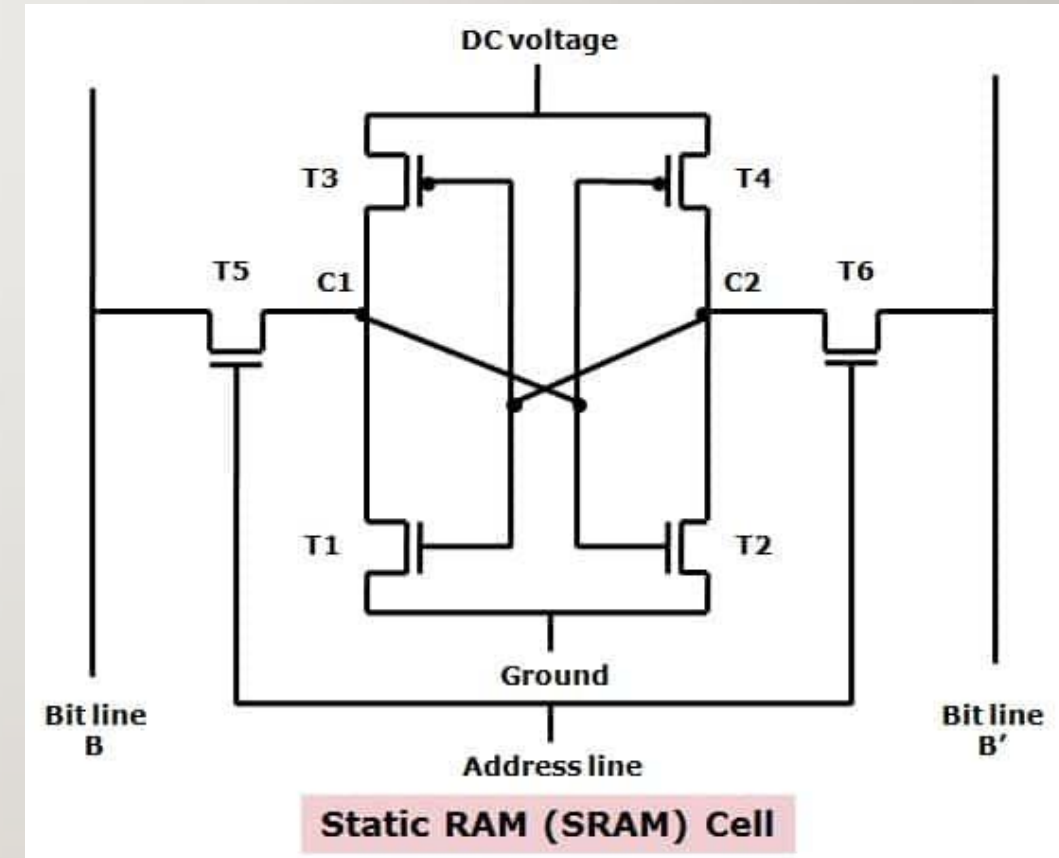
- Write operation:
 - 1 Voltage signal is applied to the **bit line**
 - Low voltage = 0;
 - High voltage = 1;
 - 2 A signal is then applied to the address line:
 - transistor closes...
 - ...charge goes to the capacitor
- *How do you think a read operation works?*

224 DYNAMIC RAM (DRAM)

- Read operation:
 - 1 Address line is activated
 - 2 Transistor turns on
 - 3 Capacitor charge goes to bit line...;
 - ...Low voltage = 0
 - ...High voltage = 1
 - 4 Cell readout discharges the capacitor:
 - **state** must be **restored**;
- *Do you have any idea of other type of technology that can be employed?*

225 STATIC RAM (SRAM)

- Binary values are stored using SR **flip-flop** configurations:
 - Remember flip-flops?
 - Same logic elements used in the **processor registers**;
 - will hold its data as long as **power** is **supplied** to it.



226 HOW TO CHOOSE BETWEEN DRAM AND SRAM?

- What are the pros / cons of DRAM?
- What are the pros / cons of SRAM?

227 DRAM VS SRAM

- Both are **volatile**:
 - power must be continuously **supplied** to preserve the bit values;
- DRAMs:
 - Periodically **refresh** capacitor's **charge**;
- SRAM:
 - No need to **periodically** refresh;

228 DRAM VS SRAM

- DRAM cell is **simpler** and **smaller** than a SRAM cell:
 - (1 transistor, 1 capacitor) vs. SR flip-flop
 - DRAM is **denser** (smaller cells = more cells per unit area) ;
 - Due to its **simplicity**:
 - DRAM is **cheaper** than **corresponding** SRAM;

229 DRAM VS SRAM

- DRAM requires the supporting **refresh** circuitry (readout):
 - This is a fixed cost, does not increase with size
 - This cost is more than compensated by the smaller cost of **DRAM** cells;
 - Thus, DRAMs tend to be favored for **large memory requirements**;

230 DRAM VS SRAM

- SRAMs are **faster** but more **expensive** than DRAMs
 - No need to **refresh** the circuitry after a **readout**;
- Because of these characteristics:
 - **DRAM**: used in **main memory**;
 - **SRAM**: used in **cache and registers**;

23 | DRAM VS SRAM

Memory technology	Typical access time	\$ per GiB in 2012
SRAM semiconductor memory	0.5–2.5 ns	\$500–\$1000
DRAM semiconductor memory	50–70 ns	\$10–\$20
Flash semiconductor memory	5,000–50,000 ns	\$0.75–\$1.00
Magnetic disk	5,000,000–20,000,000 ns	\$0.05–\$0.10

Figure: Access time and price per bit (Source: (Patterson and Hennessy, 2013))

232 READ-ONLY MEMORY (ROM)

- Contains a **permanent** pattern of data that cannot be **changed**;
 - Therefore:
 - Possible to **read** a **ROM**;
 - Not possible to **write** data **more than once**;
 - Advantage:
 - Store Data / Program is **permanently**;
 - No need to load from a **secondary storage device**;
 - Original BIOS were stored in ROM chips.
 - **Nonvolatile**: no power source is required;

233 READ-ONLY MEMORY (ROM)

- Not possible to **write** data **more than once**:
 - Data is **wired** into the **chip** as part of the **fabrication process**
 - Data insertion is **expensive**;
 - No room for **error**:
 - If one bit is **wrong**, the **whole batch of ROMs** must be thrown out.

234 READ-ONLY MEMORY (ROM)

- Different types of **ROM** (1/4):
 - **Programmable ROM (PROM):**
 - **nonvolatile** and may be written **into only once**;
 - Writing may be performed at a later time than the original chip fabrication.
 - Special equipment is required for the **writing process**;
 - More expensive than **ROM**;

235 READ-ONLY MEMORY (ROM)

- Different types of ROM (2/4):
 - Erasable programmable read-only memory (**EPROM**):
 - Can be **altered** multiple times;
 - **Entire memory contents** need to be **erased**;
 - Special equipment is required for the erasure process:
 - **Ultra-violet radiation**;
 - Slow **erasure** procedure (>20 minutes);
 - More expensive than **PROM**;

236 READ-ONLY MEMORY (ROM)

- Different types of ROM (3/4):
 - Electrically erasable programmable read-only memory (**EEPROM**):
 - Can be altered **multiple** times;
 - Special equipment is required for the **erasure process**:
 - **No** need to erase **prior contents**...
 - ...only the **byte** or **bytes** addressed are updated.
 - Electrically erasable;
 - **Faster** erasure procedure than **EPROM**;
 - More expensive than **EPROM**;

237 READ-ONLY MEMORY (ROM)

- Different types of ROM (4/4):
 - Flash memory:
 - named because of the **speed** with which it can be reprogrammed;
 - between **EPROM** and **EEPROM** in both **cost** and **functionality**:
 - Uses an **electrical** erasing technology;
 - Possible to erase **blocks** of memory;
 - Does not provide **byte-level** erasure;
 - Technology used in **pen** drives;

238 ERROR CORRECTION

- Semiconductor memory systems are subject to errors (1/2):
- **Hard Failures: Permanent** physical defect;
 - Damaged memory cell(s) cannot **reliably** store data:
 - become stuck at **0 or 1** or...
 - ...**switch erratically** between 0 and 1
 - Caused by:
 - Harsh environmental abuse;
 - Manufacturing defects;
 - Wear and tear;

239 ERROR CORRECTION

- Semiconductor memory systems are subject to errors (2/2):
 - Soft Failures: **Random, nondestructive** event
 - Alters the contents of one or more memory cells
 - Without **damaging** the memory.
 - Caused by:
 - **Power supply problems;**
 - Alpha particles (**radioactive** decay).
 - **Hard and soft errors are clearly undesirable...**So what can we do to address them?

240 ERROR CORRECTION

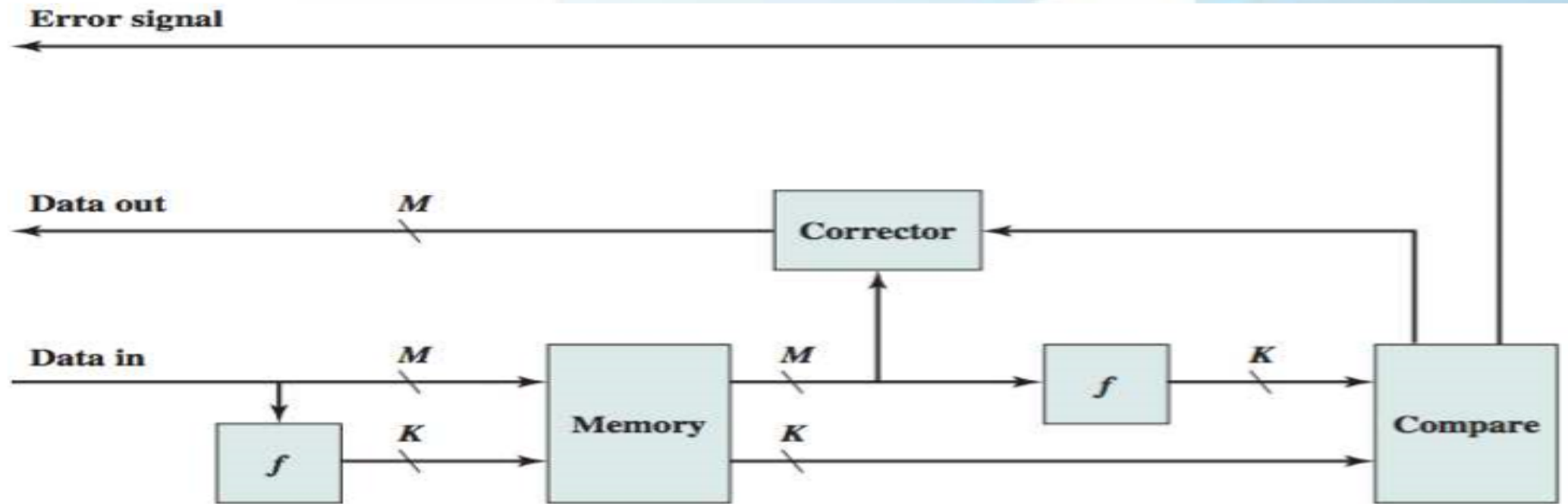


Figure: Error correcting code function (Source: (Stallings, 2015))

241 ERROR CORRECTION

- From the previous picture (1/3):
 - 1 M-bit data is to be written into memory;
 - 2 Computation $f(M)$ is performed on the data (K check bits);
 - 3 Both the data and the code ($M + K$ bits) are stored in memory;

242 ERROR CORRECTION

- From the previous picture (2/3):
 - 4 Eventually, the stored word will be read out;
 - 5 Computation $f(M)$ is again performed on the data (K check bits);
 - 6 A comparison is made between both check bits;

243 ERROR CORRECTION

- From the previous picture (3/3):
 - 7 The comparison yields one of three results:
 - **No errors** are detected:
 - Fetched data bits are **sent out**;
 - An **error is detected**, and it is **possible** to correct the error:
 - Data bits plus error correction bits are fed into a **corrector**
 - Producing a **corrected** set of **M** bits to be sent out.
 - An **error is detected**, but it is **not possible** to correct it:
 - This condition is **reported**.

244 READING ASSIGNMENTS

- Error-correcting codes